

Block 2: Knowledge Representation

Short explanations, worked examples and self-check questions for every unit, with every idea also explained in Malayalam.

ഓരോ ആശയവും ലളിതമായ ഇംഗ്ലീഷിലും മലയാളത്തിലും, ഉദാഹരണങ്ങളോടെ.

UNIT 4

Predicate and Propositional Logic

ഈ unit-ൽ propositions, logical connectives, truth tables, rules of inference, laws of logic എന്നിവ പഠിച്ച്, predicate logic-ലെ quantifiers ($\forall$, $\exists$) വരെ എത്തുന്നു. അവസാനം CNF/DNF conversion, natural deduction, resolution refutation എന്നിവ ഉപയോഗിച്ച് ഒരു conclusion ശരിയാണെന്ന് എങ്ങനെ prove ചെയ്യാമെന്ന് നോക്കുന്നു.

Propositions

Atomic and compound propositions

Logical connectives

Truth tables of the connectives

Precedence and well-formed formulas (wff)

Truth tables with three or more variables

Propositional rules of inference

Rules of replacement (laws of logic)

Validity and satisfiability

Introduction to predicate logic

Quantifiers and their negation

Translating English into logic

Inferencing in predicate logic

Proof systems

Natural deduction

Normal forms: CNF and DNF

Clausal form and the resolution rule

Resolution refutation (unsatisfiability)

Propositions

Definition: A proposition is a declarative sentence that is either true or false, but not both.

ശരിയോ തെറ്റോ ആയ, എന്നാൽ രണ്ടും ഒരുമിച്ച് ആകാത്ത, ഒരു declarative sentence ആണ് proposition.

Symbolic logic (developed seriously by George Boole) represents facts and relationships by symbols and then manipulates the symbols, not the original sentences, to draw conclusions. Its building blocks are **propositions**:

declarative sentences that are either true or false, but not both. Questions, commands, requests, exclamations, opinions and sentences with unknown variables (like " $a + 5 = b$ ") are not propositions.

മലയാളത്തിൽ

Proposition എന്നാൽ "ശരിയോ തെറ്റോ" എന്ന് ഉറപ്പിച്ച് പറയാൻ പറ്റുന്ന ഒരു declarative sentence. "Delhi is the capital of India" ശരി, "15 is a prime number" തെറ്റ്; രണ്ടും propositions ആണ്, കാരണം ഉത്തരം ഉറപ്പാണ്. പക്ഷേ "ചായ കുടിച്ചോ?" എന്ന ചോദ്യമോ, "വായിൽ അടയ്ക്കൂ" എന്ന ആജ്ഞയോ, "Mumbai is a good city" എന്ന അഭിപ്രായമോ proposition അല്ല. Proposition ആകാൻ അത് തെറ്റാണെങ്കിലും കഴിമില്ല, ശരിയോ തെറ്റോ എന്ന് തീരുമാനിക്കാൻ പറ്റണം എന്നതാണ് പ്രധാനം.

Proposition: a declarative sentence with exactly one truth value, True (T, 1) or False (F, 0)

Propositional logic is also called binary logic (only two values)

Tautology: a proposition that is always true

Contradiction: a proposition that is always false

Truth table: a table listing the truth value of a proposition for every combination of its parts

EXAMPLE 1

Which are propositions? (a) Water freezes at 100° Celsius. (b) 2 is the only even prime number. (c) p is less than 5. (d) Let's play football in the evening. (e) Who is the Prime Minister of India? (f) Women are more intelligent than men.

(a) Yes, it is a false proposition (water freezes at 0°C). (b) Yes, true. (c) No, its truth depends on the unknown p . (d) No, it is a suggestion. (e) No, it is a question. (f) No, it is an opinion: it is neither universally true nor universally false.

Exam tip: A false statement is still a proposition. " $1 + 2 = 4$ " is a proposition (false). Do not answer "No" just because the sentence is wrong.

Atomic and compound propositions

Atomic proposition: A simple proposition that cannot be broken into smaller propositions; it is denoted by a single symbol such as P.

Atomic proposition: ചെറിയ propositions ആയി പിരിക്കാൻ കഴിയാത്ത ലളിതമായ proposition; P പോലെ ഒരൊറ്റ symbol കൊണ്ട് ഇതിനെ സൂചിപ്പിക്കുന്നു.

Compound proposition: A proposition formed by combining atomic propositions with logical connectives.

Compound proposition: Atomic propositions-നെ logical connectives ഉപയോഗിച്ച് ചേർത്ത് ഉണ്ടാക്കുന്ന proposition.

An **atomic proposition** is the simplest kind: a single statement, represented by one symbol such as P, Q or R. A **compound proposition** is formed by joining atomic propositions with logical connectives (and, or, not, if...then, if and only if). The symbols P, Q, R used for propositions are called **atoms** or atomic formulas.

മലയാളത്തിൽ

Atomic proposition ഒരു ഇഷ്ടിക പോലെയാണ്; അതിനെ വീണ്ടും ചെറിയ statements ആയി പിരിക്കാൻ പറ്റില്ല. " $7 - 3 = 4$ " ഒരു atomic proposition ആണ്. രണ്ടോ അതിലധികമോ ഇഷ്ടികകൾ "and", "or" പോലുള്ള connectives എന്ന സിമന്റ് കൊണ്ട് ചേർത്താൽ കിട്ടുന്നത് compound proposition. "The Sun is very bright today, and it's very hot outside" എന്നതിൽ രണ്ട് atomic statements "and" കൊണ്ട് ചേർന്നിരിക്കുന്നു.

Atomic: P : The sun rises in the west (F), Q : Sugar is sweet (T)

Compound: $P \wedge Q$, $P \vee \neg Q$, $P \rightarrow Q$, ...

EXAMPLE

Atomic or compound? (a) Jack either plays cricket or football. (b) Dermatology is the branch of medical science which deals with the skin. (c) IGNOU is a central university and offers courses through the distance learning mode. (d) Chandigarh is the capital of the Indian states Haryana and Punjab.

(a) Compound: "Jack plays cricket" $\vee$ "Jack plays football". (b) Atomic: one single fact. (c) Compound: two facts joined by "and". (d) Compound: "Chandigarh is the capital of Haryana" $\wedge$ "Chandigarh is the capital of Punjab".

Exam tip: Look for hidden connectives. "A is the capital of X and Y" is two facts, so it is compound, even though "and" joins nouns, not sentences.

Logical connectives

Definition: A logical connective is an operator ($\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$) that joins propositions so that the truth value of the result depends only on the truth values of its parts.

Propositions-നെ ചേർക്കുന്ന ഒരു operator ($\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$) ആണ് logical connective; ഫലത്തിന്റെ truth value അതിന്റെ ഭാഗങ്ങളുടെ truth values-നെ മാത്രം ആശ്രയിച്ചിരിക്കുന്നു.

Logical connectives (logical operators) join propositions so that the truth value of the result depends only on the truth values of the parts. There are five main ones. In $A \rightarrow B$, A is the **hypothesis** (antecedent) and B the **conclusion** (consequent). Implication only promises something when A is true: "If Mary scores well, I will buy her a phone" is not broken if she scores badly and still gets a phone. IGNOU writes negation as $\sim A$; in this guide we use $\neg A$ (same meaning).

മലയാളത്തിൽ

Connectives എന്നത് propositions-നെ ചേർക്കുന്ന "glue" ആണ്. $\wedge$ (AND) രണ്ടും ശരിയാകുമ്പോൾ മാത്രം ശരി, $\vee$ (OR) ഒന്നെങ്കിലും ശരിയായാൽ ശരി, $\neg$ (NOT) ഉത്തരം മറിച്ചിടുന്നു. $\rightarrow$ (implication) ഒരു വാഗ്ദാനം പോലെയാണ്: "90% കിട്ടിയാൽ ഫോൺ വാങ്ങിത്തരാം" എന്ന് അച്ഛൻ പറഞ്ഞാൽ, 90% കിട്ടിയിട്ടും ഫോൺ കിട്ടിയില്ലെങ്കിൽ മാത്രമാണ് വാഗ്ദാനം ലംഘിക്കുന്നത്. $\leftrightarrow$ (biconditional) രണ്ടും ഒരേ value ആകുമ്പോൾ ശരി.

Conjunction: $A \wedge B$, "A and B"

Disjunction (inclusive or): $A \vee B$, "A or B"

Exclusive or: $A \oplus B$, "either A or B, but not both"

Negation: $\neg A$ (IGNOU: $\sim A$), "not A"

Implication: $A \rightarrow B$, "if A then B", "A implies B", "A is sufficient for B", "A only if B", "B is necessary for A"

Biconditional: $A \leftrightarrow B$, "A if and only if B", same as $(A \rightarrow B) \wedge (B \rightarrow A)$

For $A \rightarrow B$: converse $B \rightarrow A$, inverse $\neg A \rightarrow \neg B$, contrapositive $\neg B \rightarrow \neg A$

EXAMPLE 1

Let A = "You score above 90%", B = "You will get a mobile phone". Write $A \wedge B$, $A \vee B$, $\neg A$, $A \rightarrow B$, $A \leftrightarrow B$ in English.

$A \wedge B$: You score above 90% and you will get a mobile phone.

$A \vee B$: You score above 90% or you will get a mobile phone.

$\neg A$: You do not score above 90%.

$A \rightarrow B$: If you score above 90%, you will get a mobile phone.

$A \leftrightarrow B$: You will get a mobile phone if and only if you score above 90%.

EXAMPLE 2

Write the converse, inverse and contrapositive of "If it rains, the ground is wet" ($R \rightarrow W$).

Converse $W \rightarrow R$: If the ground is wet, it rained.

Inverse $\neg R \rightarrow \neg W$: If it does not rain, the ground is not wet.

Contrapositive $\neg W \rightarrow \neg R$: If the ground is not wet, it did not rain.

Only the contrapositive is equivalent to the original (this is the Transposition rule of Topic 8). The converse and inverse are not (the ground can be wet because someone watered it).

Exam tip: "A only if B" is $A \rightarrow B$, not $B \rightarrow A$. "Kids go to the park only if it is not hot" means: going to the park $\rightarrow$ not hot.

Truth tables of the connectives

Definition: A truth table is a table that lists the truth value of a formula for every possible combination of truth values of its atoms (2^n rows for n atoms).

ഒരു formula-യിലെ atoms-ന്റെ സാധ്യമായ എല്ലാ truth value combinations-നും ആ formula-യുടെ truth value കാണിക്കുന്ന പട്ടികയാണ് truth table (n atoms-ന് 2^n rows).

The meaning (semantics) of each connective is fixed by its truth table. With n atoms there are 2^n rows. The key facts: $\wedge$ is true only when both are true; $\vee$ is false only when both are false; $\rightarrow$ is false only when the hypothesis is true and the conclusion false; $\leftrightarrow$ is true when both sides have the same value; $\oplus$ is true when they differ.

മലയാളത്തിൽ

Truth table എന്നത് എല്ലാ സാധ്യതകളും എഴുതി നോക്കുന്ന ഒരു പട്ടിക. രണ്ട് variables ഉണ്ടെങ്കിൽ $2^2 = 4$ rows. $\rightarrow$ (implication)-ന്റെ table ആണ് കുട്ടികൾക്ക് ഏറ്റവും കൺഫ്യൂഷൻ: " $T \rightarrow F$ " മാത്രം F ആണ്, ബാക്കി മൂന്ന് rows-ഉം T. Hypothesis തെറ്റാണെങ്കിൽ വാഗ്ദാനം ലംഘിക്കപ്പെടുന്നില്ല, അതുകൊണ്ട് $F \rightarrow$ എന്തും T ആണ്. ഈ table മനപ്പാഠം ആക്കിയാൽ ബാക്കി എല്ലാം എളുപ്പം.

A	$\neg A$
F	T
T	F

A	B	$A \wedge B$	$A \vee B$	$A \oplus B$	$A \rightarrow B$	$A \leftrightarrow B$
F	F	F	F	F	T	T
F	T	F	T	T	T	F
T	F	F	T	T	F	F
T	T	T	T	F	T	T

$\neg A$ is T $\Leftrightarrow$ A is F

$A \wedge B$ is T $\Leftrightarrow$ both A, B are T

$A \vee B$ is F $\Leftrightarrow$ both A, B are F

$A \rightarrow B$ is F $\Leftrightarrow$ A is T and B is F

$A \leftrightarrow B$ is T $\Leftrightarrow$ A and B have the same truth value

EXAMPLE

P : The sun rises in the west (F), Q : Sugar is sweet (T). Find the truth values of $P \wedge Q$, $P \vee Q$, $P \rightarrow Q$, $Q \rightarrow P$, $P \leftrightarrow Q$.

$P \wedge Q = F \wedge T = F$. $P \vee Q = F \vee T = T$. $P \rightarrow Q = F \rightarrow T = T$. $Q \rightarrow P = T \rightarrow F = F$. $P \leftrightarrow Q$: values differ, so F.

Exam tip: Always list rows in a fixed order (FF, FT, TF, TT, like binary counting 00, 01, 10, 11). Examiners check columns quickly; a fixed order avoids missing a row.

Precedence and well-formed formulas (wff)

Well-formed formula (wff): Every atom is a wff; if A and B are wffs, then $(\neg A)$, $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$ and $(A \leftrightarrow B)$ are wffs; nothing else is a wff.

Well-formed formula (wff): ഓരോ atom-ഉം ഒരു wff ആണ്; A, B എന്നിവ wffs ആണെങ്കിൽ $(\neg A)$, $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$, $(A \leftrightarrow B)$ എന്നിവയും wffs ആണ്; ഇവയല്ലാതെ മറ്റൊന്നും wff അല്ല.

Precedence: The order in which connectives are applied when brackets are omitted: first $\neg$, then $\wedge$, then $\vee$, then $\rightarrow$ and $\leftrightarrow$.

Precedence: Brackets ഇല്ലാത്തപ്പോൾ connectives പ്രയോഗിക്കുന്ന ക്രമം: ആദ്യം $\neg$, പിന്നെ $\wedge$, പിന്നെ $\vee$, അവസാനം $\rightarrow$, $\leftrightarrow$.

A well-formed

formula (wff) is a legal logical expression, defined recursively: (1) an atom is a wff; (2) if A is a wff, $(\neg A)$ is a wff; (3) if A and B are wffs, then $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$, $(A \leftrightarrow B)$ are wffs; (4) nothing else is a wff. To drop brackets we use **precedence**: $\neg$ is applied first, then $\wedge$, then $\vee$ (and $\oplus$), then $\rightarrow$ and $\leftrightarrow$. Between two connectives of the same level, the left-most is applied first.

മലയാളത്തിൽ

Wff എന്നത് logic-ന്റെ "grammar" അനുസരിച്ച് ശരിയായി എഴുതിയ formula. $(P \rightarrow (Q \wedge))$ പോലെ ഒരു ഭാഗം കാണാതായാൽ അത് wff അല്ല. Maths-ൽ BODMAS പോലെ logic-നും ഒരു order ഉണ്ട്: ആദ്യം $\neg$, പിന്നെ $\wedge$, പിന്നെ $\vee$, അവസാനം $\rightarrow$, $\leftrightarrow$. Brackets ഇല്ലാത്ത formula കിട്ടിയാൽ ഈ order വെച്ച് brackets ഇട്ട് നോക്കിയാൽ എളുപ്പമാകും.

Precedence (highest to lowest): $\neg > \wedge > \vee, \oplus > \rightarrow, \leftrightarrow$

IGNOU's alternative list (increasing priority): $\leftrightarrow, \rightarrow, \vee, \wedge, \neg$ ($\leftrightarrow$ lowest)

EXAMPLE 1

Fully parenthesise $P \rightarrow Q \wedge \neg R$.

Step 1: $\neg$ has the highest priority: $P \rightarrow Q \wedge (\neg R)$.

Step 2: $\wedge$ is above $\rightarrow$: $P \rightarrow (Q \wedge (\neg R))$.

Step 3: only $\rightarrow$ is left: $(P \rightarrow (Q \wedge (\neg R)))$.

EXAMPLE 2

Show that $((P \rightarrow (Q \wedge (\neg R))) \leftrightarrow S)$ is a wff, and say why $(P \rightarrow (Q \wedge))$ and $(P (Q \wedge R))$ are not.

P, Q, R, S are atoms, so wffs (rule 1). $(\neg R)$ is a wff (rule 2). $(Q \wedge (\neg R))$ is a wff (rule 3). $(P \rightarrow (Q \wedge (\neg R)))$ is a wff (rule 3). Joining it with S by $\leftrightarrow$ gives a wff (rule 3).

$(P \rightarrow (Q \wedge))$: $\wedge$ needs two wffs, but its right side is empty. $(P (Q \wedge R))$: two wffs are placed side by side with no connective. Neither can be built by rules 1-3.

Exam tip: IGNOU's two precedence lists disagree slightly on $\rightarrow$ versus $\leftrightarrow$. In your answers, always put brackets around $\rightarrow$ and $\leftrightarrow$ parts so there is no ambiguity.

Truth tables with three or more variables

For a compound formula, build the truth table column by column, from the innermost sub-formula outwards.

With 3 variables you need 8 rows; with 4, 16 rows. The unit's table shows that $\wedge$ and $\vee$ are associative: $(\alpha_1 \wedge \alpha_2) \wedge \alpha_3$ and $\alpha_1 \wedge (\alpha_2 \wedge \alpha_3)$ have the same column.

മലയാളത്തിൽ

വലിയ formula കണ്ടാൽ പേടിക്കേണ്ട. ഉള്ളി തൊലിക്കുന്നതുപോലെ, ഉള്ളിലെ bracket ആദ്യം ഒരു column ആക്കുക, പിന്നെ അടുത്ത layer, ഒടുവിൽ മുഴുവൻ formula. മൂന്ന് variables ആണെങ്കിൽ 8 rows: ആദ്യ column-ൽ 4 F, 4 T; രണ്ടാമത്തേതിൽ 2 F, 2 T മാറിമാറി; മൂന്നാമത്തേതിൽ F, T മാറിമാറി.

α_1	α_2	α_3	$\alpha_1 \wedge \alpha_2$	$\alpha_2 \wedge \alpha_3$	$\alpha_1 \vee \alpha_2$	$\alpha_2 \vee \alpha_3$	$\alpha_1 \wedge \alpha_2 \wedge \alpha_3$	$\alpha_1 \vee \alpha_2 \vee \alpha_3$
F	F	F	F	F	F	F	F	F
F	F	T	F	F	F	T	F	T
F	T	F	F	F	T	T	F	T
F	T	T	F	T	T	T	F	T
T	F	F	F	F	T	F	F	T
T	F	T	F	F	T	T	F	T
T	T	F	T	F	T	T	F	T
T	T	T	T	T	T	T	T	T

EXAMPLE 1 (CHECK YOUR PROGRESS 3, Q1A)

Draw the truth table of $\alpha_2 \leftrightarrow (\neg\alpha_1 \rightarrow (\alpha_1 \vee \alpha_2))$.

α_1	α_2	$\neg\alpha_1$	$\alpha_1 \vee \alpha_2$	$\neg\alpha_1 \rightarrow (\alpha_1 \vee \alpha_2)$	$\alpha_2 \leftrightarrow (...)$
F	F	T	F	F	T
F	T	T	T	T	T
T	F	F	T	T	F
T	T	F	T	T	T

The formula is false only for $\alpha_1 = T, \alpha_2 = F$, so it is a contingency (neither tautology nor contradiction).

EXAMPLE 2 (CHECK YOUR PROGRESS 3, Q1B)

Draw the truth table of $(\neg\alpha_1 \leftrightarrow (\alpha_2 \leftrightarrow \alpha_3)) \vee (\alpha_3 \wedge \alpha_2)$.

α_1	α_2	α_3	$\neg\alpha_1$	$\alpha_2 \leftrightarrow \alpha_3$	$\neg\alpha_1 \leftrightarrow (\alpha_2 \leftrightarrow \alpha_3)$	$\alpha_3 \wedge \alpha_2$	Result
F	F	F	T	T	T	F	T
F	F	T	T	F	F	F	F
F	T	F	T	F	F	F	F
F	T	T	T	T	T	T	T
T	F	F	F	T	F	F	F
T	F	T	F	F	T	F	T
T	T	F	F	F	T	F	T
T	T	T	F	T	F	T	T

True in 5 rows, false in 3: a contingency.

Exam tip: Write one column for every connective, even simple ones like $\neg\alpha_1$. Marks are given for intermediate columns, and they help you avoid mistakes.

Propositional rules of inference

Definition: A rule of inference is a truth-preserving template that derives a new statement (the conclusion) from statements already known to be true (the premises).

ശരിയാണെന്ന് അറിയാവുന്ന statements-ൽ (premises) നിന്ന് ഒരു പുതിയ statement (conclusion) derive ചെയ്യുന്ന, truth നിലനിർത്തുന്ന ഒരു template ആണ് rule of inference.

Rules of inference are templates for deriving new true statements (conclusions) from statements already known to be true (premises). They preserve truth: if the premises are true, the conclusion must be true. Each rule below can be checked by a truth table: (premises joined by $\wedge$) $\rightarrow$ conclusion is a tautology. The unit lists nine rules.

മലയാളത്തിൽ

Rules of inference എന്നത് ഒരു detective-ന്റെ ന്യായവാദ രീതികൾ പോലെയാണ്. "മഴ പെയ്താൽ റോഡ് നനയും; മഴ പെയ്തു; അതുകൊണ്ട് റോഡ് നനഞ്ഞു" എന്നത് Modus Ponens. "മഴ പെയ്താൽ റോഡ് നനയും; റോഡ് നനഞ്ഞിട്ടില്ല; അതുകൊണ്ട് മഴ പെയ്തില്ല" എന്നത് Modus Tollens. Premises ശരിയാണെങ്കിൽ ഈ rules വഴി കിട്ടുന്ന conclusion-ഉം ഉറപ്പായും ശരിയായിരിക്കും.

Rule	Premises	Conclusion
Modus Ponens (MP)	$\alpha_1 \rightarrow \alpha_2, \alpha_1$	α_2
Modus Tollens (MT)	$\alpha_1 \rightarrow \alpha_2, \neg \alpha_2$	$\neg \alpha_1$
Disjunctive Syllogism (DS)	$\alpha_1 \vee \alpha_2, \neg \alpha_1$ (or $\neg \alpha_2$)	α_2 (or α_1)
Addition	α_1	$\alpha_1 \vee \alpha_2$
Simplification	$\alpha_1 \wedge \alpha_2$	α_1 (or α_2)
Conjunction	α_1, α_2	$\alpha_1 \wedge \alpha_2$
Hypothetical Syllogism (HS)	$\alpha_1 \rightarrow \alpha_2, \alpha_2 \rightarrow \alpha_3$	$\alpha_1 \rightarrow \alpha_3$
Absorption	$\alpha_1 \rightarrow \alpha_2$	$\alpha_1 \rightarrow (\alpha_1 \wedge \alpha_2)$
Constructive Dilemma (CD)	$\alpha_1 \vee \alpha_3, (\alpha_1 \rightarrow \alpha_2) \wedge (\alpha_3 \rightarrow \alpha_4)$	$\alpha_2 \vee \alpha_4$

EXAMPLE 1

Prove Modus Ponens is a valid rule by showing $(P \wedge (P \rightarrow Q)) \rightarrow Q$ is a tautology.

P	Q	$P \rightarrow Q$	$P \wedge (P \rightarrow Q)$	$(P \wedge (P \rightarrow Q)) \rightarrow Q$
F	F	T	F	T
F	T	T	F	T
T	F	F	F	T
T	T	T	T	T

The last column is all T, so the formula is a tautology and MP is valid.

EXAMPLE 2

Name the rule used: (a) "If I study, I pass. I did not pass. So I did not study." (b) "Either the fan is broken or the power is off. The power is not off. So the fan is broken." (c) "If it rains, the match is cancelled. If the match is cancelled, I stay home. So if it rains, I stay home."

(a) Modus Tollens. (b) Disjunctive Syllogism. (c) Hypothetical Syllogism.

Exam tip: Two classic fallacies are not rules: " $\alpha_1 \rightarrow \alpha_2, \alpha_2$, so α_1 " (affirming the consequent) and " $\alpha_1 \rightarrow \alpha_2, \neg \alpha_1$, so $\neg \alpha_2$ " (denying the antecedent). Both fail in the row $\alpha_1 = F, \alpha_2 = T$. Note: the unit later says "eight" inference rules, but section 4.7 actually lists nine.

Rules of replacement (laws of logic)

Rule of replacement: A rule that replaces a formula, or any part of it, by a logically equivalent formula.

Rule of replacement: ഒരു formula-യെ, അല്ലെങ്കിൽ അതിന്റെ ഏത് ഭാഗത്തെയും, logically equivalent ആയ മറ്റൊരു formula കൊണ്ട് മാറ്റിവെക്കുന്ന rule.

Logical equivalence: $A \equiv B$ when A and B have the same truth value in every row, i.e. when $A \leftrightarrow B$ is a tautology.

Logical equivalence: Truth table-ലെ ഓരോ row-ലും A-ക്കും B-ക്കും ഒരേ truth value ആണെങ്കിൽ, അതായത് $A \leftrightarrow B$ ഒരു tautology ആണെങ്കിൽ, $A \equiv B$.

A replacement rule lets you swap a formula for a **logically equivalent** one (same truth value in every row). Two formulas A and B are equivalent, written $A \equiv B$, exactly when $A \leftrightarrow B$ is a tautology. These laws are used to

simplify formulas and to convert them to CNF/DNF. Unlike inference rules, replacement rules work in both directions and can be applied to any part of a formula.

മലയാളത്തിൽ

Replacement rules എന്നാൽ ഒരേ അർത്ഥമുള്ള രണ്ട് രൂപങ്ങൾ. "നാളെ ലീവ് ആണ്" എന്നും "നാളെ ഓഫീസ് ഇല്ല" എന്നും പറഞ്ഞാൽ അർത്ഥം ഒന്നുതന്നെ; അതുപോലെ $\neg(A \wedge B)$ എന്നും $\neg A \vee \neg B$ എന്നും എഴുതാം (De Morgan). രണ്ട് formulas equivalent ആണോ എന്ന് അറിയാൻ truth table-ൽ അവയുടെ columns ഓരോ row-ലും ഒന്നാണോ എന്ന് നോക്കിയാൽ മതി.

Rule	Formula	Equivalent
Tautology (idempotence)	$\alpha_1 \wedge \alpha_1; \alpha_1 \vee \alpha_1$	α_1
Double Negation (DN)	$\neg(\neg\alpha_1)$	α_1
Commutativity	$\alpha_1 \wedge \alpha_2; \alpha_1 \vee \alpha_2$	$\alpha_2 \wedge \alpha_1; \alpha_2 \vee \alpha_1$
Associativity	$(\alpha_1 \wedge \alpha_2) \wedge \alpha_3; (\alpha_1 \vee \alpha_2) \vee \alpha_3$	$\alpha_1 \wedge (\alpha_2 \wedge \alpha_3); \alpha_1 \vee (\alpha_2 \vee \alpha_3)$
De Morgan's Laws	$\neg(\alpha_1 \wedge \alpha_2); \neg(\alpha_1 \vee \alpha_2)$	$\neg\alpha_1 \vee \neg\alpha_2; \neg\alpha_1 \wedge \neg\alpha_2$
Transposition	$\alpha_1 \rightarrow \alpha_2$	$\neg\alpha_2 \rightarrow \neg\alpha_1$
Exportation	$\alpha_1 \rightarrow (\alpha_2 \rightarrow \alpha_3)$	$(\alpha_1 \wedge \alpha_2) \rightarrow \alpha_3$
Distribution	$\alpha_1 \wedge (\alpha_2 \vee \alpha_3); \alpha_1 \vee (\alpha_2 \wedge \alpha_3)$	$(\alpha_1 \wedge \alpha_2) \vee (\alpha_1 \wedge \alpha_3); (\alpha_1 \vee \alpha_2) \wedge (\alpha_1 \vee \alpha_3)$
Material Implication (MI)	$\alpha_1 \rightarrow \alpha_2$	$\neg\alpha_1 \vee \alpha_2$
Material Equivalence (ME)	$\alpha_1 \leftrightarrow \alpha_2$	$(\alpha_1 \rightarrow \alpha_2) \wedge (\alpha_2 \rightarrow \alpha_1)$

EXAMPLE 1 (CHECK YOUR PROGRESS 3, Q2)

Verify De Morgan's laws using truth tables.

A	B	$\neg(A \wedge B)$	$\neg A \vee \neg B$	$\neg(A \vee B)$	$\neg A \wedge \neg B$
F	F	T	T	T	T
F	T	T	T	F	F
T	F	T	T	F	F
T	T	F	F	F	F

Columns 3 and 4 are identical, and columns 5 and 6 are identical, so $\neg(A \wedge B) \equiv \neg A \vee \neg B$ and $\neg(A \vee B) \equiv \neg A \wedge \neg B$.

EXAMPLE 2 (EQUIVALENCE CHECK)

Is $P \rightarrow Q$ equivalent to its contrapositive $\neg Q \rightarrow \neg P$? To its converse $Q \rightarrow P$?

P	Q	$P \rightarrow Q$	$\neg Q \rightarrow \neg P$	$Q \rightarrow P$	$\neg P \vee Q$
F	F	T	T	T	T
F	T	T	T	F	T
T	F	F	F	T	F
T	T	T	T	T	T

$P \rightarrow Q \equiv \neg Q \rightarrow \neg P \equiv \neg P \vee Q$ (Transposition and MI). But $P \rightarrow Q \not\equiv Q \rightarrow P$: they differ in rows 2 and 3.

EXAMPLE 3 (EQUIVALENCE BY LAWS)

Simplify $\neg(P \rightarrow Q)$ using the laws.

$\neg(P \rightarrow Q) \equiv \neg(\neg P \vee Q)$ (MI) $\equiv \neg\neg P \wedge \neg Q$ (De Morgan) $\equiv P \wedge \neg Q$ (DN). So "it is not true that if P then Q" means "P and not Q".

Exam tip: The unit's table prints Commutativity for $\vee$ as " $\alpha_1 \vee \alpha_2 \equiv \alpha_1 \vee \alpha_2$ "; this is a misprint, the correct law is $\alpha_1 \vee \alpha_2 \equiv \alpha_2 \vee \alpha_1$. Also remember $\neg(P \rightarrow Q) \equiv P \wedge \neg Q$; it appears in almost every resolution question.

Validity and satisfiability

Valid (tautology): A formula that is true under every truth assignment.

Valid (tautology): എല്ലാ truth assignments-ലും ശരിയാകുന്ന formula.

Satisfiable: A formula that is true under at least one truth assignment.

Satisfiable: കുറഞ്ഞത് ഒരു truth assignment-ൽ എങ്കിലും ശരിയാകുന്ന formula.

Unsatisfiable (contradiction): A formula that is false under every truth assignment.

Unsatisfiable (contradiction): എല്ലാ truth assignments-ലും തെറ്റാകുന്ന formula.

A formula is **valid** (a tautology) if it is true for every combination of truth values. It is **satisfiable** if it is true for at least one combination. It is **unsatisfiable** (a contradiction) if it is false in every row. A formula that is satisfiable but not valid is called a **contingency**. An argument with premises $P_1, \dots, P_n$ and conclusion C is valid when $(P_1 \wedge \dots \wedge P_n) \rightarrow C$ is a tautology, i.e. there is no row where all premises are T and C is F.

മലയാളത്തിൽ

Valid formula എപ്പോഴും ശരി, ഏത് row എടുത്താലും T (ഉദാ: $P \vee \neg P$, "ഇന്ന് മഴ പെയ്യും അല്ലെങ്കിൽ പെയ്യില്ല"). Satisfiable എന്നാൽ ഒരു row-ലെങ്കിലും T. Unsatisfiable എന്നാൽ എല്ലാ row-ഉം F (ഉദാ: $P \wedge \neg P$). Argument valid ആണോ എന്നറിയാൻ: എല്ലാ premises-ഉം T ആയ rows മാത്രം നോക്കുക; അവിടെയെല്ലാം conclusion-ഉം T ആണെങ്കിൽ valid.

Valid (tautology): T in all rows, written $\models A$

Satisfiable: T in at least one row

Unsatisfiable (contradiction): F in all rows

Contingency: satisfiable but not valid

A is valid $\Leftrightarrow \neg A$ is unsatisfiable

$P_1, \dots, P_n \models C \Leftrightarrow (P_1 \wedge \dots \wedge P_n) \rightarrow C$ is valid $\Leftrightarrow P_1 \wedge \dots \wedge P_n \wedge \neg C$ is unsatisfiable

EXAMPLE 1 (UNIT EXAMPLE)

Show that $\alpha_1 \vee (\alpha_2 \vee \alpha_3) \vee (\neg\alpha_2 \wedge \neg\alpha_3)$ is valid.

α_1	α_2	α_3	$\neg\alpha_2$	$\neg\alpha_3$	$\alpha_2 \vee \alpha_3$	$\neg\alpha_2 \wedge \neg\alpha_3$	Result
F	F	F	T	T	F	T	T
F	F	T	T	F	T	F	T
F	T	F	F	T	T	F	T
F	T	T	F	F	T	F	T
T	F	F	T	T	F	T	T
T	F	T	T	F	T	F	T
T	T	F	F	T	T	F	T
T	T	T	F	F	T	F	T

All T, so the formula is valid. (Reason: $\neg\alpha_2 \wedge \neg\alpha_3 \equiv \neg(\alpha_2 \vee \alpha_3)$, and $X \vee \neg X$ is always true.)

EXAMPLE 2 (UNIT EXAMPLE, CORRECTED)

Is $\alpha_1 \wedge ((\alpha_2 \wedge \alpha_3) \vee (\alpha_1 \wedge \alpha_3))$ valid, satisfiable or unsatisfiable?

α_1	α_2	α_3	$\alpha_2 \wedge \alpha_3$	$\alpha_1 \wedge \alpha_3$	$(\alpha_2 \wedge \alpha_3) \vee (\alpha_1 \wedge \alpha_3)$	Result
F	F	F	F	F	F	F
F	F	T	F	F	F	F
F	T	F	F	F	F	F
F	T	T	T	F	T	F
T	F	F	F	F	F	F
T	F	T	F	T	T	T
T	T	F	F	F	F	F
T	T	T	T	T	T	T

True in 2 rows ($\alpha_1 = \alpha_3 = T$), false in the rest: satisfiable, but not valid.

EXAMPLE 3 (CHECKING AN ARGUMENT)

Is the argument " $P \rightarrow Q, Q, \text{ therefore } P$ " valid?

Look for a row with both premises T and the conclusion F. Row $P = F, Q = T$: $P \rightarrow Q = T, Q = T$, but $P = F$. So the argument is **invalid** (fallacy of affirming the consequent). Compare " $P \rightarrow Q, P, \text{ therefore } Q$ ": the only row with both premises T is $P = Q = T$, where $Q = T$, so it is valid (Modus Ponens).

Exam tip: The unit's truth table for Example 2 has wrong entries (for instance it shows $\alpha_1 \wedge \alpha_3 = T$ when $\alpha_1 = F$, and a final T in rows T F F and T T F). The correct table is above: the formula simplifies to $\alpha_1 \wedge \alpha_3$. The conclusion "satisfiable" is still correct. Also, in Example 1 the unit labels a column " $\alpha_1 \vee \alpha_2$ " but its values are $\alpha_2 \vee \alpha_3$.

Introduction to predicate logic

Predicate: A sentence $p(x)$ containing a variable x ; it becomes a proposition only when x is given a value.

Predicate: ഒരു variable x അടങ്ങിയ $p(x)$ എന്ന sentence; x -ന് ഒരു value കൊടുക്കുമ്പോൾ മാത്രമേ അത് ഒരു proposition ആകൂ.

Universe of discourse: The set U of values that the variable of a predicate may take.

Universe of discourse: ഒരു predicate-ലെ variable-ന് എടുക്കാവുന്ന values-ന്റെ set U .

A **predicate** (propositional function) $p(x)$ is a sentence containing a variable x ; it is not a proposition until x is given a value. The set of values x can take is the **universe of discourse** U . Propositions use only connectives; predicates use connectives and **quantifiers**: $\forall$ ("for all", universal) and $\exists$ ("there exists", existential). In short, a proposition is a specialised statement, a predicate is a generalised one.

മലയാളത്തിൽ

" $x > 5$ " എന്ന് പറഞ്ഞാൽ ശരിയോ തെറ്റോ എന്ന് പറയാൻ പറ്റില്ല, x എന്താണെന്ന് അറിയണം. ഇതാണ് predicate. $x = 6$ കൊടുത്താൽ " $6 > 5$ " എന്ന true proposition കിട്ടും, $x = 0$ കൊടുത്താൽ false proposition. ഒരു form-ലെ ഒഴിഞ്ഞ കോളം പോലെയാണ് x ; അത് പൂരിപ്പിച്ചാലേ statement പൂർണ്ണമാകൂ. അല്ലെങ്കിൽ $\forall$ അല്ലെങ്കിൽ $\exists$ ഉപയോഗിച്ച് "എല്ലാവർക്കും" അല്ലെങ്കിൽ "ഒരാൾക്കെങ്കിലും" എന്ന് പറയണം.

$p(x)$: predicate on universe U

$p(a)$, $a \in U$: a proposition

$(\forall x \in U) p(x)$: $p(x)$ is true for every x in U

$(\exists x \in U) p(x)$: $p(x)$ is true for at least one x in U

EXAMPLE 1

Let $p(x)$: $x > 5$. Find the truth value of $p(6)$, $p(0)$, $(\exists x \in \mathbb{N}) p(x)$, $(\forall x \in \mathbb{N}) p(x)$.

$p(6)$: $6 > 5$, T. $p(0)$: $0 > 5$, F. $(\exists x \in \mathbb{N}) p(x)$: T, take $x = 6$. $(\forall x \in \mathbb{N}) p(x)$: F, $x = 1$ fails.

EXAMPLE 2 (UNIT EXAMPLES)

Find the truth values: (a) $(\exists x \in \mathbb{R})(x + 1 > 0)$ (b) $(\exists x \in \mathbb{N})(x - 1/2 = 0)$ (c) $(\forall x \in \mathbb{N})(x^2 > x)$.

(a) True: $x = 0$ gives $1 > 0$. (b) False: the only solution is $x = 1/2$, which is not a natural number. (c) False: $x = 1$ gives $1^2 = 1$, and $1 > 1$ is false. One counterexample is enough to make a $\forall$ statement false.

Exam tip: The unit reads $(\forall x \in \mathbb{N})(x^2 > x)$ as "for every x not in $\mathbb{N}$ "; that is a misprint. It is "for every x in $\mathbb{N}$ ", and it is false because of $x = 1$.

Quantifiers and their negation

Universal quantifier ($\forall$): $(\forall x) p(x)$ means $p(x)$ is true for every x in the universe U .

Universal quantifier ($\forall$): Universe U -ലെ എല്ലാ x -നും $p(x)$ ശരിയാണ് എന്നാണ് $(\forall x) p(x)$ അർത്ഥമാക്കുന്നത്.

Existential quantifier ($\exists$): $(\exists x) p(x)$ means $p(x)$ is true for at least one x in U .

Existential quantifier ($\exists$): U -ലെ കുറഞ്ഞത് ഒരു x -നെക്കുറിച്ചും $p(x)$ ശരിയാണ് എന്നാണ് $(\exists x) p(x)$ അർത്ഥമാക്കുന്നത്.

Negation of quantifiers: $\neg(\forall x) p(x) \equiv (\exists x) \neg p(x)$ and $\neg(\exists x) p(x) \equiv (\forall x) \neg p(x)$: flip the quantifier and move $\neg$ inside.

Negation of quantifiers: $\neg(\forall x) p(x) \equiv (\exists x) \neg p(x)$, $\neg(\exists x) p(x) \equiv (\forall x) \neg p(x)$: quantifier മാറ്റി $\neg$ ഉള്ളിലേക്ക് നീക്കുന്നു.

To negate a quantified statement, flip the quantifier and push the $\neg$ inside. "It is not true that there is a child in the class" means "every child is not in the class". This pair of rules is the quantifier form of De Morgan's laws.

മലയാളത്തിൽ

"ക്ലാസിൽ ഒരു കുട്ടിയെങ്കിലും ഉണ്ട്" എന്നതിന്റെ negation "ക്ലാസിൽ ഒരു കുട്ടിയും ഇല്ല", അതായത് "എല്ലാ കുട്ടികളും ക്ലാസിന് പുറത്താണ്". $\exists$ ഉള്ളതിനെ negate ചെയ്താൽ $\forall$ വരും, $\forall$ ഉള്ളതിനെ negate ചെയ്താൽ $\exists$ വരും, ഒപ്പം $\neg$ ഉള്ളിലേക്ക് പോകും. "എല്ലാവരും പാസായി" എന്നത് തെറ്റാണെങ്കിൽ, "ഒരാളെങ്കിലും തോറ്റു" എന്നാണ് അർത്ഥം, "എല്ലാവരും തോറ്റു" എന്നല്ല.

$$\neg(\forall x \in U) p(x) \equiv (\exists x \in U) \neg p(x)$$

$$\neg(\exists x \in U) p(x) \equiv (\forall x \in U) \neg p(x)$$

$$(\exists x \in U) p(x) \equiv \neg(\forall x \in U) \neg p(x)$$

EXAMPLE 1

Negate: (a) "Every student passed." (b) "There is at least one child in the class." (c) $(\forall x \in \mathbb{N})(x^2 > x)$.

(a) $\neg \forall x \text{ Passed}(x) \equiv \exists x \neg \text{Passed}(x)$: "Some student did not pass." (b) $\neg \exists x \text{ InClass}(x) \equiv \forall x \neg \text{InClass}(x)$: "No child is in the class." (c) $(\exists x \in \mathbb{N})(x^2 \leq x)$, which is true ($x = 1$).

EXAMPLE 2

Negate $\forall x (\text{Student}(x) \rightarrow \text{Clever}(x))$ and simplify.

$$\neg \forall x (\text{Student}(x) \rightarrow \text{Clever}(x))$$

$$\equiv \exists x \neg (\text{Student}(x) \rightarrow \text{Clever}(x)) \text{ (quantifier negation)}$$

$$\equiv \exists x \neg (\neg \text{Student}(x) \vee \text{Clever}(x)) \text{ (MI)}$$

$$\equiv \exists x (\text{Student}(x) \wedge \neg \text{Clever}(x)) \text{ (De Morgan, DN)}$$

"Some student is not clever."

Exam tip: The negation of "all are" is "some are not", never "none are".

Translating English into logic

Definition: An interpretation is an assignment of meaning to the symbols of a formula, i.e. fixing which statement, property or relation each symbol stands for.

ഒരു formula-യിലെ symbols-ന് അർത്ഥം നൽകുന്നതാണ് interpretation, അതായത് ഓരോ symbol-ഉം ഏത് statement, property അല്ലെങ്കിൽ relation ആണ് സൂചിപ്പിക്കുന്നത് എന്ന് നിശ്ചയിക്കൽ.

By an **interpretation** we mean assigning meaning to symbols. To translate: (1) choose a letter for each atomic statement (or a predicate for each property/relation), (2) find the connective words, (3) write the formula with brackets. With quantifiers, "All A are B" uses $\rightarrow$ inside $\forall$, and "Some A are B" uses $\wedge$ inside $\exists$.

മലയാളത്തിൽ

English sentence-നെ logic-ലേക്ക് മാറ്റുന്നത് ഒരു ഭാഷാ വിവർത്തനം പോലെയാണ്. ആദ്യം ഓരോ ചെറിയ statement-നും ഒരു അക്ഷരം കൊടുക്കുക, പിന്നെ "and", "or", "if", "only if", "unless" പോലുള്ള വാക്കുകൾ connectives ആക്കുക. Quantifiers-ൽ ഒരു സൂത്രം ഓർക്കുക: $\forall$ കൂടെ $\rightarrow$, $\exists$ കൂടെ $\wedge$. "എല്ലാ മലയാളികളും ചായ കുടിക്കും" = $\forall x$ (Malayali(x) $\rightarrow$ DrinksTea(x)).

"A only if B": $A \rightarrow B$

"A if B", "B is sufficient for A": $B \rightarrow A$

"A unless B": $\neg B \rightarrow A \equiv A \vee B$

"Neither A nor B": $\neg A \wedge \neg B$

"All S are P": $\forall x (S(x) \rightarrow P(x))$

"Some S are P": $\exists x (S(x) \wedge P(x))$

"No S is P": $\forall x (S(x) \rightarrow \neg P(x))$

EXAMPLE 1 (UNIT EXAMPLE, CORRECTED)

Symbolise: (a) Weather is not hot today, and it is windier than yesterday. (b) Kids will go to the park only if it is not hot. (c) If kids do not go to the park, then they can play near the pool. (d) If kids play near the pool, then we can offer them a juice. Use H: Weather is not hot today, W: It is windier than yesterday, K: Kids will go to the park, P: They can play near the pool, J: We can offer them juice.

(a) $H \wedge W$

(b) "K only if not hot" is $K \rightarrow (\text{not hot})$. Since H already means "not hot", this is $K \rightarrow H$.

(c) $\neg K \rightarrow P$

(d) $P \rightarrow J$

EXAMPLE 2 (CHECK YOUR PROGRESS 3, Q3)

Write wffs for: (a) Every person has a mother. (b) There is a woman and she is the mother of Siya.

(a) Person(x): x is a person; Mother(y, x): y is the mother of x.

$\forall x (\text{Person}(x) \rightarrow \exists y \text{Mother}(y, x))$

(b) Woman(x): x is a woman.

$\exists x (\text{Woman}(x) \wedge \text{Mother}(x, \text{Siya}))$

EXAMPLE 3

Translate: (a) "I will go out unless it rains" (G, R). (b) "Some students are not lazy." (c) "No bird is a mammal."

(a) $\neg R \rightarrow G$, equivalently $G \vee R$.

(b) $\exists x (\text{Student}(x) \wedge \neg \text{Lazy}(x))$

(c) $\forall x (\text{Bird}(x) \rightarrow \neg \text{Mammal}(x))$

Exam tip: The unit writes (b) of Example 1 as $K \rightarrow \neg H$. Because H is defined as "weather is not hot", $\neg H$ means "hot", so $K \rightarrow \neg H$ says kids go only if it is hot. The correct formula is $K \rightarrow H$. Also, writing $\forall x (\text{Student}(x) \wedge \text{Clever}(x))$ for "all students are clever" is wrong: it claims everything in the universe is a student.

Inferencing in predicate logic

Entailment: Deriving new statements from given statements using the rules of inference.

Entailment: തന്നിരിക്കുന്ന statements-ൽ നിന്ന് rules of inference ഉപയോഗിച്ച് പുതിയ statements derive ചെയ്യൽ.

Proof by contradiction: Assume a statement and derive a contradiction from it; this shows the assumption is false.

Proof by contradiction: ഒരു statement assume ചെയ്ത് അതിൽ നിന്ന് ഒരു contradiction derive ചെയ്യൽ; ഇത് ആ assumption തെറ്റാണെന്ന് കാണിക്കുന്നു.

Counterexample: A case in which p is true and q is false; it disproves $p \rightarrow q$.

Counterexample: p ശരിയും q തെറ്റും ആകുന്ന ഒരു സന്ദർഭം; ഇത് $p \rightarrow q$ തെറ്റാണെന്ന് തെളിയിക്കുന്നു.

Deriving new statements from given ones with the rules of inference is called **entailment**. The same rules of section 4.7 apply. Two useful proof ideas appear here: **proof by contradiction** (assume something, reach a contradiction, so the assumption is false) and **counterexamples** (to disprove $p \rightarrow q$, find a case where p is true and q is false).

മലയാളത്തിൽ

ഒരു കള്ളനെ പിടിക്കാൻ പോലീസ് ചെയ്യുന്നതുപോലെ: "ഇവൻ സത്യം പറയുന്നു എന്ന് കരുതാം" എന്ന് തുടങ്ങി, അത് ഒരു വൈരുദ്ധ്യത്തിൽ എത്തിയാൽ ആ assumption തെറ്റ്. ഒരു general statement തെറ്റാണെന്ന് കാണിക്കാൻ ഒരൊറ്റ counterexample മതി: "എല്ലാ odd numbers-ഉം prime ആണ്" എന്നതിന് 15 മതി.

Counterexample to $p \rightarrow q$: a case where $p \wedge \neg q$ is true
From S and $\neg S$, any statement can be derived (explosion)

EXAMPLE 1 (TRUTH-TELLERS AND LIARS)

In a village, people either always tell the truth or always lie. A says "B always tells the truth". B says "A and I are of opposite types". What are A and B?

Assume A is a truth-teller. Then B is a truth-teller (A's statement is true). Then B's statement is true, so A and B are of opposite types. But both are truth-tellers: contradiction. So A is a liar.

Since A lies, "B always tells the truth" is false, so B is a liar.

Check: B says "we are opposite types"; both are liars, so this is false, which fits B being a liar. Answer: both A and B are liars.

EXAMPLE 2 (COUNTEREXAMPLE)

Disprove: "If n is an odd integer, then n is prime."

Take $n = 15$: 15 is odd (p true), but $15 = 3 \times 5$ is not prime (q false). So $p \wedge \neg q$ holds and the statement is false.

EXAMPLE 3 (UNIT EXAMPLE)

Symbolise and prove: (i) If BOOK_X is literally true, the Earth was made in six days. (ii) If the Earth was made in six days, carbon dating is useless and scientists are liars. (iii) Scientists are not liars. (iv) BOOK_X is literally true. Hence (v) God does not exist.

B: BOOK_X is literally true, E: Earth made in six days, C: carbon dating is useless, S: scientists are liars, G: God exists.

1. $B \rightarrow E$ (premise)
2. $E \rightarrow (C \wedge S)$ (premise)
3. $\neg S$ (premise)
4. B (premise)
5. E (MP, 1, 4)
6. $C \wedge S$ (MP, 2, 5)
7. S (Simplification, 6)
8. $S \vee \neg G$ (Addition, 7)
9. $\neg G$ (DS, 8, 3)

The proof is formally correct, but lines 3 and 7 ($\neg S$ and S) contradict each other: the premises are inconsistent. From inconsistent premises any conclusion (even G) can be derived, so this argument tells us nothing about the real world.

Exam tip: In the puzzle, always finish by checking that your answer is consistent with both statements. For "disprove" questions, one clearly stated counterexample earns full marks.

Proof systems

Proof: A sequence of statements, each a premise, an axiom or derived from earlier lines by a rule; the last line is the theorem.

Proof: Statements-ന്റെ ഒരു sequence; ഓരോന്നും ഒരു premise, ഒരു axiom, അല്ലെങ്കിൽ മുമ്പത്തെ lines-ൽ നിന്ന് ഒരു rule വഴി derive ചെയ്തത് ആണ്; അവസാനത്തെ line ആണ് theorem.

$\Gamma \vdash C$ (**derivable**): C can be derived from the premises Γ using the inference rules (syntactic).

$\Gamma \models C$ (**entails**): C is true in every truth assignment that makes all of Γ true (semantic).

$\Gamma \vdash C$ (**derivable**): Inference rules ഉപയോഗിച്ച് premises Γ -ൽ നിന്ന് C derive ചെയ്യാം (syntactic).

$\Gamma \models C$ (**entails**): Γ -ലെ എല്ലാ statements-ഉം ശരിയാക്കുന്ന ഓരോ truth assignment-ലും C ശരിയാണ് (semantic).

A proof is a sequence of statements, each of which is a premise, an axiom, or follows from earlier lines by a rule; the last line is the **theorem**. The two main styles are **axiomatic proof systems** (derive from axioms and hypotheses) and **natural deduction**. We write $\Gamma \vdash C$ when C can be derived from premises Γ by rules (syntactic), and $\Gamma \models C$ when C is true in every row where all of Γ are true (semantic). A sound and complete system has $\vdash$ and $\models$ agreeing.

മലയാളത്തിൽ

Proof എന്നത് ഒരു കോണിപ്പടി പോലെയാണ്: ഓരോ പടിയും premise ആയിരിക്കണം, അല്ലെങ്കിൽ മുൻപത്തെ പടികളിൽ നിന്ന് ഏതെങ്കിലും rule വഴി കിട്ടിയതാവണം. ഓരോ line-നും അരികിൽ കാരണം (MP, MT, DS...) എഴുതണം. $\vdash$ എന്നാൽ "rules ഉപയോഗിച്ച് derive ചെയ്യാം", $\models$ എന്നാൽ "truth table പ്രകാരം ശരി".

$\Gamma \vdash C$: C is derivable from Γ using inference/replacement rules

$\Gamma \models C$: every assignment making all of Γ true makes C true

Conditional proof: to prove $A \rightarrow B$, assume A , derive B , then conclude $A \rightarrow B$

EXAMPLE 1 (UNIT EXAMPLE)

Show $\alpha_2 \wedge \alpha_1 \vdash \alpha_1 \vee \alpha_2$.

1. $\alpha_2 \wedge \alpha_1$ (premise)
2. α_1 (Simplification, 1)
3. $\alpha_1 \vee \alpha_2$ (Addition, 2)

EXAMPLE 2 (UNIT EXAMPLE)

Derive $\alpha_1 \rightarrow \neg\alpha_4$ from $\alpha_1 \rightarrow (\alpha_2 \vee \alpha_3)$, $\alpha_2 \rightarrow \neg\alpha_1$, $\alpha_4 \rightarrow \neg\alpha_3$.

1. $\alpha_1 \rightarrow (\alpha_2 \vee \alpha_3)$ (premise)
2. $\alpha_2 \rightarrow \neg\alpha_1$ (premise)
3. $\alpha_4 \rightarrow \neg\alpha_3$ (premise)
4. α_1 (assumption for conditional proof)
5. $\alpha_2 \vee \alpha_3$ (MP, 1, 4)
6. $\neg\neg\alpha_1$ (DN, 4)
7. $\neg\alpha_2$ (MT, 2, 6)
8. α_3 (DS, 5, 7)
9. $\neg\neg\alpha_3$ (DN, 8)
10. $\neg\alpha_4$ (MT, 3, 9)
11. $\alpha_1 \rightarrow \neg\alpha_4$ (conditional proof, 4-10)

Exam tip: The unit's version cites steps loosely (e.g. "DS, Step 1, 2, 4"). In your answer, cite exactly the lines each rule uses, and show the DN step before applying MT to a negated consequent.

Natural deduction

Definition: Natural deduction is a proof method that applies rules of inference to the premises, as in human reasoning, instead of using truth tables. Indirect form: add $\neg C$ and derive a contradiction.

Truth tables-ന് പകരം, മനുഷ്യന്റെ ന്യായവാദം പോലെ, premises-ൽ rules of inference പ്രയോഗിക്കുന്ന proof method ആണ് natural deduction. Indirect രൂപം: $\neg C$ ചേർത്ത് ഒരു contradiction derive ചെയ്യുന്നു.

Truth tables grow as 2^k rows for k variables, which becomes unmanageable ($k = 50$ gives about 10^{15} rows).

Natural deduction instead works on the individual premises with inference rules, like human reasoning. The unit's summary describes the indirect version: add the negated conclusion as an extra premise and derive a contradiction; the contradiction shows the conclusion follows. (Deriving the conclusion directly from the premises is the direct version.)

മലയാളത്തിൽ

50 variables ഉള്ള truth table എഴുതാൻ ജീവിതകാലം മതിയാകില്ല. അതുകൊണ്ട് natural deduction ഉപയോഗിക്കുന്നു: premises-ൽ നിന്ന് rules വഴി പടിപടിയായി conclusion-ലേക്ക്. Indirect രീതിയിൽ, "conclusion തെറ്റാണ്" എന്ന് കരുതി ഒരു premise ആയി ചേർക്കുന്നു; എന്നിട്ട് ഒരു വൈരുദ്ധ്യം (Q -ഉം $\neg Q$ -ഉം) കിട്ടിയാൽ, ആ assumption തെറ്റ്, conclusion ശരി.

Indirect proof: premises + $\neg C \vdash (X \text{ and } \neg X) \Rightarrow \text{premises} \vdash C$

EXAMPLE 1 (UNIT EXAMPLE)

Show that Z follows from $X, X \rightarrow Y, Y \rightarrow Z$.

1. $\neg Z$ (negated conclusion)
2. X (premise)
3. $X \rightarrow Y$ (premise)
4. Y (MP, 2, 3)
5. $Y \rightarrow Z$ (premise)
6. Z (MP, 4, 5)
7. Lines 1 and 6 contradict ($Z \wedge \neg Z$). So the assumption $\neg Z$ is false and Z follows.

EXAMPLE 2 (UNIT EXAMPLE)

Show that $\neg P$ follows from $R \vee S, S \rightarrow \neg Q, P \rightarrow Q, R \rightarrow \neg Q$.

1. $\neg\neg P$ (negated conclusion)
2. P (DN, 1)
3. $P \rightarrow Q$ (premise)
4. Q (MP, 2, 3)
5. $S \rightarrow \neg Q$ (premise)
6. $\neg\neg Q$ (DN, 4)
7. $\neg S$ (MT, 5, 6)
8. $R \vee S$ (premise)
9. R (DS, 8, 7)
10. $R \rightarrow \neg Q$ (premise)
11. $\neg Q$ (MP, 9, 10)
12. Lines 4 and 11: Q and $\neg Q$, a contradiction. Hence $\neg P$ follows.

EXAMPLE 3 (DIRECT PROOF)

Prove $P \rightarrow Q, Q \rightarrow R, \neg R \vdash \neg P$.

1. $P \rightarrow Q$ (premise)
2. $Q \rightarrow R$ (premise)
3. $\neg R$ (premise)
4. $P \rightarrow R$ (HS, 1, 2)
5. $\neg P$ (MT, 4, 3)

Exam tip: State the contradiction explicitly ("line 4 gives Q , line 11 gives $\neg Q$ "). Without that line, the indirect proof is incomplete.

Normal forms: CNF and DNF

Literal: An atom or its negation (A or $\neg A$).

Literal: ഒരു atom അല്ലെങ്കിൽ അതിന്റെ negation (A അല്ലെങ്കിൽ $\neg A$).

CNF: A conjunction of clauses, where each clause is a disjunction of literals (an AND of ORs).

CNF: Clauses-ന്റെ ഒരു conjunction; ഓരോ clause-ഉം literals-ന്റെ ഒരു disjunction ആണ് (ORs-ന്റെ AND).

DNF: A disjunction of conjunctions of literals (an OR of ANDs).

DNF: Literals-ന്റെ conjunctions-ന്റെ ഒരു disjunction (ANDs-ന്റെ OR).

A **literal** is an atom or its negation (A or $\neg A$). A **clause** is a disjunction of literals. A formula is in **Conjunctive Normal Form (CNF)** if it is a conjunction of clauses (an AND of ORs), and in **Disjunctive Normal Form (DNF)** if it is a disjunction of conjunctions of literals (an OR of ANDs). Every propositional formula can be converted to both forms using the replacement rules.

മലയാളത്തിൽ

CNF എന്നാൽ "AND of ORs": $(\dots \vee \dots) \wedge (\dots \vee \dots)$. DNF എന്നാൽ "OR of ANDs": $(\dots \wedge \dots) \vee (\dots \wedge \dots)$. Conversion-ന് ഒരു fixed recipe ഉണ്ട്, ഒരു കഠിന വെക്കുന്നത് പോലെ step by step: ആദ്യം $\leftrightarrow, \rightarrow$ ഒഴിവാക്കുക; പിന്നെ $\neg$ ഉള്ളിലേക്ക് തള്ളുക (De Morgan, DN); അവസാനം distributive law. ഏത് order-ൽ ചെയ്യണം എന്നത് തെറ്റിക്കരുത്.

Step 1 (remove $\leftrightarrow, \rightarrow$): $E \leftrightarrow G \equiv (E \rightarrow G) \wedge (G \rightarrow E)$; $E \rightarrow G \equiv \neg E \vee G$

Step 2 (move $\neg$ inwards): $\neg\neg E \equiv E$; $\neg(E \vee G) \equiv \neg E \wedge \neg G$; $\neg(E \wedge G) \equiv \neg E \vee \neg G$

Step 3 (distribute):

for CNF: $E \vee (G \wedge H) \equiv (E \vee G) \wedge (E \vee H)$

for DNF: $E \wedge (G \vee H) \equiv (E \wedge G) \vee (E \wedge H)$

EXAMPLE 1 (IDENTIFY)

Let A, B, C be atoms. Classify: (a) $(\neg A \wedge B) \vee (A \wedge \neg B \wedge \neg C)$ (b) $(\neg A \vee B) \wedge (A \vee \neg B \vee \neg C)$ (c) $(\neg A \vee B) \wedge (A \vee (\neg B \wedge C))$ (d) $(A \rightarrow B) \wedge (\neg B \wedge \neg A)$.

(a) DNF (OR of ANDs). (b) CNF (AND of ORs). (c) Neither: the second conjunct $A \vee (\neg B \wedge C)$ has an $\wedge$ inside an $\vee$, so it is not a clause. (d) Neither: it contains $\rightarrow$.

EXAMPLE 2 (UNIT EXAMPLE)

Obtain a DNF and a CNF for $\neg(A \rightarrow (\neg B \wedge C))$.

$\neg(A \rightarrow (\neg B \wedge C))$

$\equiv \neg(\neg A \vee (\neg B \wedge C))$ (MI)

$\equiv \neg\neg A \wedge \neg(\neg B \wedge C)$ (De Morgan)

$\equiv A \wedge (\neg\neg B \vee \neg C)$ (DN, De Morgan)

$\equiv A \wedge (B \vee \neg C)$ (DN) $\leftarrow$ this is already a **CNF** (clauses A and $B \vee \neg C$)

$\equiv (A \wedge B) \vee (A \wedge \neg C)$ (distribute $\wedge$ over $\vee$) $\leftarrow$ **DNF**

EXAMPLE 3 (UNIT EXAMPLE)

Obtain a CNF for $D \rightarrow (A \rightarrow (B \wedge C))$.

$$D \rightarrow (A \rightarrow (B \wedge C))$$

$$\equiv D \rightarrow (\neg A \vee (B \wedge C)) \text{ (MI, inner)}$$

$$\equiv \neg D \vee (\neg A \vee (B \wedge C)) \text{ (MI, outer)}$$

$$\equiv (\neg D \vee \neg A) \vee (B \wedge C) \text{ (associativity) } \leftarrow \text{ this is a DNF: } \neg D \vee \neg A \vee (B \wedge C)$$

$$\equiv (\neg D \vee \neg A \vee B) \wedge (\neg D \vee \neg A \vee C) \text{ (distribute } \vee \text{ over } \wedge) \leftarrow \text{ CNF}$$

EXAMPLE 4

Convert $P \leftrightarrow Q$ to CNF and DNF.

$$P \leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P) \text{ (ME)}$$

$$\equiv (\neg P \vee Q) \wedge (\neg Q \vee P) \text{ (MI) } \leftarrow \text{ CNF}$$

Distributing: $(\neg P \wedge \neg Q) \vee (\neg P \wedge P) \vee (Q \wedge \neg Q) \vee (Q \wedge P)$. The middle two terms are always false, so drop them:

$$\equiv (P \wedge Q) \vee (\neg P \wedge \neg Q) \leftarrow \text{ DNF}$$

EXAMPLE 5

Convert $(P \vee Q) \rightarrow R$ to CNF and DNF.

$$(P \vee Q) \rightarrow R \equiv \neg(P \vee Q) \vee R \text{ (MI)}$$

$$\equiv (\neg P \wedge \neg Q) \vee R \text{ (De Morgan) } \leftarrow \text{ DNF}$$

$$\equiv (\neg P \vee R) \wedge (\neg Q \vee R) \text{ (distribute } \vee \text{ over } \wedge) \leftarrow \text{ CNF}$$

Exam tip: A single literal like A counts as a clause, and also as a conjunction of one literal, so $A \wedge (B \vee \neg C)$ is CNF and $\neg D \vee \neg A \vee (B \wedge C)$ is DNF. The unit's text once writes "CNF of $(\neg A \rightarrow (\neg B \wedge C))$ "; it means the same formula $\neg(A \rightarrow (\neg B \wedge C))$ worked above.

Clausal form and the resolution rule

Clause: A disjunction of literals, written as a set of literals ($X \vee Y$ as $\{X, Y\}$); the empty clause $\{\}$ stands for FALSE.

Clause: Literals-ന്റെ ഒരു disjunction; literals-ന്റെ set ആയി എഴുതുന്നു ($X \vee Y$ എന്നത് $\{X, Y\}$); empty clause $\{\}$ FALSE-നെ സൂചിപ്പിക്കുന്നു.

Resolution rule: From two clauses containing complementary literals L and $\neg L$, derive the resolvent: a clause of all their remaining literals.

Resolution rule: Complementary literals L , $\neg L$ അടങ്ങിയ രണ്ട് clauses-ൽ നിന്ന്, അവയിലെ ബാക്കി എല്ലാ literals-ഉം ചേർന്ന resolvent എന്ന clause derive ചെയ്യുന്നു.

A clause can be written as a set of literals: $X \vee Y$ becomes $\{X, Y\}$. A CNF formula becomes a set of clauses: $A \wedge (\neg B \vee C)$ becomes $\{A\}, \{\neg B, C\}$. The empty clause $\{\}$ is an empty disjunction; it can never be true, so it stands for FALSE. The **resolution rule** (Robinson, 1965): if one clause contains a literal L and another contains $\neg L$ (a **complementary pair**), combine the remaining literals of both into a new clause, the **resolvent**.

മലയാളത്തിൽ

Resolution rule ഒരു "cancel" പരിപാടിയാണ്: ഒരു clause-ൽ Q -ഉം മറ്റൊന്നിൽ $\neg Q$ -ഉം ഉണ്ടെങ്കിൽ, അവ രണ്ടും വെട്ടിക്കളഞ്ഞ് ബാക്കിയുള്ളതെല്ലാം "or" കൊണ്ട് ചേർക്കുക. ഉദാ: "ചായ അല്ലെങ്കിൽ കാപ്പി" ($Q \vee R$), "ചായ ഇല്ലെങ്കിൽ ജ്യൂസ്" ($\neg Q \vee S$) എന്നിവയിൽ നിന്ന് "കാപ്പി അല്ലെങ്കിൽ ജ്യൂസ്" ($R \vee S$). Modus Ponens ഇതിന്റെ ഒരു special case മാത്രം.

Resolution: $\{\alpha_1, \dots, \alpha_m, L\}, \{\gamma_1, \dots, \gamma_n, \neg L\} \vdash \{\alpha_1, \dots, \alpha_m, \gamma_1, \dots, \gamma_n\}$

Special case (MP): $P, \neg P \vee Q \vdash Q$

$\{L\}, \{\neg L\} \vdash \{\}$ (FALSE)

Clause-form rules: $\beta_1 \vee \dots \vee \beta_m \Rightarrow \{\beta_1, \dots, \beta_m\}$; $\beta_1 \wedge \dots \wedge \beta_m \Rightarrow \{\beta_1\}, \dots, \{\beta_m\}$

EXAMPLE 1 (UNIT EXAMPLE)

Convert $A \wedge (B \rightarrow C)$ to clausal form.

Step 1: $A \wedge (\neg B \vee C)$ (MI). Step 2: split the $\wedge$: clauses $\{A\}, \{\neg B, C\}$.

EXAMPLE 2 (UNIT EXAMPLE, CORRECTED)

Find the CNF and clauses of $\neg(\neg Z \wedge (\neg X \rightarrow \neg Y))$.

Step 1: $\neg(\neg Z \wedge (\neg\neg X \vee \neg Y))$ (MI)

Step 2: $\neg(\neg Z \wedge (X \vee \neg Y))$ (DN)

Step 3: $\neg\neg Z \vee \neg(X \vee \neg Y)$ (De Morgan)

Step 4: $Z \vee \neg(X \vee \neg Y)$ (DN)

Step 5: $Z \vee (\neg X \wedge \neg\neg Y)$ (De Morgan)

Step 6: $Z \vee (\neg X \wedge Y)$ (DN)

Step 7: $(Z \vee \neg X) \wedge (Z \vee Y)$ (distribution)

Clauses: $\{Z, \neg X\}, \{Z, Y\}$.

EXAMPLE 3 (RESOLVENTS)

Find the resolvent, if any: (a) $C_1: Q \vee R, C_2: \neg Q \vee S$ (b) $C_1: \neg S \vee \neg Q \vee R, C_2: \neg P \vee Q$ (c) $C_1: \neg Q \vee R, C_2: \neg Q \vee S$

(a) Q and $\neg Q$ are complementary: resolvent $R \vee S$.

(b) Q and $\neg Q$: resolvent $\neg S \vee R \vee \neg P$.

(c) No complementary pair (both have $\neg Q$), so resolution cannot be applied.

Exam tip: The unit states the resolution rule without the complementary literal (" $\{\alpha_1 \dots \alpha_m\}$ and $\{\gamma_1 \dots \gamma_n\}$ give $\{\alpha_1 \dots \alpha_m, \gamma_1 \dots \gamma_n\}$ "; the correct rule needs L in one clause and $\neg L$ in the other, as above. Also note the unit's Example 3 heading prints the formula as $\sim Z \wedge \sim((\sim X) \rightarrow (\sim Y))$, but its working (and answer) is for $\neg(\neg Z \wedge (\neg X \rightarrow \neg Y))$. The printed formula would give $\neg Z \wedge \neg X \wedge Y$ instead. Resolve on only one complementary pair at a time.

Resolution refutation (unsatisfiability)

Definition: Resolution refutation proves that premises P entail C by adding the clauses of $\neg C$ to those of P and deriving the empty clause $\{\}$ by repeated resolution.

P -യുടെ clauses-ലേക്ക് $\neg C$ -യുടെ clauses ചേർത്ത്, ആവർത്തിച്ചുള്ള resolution വഴി empty clause $\{\}$ derive ചെയ്ത്, premises P , C -യെ entail ചെയ്യുന്നു എന്ന് തെളിയിക്കുന്ന proof ആണ് resolution refutation.

Resolution is a sound and complete refutation method: a set of clauses is unsatisfiable exactly when repeated resolution can produce the empty clause $\{\}$. To prove that premises P entail conclusion C : (a) convert every premise to CNF/clauses, (b) add the clauses of $\neg C$, (c) resolve until $\{\}$ appears. If $\{\}$ appears, $P \models C$. If no new clauses can be produced and $\{\}$ has not appeared, C does not follow.

മലയാളത്തിൽ

Resolution refutation ഒരു വക്കീലിന്റെ തന്ത്രം പോലെയാണ്: "എതിർകക്ഷി പറയുന്നത് ($\neg C$) ശരിയാണെന്ന് തൽക്കാലം സമ്മതിക്കാം" എന്ന് തുടങ്ങി, അത് premises-മായി ചേർത്ത് resolve ചെയ്ത് {} (FALSE) എത്തിച്ചാൽ, $\neg C$ അസാധ്യം; അതുകൊണ്ട് C ശരി. എല്ലാം ആദ്യം clause form-ലേക്ക് മാറ്റണം എന്നതാണ് ഏറ്റവും പ്രധാന step.

$P \models C \Leftrightarrow P \cup \{\neg C\}$ is unsatisfiable $\Leftrightarrow$ resolution derives {} from clauses(P) $\cup$ clauses($\neg C$)

Any superset of an unsatisfiable set is unsatisfiable

EXAMPLE 1 (UNIT EXAMPLE)

From $X \rightarrow Y$ and $Y \rightarrow Z$, prove $X \rightarrow Z$.

Clauses: 1. $\neg X \vee Y$ (MI) 2. $\neg Y \vee Z$ (MI)

Negated conclusion: $\neg(X \rightarrow Z) \equiv \neg(\neg X \vee Z) \equiv X \wedge \neg Z$, giving 3. X and 4. $\neg Z$.

5. Y (resolve 1, 3 on X)

6. Z (resolve 5, 2 on Y)

7. {} (resolve 6, 4 on Z)

Empty clause, so $X \rightarrow Z$ follows.

EXAMPLE 2 (UNIT EXAMPLE)

Knowledge base: (a) Either it is a head, or Lisa wins. (b) If Lisa wins, then Mary will go. (c) If it is a head, then the game is over. (d) The game is not over. Prove: Mary will go.

H: it is a head, L: Lisa wins, M: Mary will go, G: game is over.

1. $H \vee L$ 2. $\neg L \vee M$ (from $L \rightarrow M$) 3. $\neg H \vee G$ (from $H \rightarrow G$) 4. $\neg G$ 5. $\neg M$ (negated conclusion)

6. $H \vee M$ (resolve 1, 2 on L)

7. $M \vee G$ (resolve 6, 3 on H)

8. M (resolve 7, 4 on G)

9. {} (resolve 8, 5 on M)

So "Mary will go" follows.

EXAMPLE 3 (UNIT EXAMPLE)

Show that $\neg S_1$ follows from $S_1 \rightarrow S_2$ and $\neg(S_1 \wedge S_2)$.

1. $\neg S_1 \vee S_2$ (MI) 2. $\neg S_1 \vee \neg S_2$ (De Morgan) 3. S_1 (negated conclusion)

4. S_2 (resolve 1, 3) 5. $\neg S_2$ (resolve 2, 3) 6. {} (resolve 4, 5). Hence $\neg S_1$. (Directly: resolving 1 and 2 on S_2 gives

$\neg S_1 \vee \neg S_1 = \neg S_1$.)

EXAMPLE 4 (UNIT EXAMPLE, WITH A CORRECTION)

Show that $C_1: R \vee S$, $C_2: \neg R \vee S$, $C_3: R \vee \neg S$, $C_4: \neg R \vee \neg S$ are unsatisfiable.

$C_5: S \vee S = S$ (resolve C_1, C_2 on R)

$C_6: \neg S$ (resolve C_3, C_4 on R)

$C_7: \{\}$ (resolve C_5, C_6). Unsatisfiable.

Truth-table check: RS = TT fails C_4 , TF fails C_3 , FT fails C_2 , FF fails C_1 . No assignment satisfies all four.

EXAMPLE 5 (UNIT EXAMPLE)

If the interest rate goes up, stock prices go down. If stock prices go down, most people are unhappy. The interest rate goes up. Show that most people are unhappy.

A: interest rate goes up, S: stock prices go down, U: most people are unhappy.

1. $\neg A \vee S$ 2. $\neg S \vee U$ 3. A 4. $\neg U$ (negated conclusion)

5. S (resolve 1, 3) 6. $\neg S$ (resolve 2, 4) 7. $\{\}$ (resolve 5, 6). So U follows.

EXAMPLE 6 (UNIT EXAMPLE)

Show $C_1: R \vee S$, $C_2: \neg S \vee W$, $C_3: \neg R \vee S$, $C_4: \neg W$ is unsatisfiable.

$C_5: S$ (resolve C_1, C_3 on R). $C_6: W$ (resolve C_5, C_2 on S). $C_7: \{\}$ (resolve C_6, C_4). Unsatisfiable.

Exam tip: The unit claims that in Example 4 "the resolvent FALSE could be obtained from C_2 and C_3 alone". That is wrong. Resolving $\neg R \vee S$ with $R \vee \neg S$ on R gives $S \vee \neg S$ (always true), and you may not cancel R and S together in one step. $\{C_2, C_3\}$ is satisfiable ($R = S = T$, or $R = S = F$). You need all four clauses. Also, in unit Example 1 the last step is labelled "Conjunction"; it is a resolution step.

Quick recap

- A proposition is a declarative sentence that is either true or false, not both; questions, commands and opinions are not propositions.
- Atomic propositions are single statements; compound propositions join them with $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$.
- $A \rightarrow B$ is false only when A is T and B is F; $A \leftrightarrow B$ is true when both sides match.
- Precedence: $\neg$ first, then $\wedge$, then $\vee$, then $\rightarrow$ and $\leftrightarrow$. A wff is built only by the recursive rules.
- n variables need 2^n truth-table rows; build columns from the inside out.
- Inference rules: MP, MT, DS, Addition, Simplification, Conjunction, HS, Absorption, Constructive Dilemma.
- Affirming the consequent and denying the antecedent are fallacies, not rules.
- Replacement laws: DN, commutativity, associativity, De Morgan, transposition, exportation, distribution, MI ($A \rightarrow B \equiv \neg A \vee B$), ME.

- Valid = true in all rows; satisfiable = true in at least one; unsatisfiable = true in none.
- Predicates become propositions when variables get values or are quantified with $\forall$ or $\exists$.
- $\neg\forall x p(x) \equiv \exists x \neg p(x)$ and $\neg\exists x p(x) \equiv \forall x \neg p(x)$. "All S are P" uses $\rightarrow$; "some S are P" uses $\wedge$.
- Natural deduction applies rules to individual premises; the indirect form adds $\neg C$ and derives a contradiction.
- CNF = AND of ORs, DNF = OR of ANDs. Recipe: remove $\leftrightarrow$ and $\rightarrow$, push $\neg$ inwards, distribute.
- Resolution refutation: clauses of the premises + clauses of $\neg C$; derive $\{\}$ to prove C.

Check yourself

Q. 1. Which are propositions? (a) How are you? (b) Akbar was the founder of the Mughal dynasty in India. (c) One should avoid eating fast food to maintain good health. (d) The first case of COVID-19 was detected in China. (e) Name the author of the "Shiva Trilogy".

(a) No, a question. (b) Yes. It is actually false (Babur founded the dynasty), but it has a definite truth value, so it is a proposition. (c) No, it is advice. (d) Yes. (e) No, a command.

Q. 2. Atomic or compound? (a) The first battle of Panipat was fought in 1556. (b) Uttarakhand was carved out of Uttar Pradesh in the year 2000. (c) Harappan civilisation is considered the oldest human civilisation and it lies in parts of present-day India, Pakistan and Afghanistan.

(a) Atomic (and false: the first battle was in 1526, but that does not affect the classification). (b) Atomic. (c) Compound: two statements joined by "and".

Q. 3. (Check Your Progress 3, Q1d) Draw the truth table of $((\alpha_1 \rightarrow \neg\alpha_2) \leftrightarrow \alpha_3) \rightarrow \neg(\alpha_1 \vee \alpha_1)$.

Note $\neg(\alpha_1 \vee \alpha_1) \equiv \neg\alpha_1$ (idempotence).

α_1	α_2	α_3	$\alpha_1 \rightarrow \neg\alpha_2$	$(\alpha_1 \rightarrow \neg\alpha_2) \leftrightarrow \alpha_3$	$\neg(\alpha_1 \vee \alpha_1)$	Result
F	F	F	T	F	T	T
F	F	T	T	T	T	T
F	T	F	T	F	T	T
F	T	T	T	T	T	T
T	F	F	T	F	F	T
T	F	T	T	T	F	F
T	T	F	F	T	F	F
T	T	T	F	F	F	T

A contingency (false in 2 rows). The unit's answer key has a wrong $\alpha_1 \rightarrow \neg\alpha_2$ column (it shows F when $\alpha_1 =$ F, and T when $\alpha_1 = \alpha_2 =$ T), which makes its final column wrong in rows F F T and F T F; the correct values there are T.

Q. 4. (Check Your Progress 3, Q1c) Draw the truth table of $((\alpha_1 \wedge \alpha_2) \rightarrow \alpha_3) \vee \neg \alpha_4$.

α_1	α_2	α_3	α_4	$\alpha_1 \wedge \alpha_2$	$(\alpha_1 \wedge \alpha_2) \rightarrow \alpha_3$	$\neg \alpha_4$	Result
F	F	F	F	F	T	T	T
F	F	F	T	F	T	F	T
F	F	T	F	F	T	T	T
F	F	T	T	F	T	F	T
F	T	F	F	F	T	T	T
F	T	F	T	F	T	F	T
F	T	T	F	F	T	T	T
F	T	T	T	F	T	F	T
T	F	F	F	F	T	T	T
T	F	F	T	F	T	F	T
T	F	T	F	F	T	T	T
T	F	T	T	F	T	F	T
T	T	F	F	T	F	T	T
T	T	F	T	T	F	F	F
T	T	T	F	T	T	T	T
T	T	T	T	T	T	F	T

False only when $\alpha_1 = \alpha_2 = T, \alpha_3 = F, \alpha_4 = T$. A contingency.

Q. 5. Is the argument " $P \rightarrow Q, \neg P$, therefore $\neg Q$ " valid? Justify with a truth table row.

Invalid. In the row $P = F, Q = T$: $P \rightarrow Q = T$ and $\neg P = T$ (both premises true), but $\neg Q = F$. This fallacy is called denying the antecedent.

Q. 6. Convert $(P \vee Q) \rightarrow R$ to CNF and DNF, and write its clauses.

$(P \vee Q) \rightarrow R \equiv \neg(P \vee Q) \vee R$ (MI) $\equiv (\neg P \wedge \neg Q) \vee R$ (De Morgan): DNF.

$\equiv (\neg P \vee R) \wedge (\neg Q \vee R)$ (distribution): CNF.

Clauses: $\{\neg P, R\}, \{\neg Q, R\}$.

Q. 7. Negate "All students are clever" symbolically and in English.

$\neg \forall x (\text{Student}(x) \rightarrow \text{Clever}(x)) \equiv \exists x \neg(\neg \text{Student}(x) \vee \text{Clever}(x)) \equiv \exists x (\text{Student}(x) \wedge \neg \text{Clever}(x))$. "Some student is not clever."

Q. 8. Using natural deduction, prove $P \rightarrow Q, Q \rightarrow R, \neg R \vdash \neg P$.

1. $P \rightarrow Q$ (premise) 2. $Q \rightarrow R$ (premise) 3. $\neg R$ (premise) 4. $P \rightarrow R$ (HS, 1, 2) 5. $\neg P$ (MT, 4, 3).

Q. 9. (Check Your Progress 4, Q1) Prove that $\{X, Y\}, \{\neg X, Z\}, \{\neg X, \neg Z\}, \{X, \neg Y\}$ is unsatisfiable.

1. $X \vee Y$ 2. $\neg X \vee Z$ 3. $\neg X \vee \neg Z$ 4. $X \vee \neg Y$

5. X (resolve 1, 4 on Y : $X \vee X = X$)

6. $\neg X$ (resolve 2, 3 on Z : $\neg X \vee \neg X = \neg X$)

7. $\{\}$ (resolve 5, 6). Unsatisfiable.

Q. 10. (Check Your Progress 4, Q2) KB1: If Mary goes to school, then Mary eats lunch. KB2: If it is Friday, then Mary goes to school or eats lunch. Prove by resolution: If it is Friday, then Mary eats lunch.

S: Mary goes to school, L: Mary eats lunch, F: it is Friday. KB1: $S \rightarrow L$, KB2: $F \rightarrow (S \vee L)$, conclusion $F \rightarrow L$.

Negated conclusion: $\neg(F \rightarrow L) \equiv F \wedge \neg L$.

1. $\neg S \vee L$ (KB1, MI) 2. $\neg F \vee S \vee L$ (KB2, MI) 3. F 4. $\neg L$

5. $S \vee L$ (resolve 2, 3)

6. L (resolve 5, 1 on S : $L \vee L = L$)

7. $\{\}$ (resolve 6, 4). Hence $F \rightarrow L$ follows.

UNIT 5

First Order Predicate Logic

"എല്ലാ മനുഷ്യരും മരിക്കും, രാമൻ മനുഷ്യനാണ്, അതിനാൽ രാമൻ മരിക്കും" എന്ന ലളിതമായ reasoning പോലും propositional logic-ന് ചെയ്യാൻ കഴിയില്ല. ഈ unit-ൽ predicates, quantifiers ($\forall$, $\exists$) ഉള്ള First Order Predicate Logic (FOPL), English വാക്യങ്ങളെ FOPL-ലേക്ക് മാറ്റുന്ന രീതി, prenex/clausal form, Skolemization, unification, resolution എന്നിവ പഠിക്കുന്നു.

Why FOPL? Limits of propositional logic

Predicates, universe of discourse and quantifiers

Symbols of FOPL: constants, variables, functions, predicates, terms, atoms

Interpretations: giving meaning to a formula

Scope, free and bound variables

Well-formed formulas; closed and open formulas

Translating English sentences into FOPL

Quantifier equivalences, negation, validity and consistency

Quantifier rules of inference: UI, UG, EI, EG

Proofs with the 12 inference rules; order of quantifiers; entailment

Prenex Normal Form (PNF)

Skolemization and (Skolem) Standard Form

Complete conversion to clausal form

Substitution, unification and the most general unifier

Resolution in FOPL: refutation proofs

Why FOPL? Limits of propositional logic

Definition: First Order Predicate Logic (FOPL) is an extension of propositional logic that splits atomic statements into predicates and their arguments and adds quantifiers ($\forall$, $\exists$).

Atomic statements-നെ predicates-ഉം അവയുടെ arguments-ഉം ആയി വിഭജിക്കുകയും quantifiers ($\forall$, $\exists$) ചേർക്കുകയും ചെയ്യുന്ന, propositional logic-ന്റെ ഒരു extension ആണ് First Order Predicate Logic (FOPL).

Consider the argument: "Every man is mortal. Raman is a man. Hence, Raman is mortal." In propositional logic (PL) we can only write P: Every man is mortal, Q: Raman is a man, R: Raman is mortal. The three symbols have no visible connection, so R cannot be derived from P and Q, even though the argument is obviously correct. First Order Predicate Logic (FOPL) fixes this by breaking an atomic statement into a **predicate** ("is mortal") and its **arguments** ("Raman"), and by adding **quantifiers** ("for all", "there exists").

മലയാളത്തിൽ

PL-ൽ ഓരോ വാക്യവും ഒരു അടഞ്ഞ പെട്ടി പോലെയാണ്: P, Q, R. പെട്ടിക്കുള്ളിൽ "Raman" ഉണ്ടെന്നോ "mortal" ഉണ്ടെന്നോ logic-ന് കാണാൻ പറ്റില്ല. FOPL ആ പെട്ടി തുറന്ന് ഉള്ളിലെ subject-ഉം predicate-ഉം വേർതിരിച്ച് കാണിക്കുന്നു:

MORTAL(Raman). അതോടൊപ്പം "എല്ലാവർക്കും" ($\forall$), "ഒരാളെങ്കിലും" ($\exists$) എന്ന് പറയാനുള്ള quantifiers-ഉം കിട്ടുന്നു. ഇതുകൊണ്ടാണ് AI-ൽ knowledge represent ചെയ്യാൻ FOPL ഇത്ര പ്രധാനമാകുന്നത്.

PL: $P, Q \vdash R$? (no link between P, Q and R, so no proof)

FOPL: $(\forall x)(\text{MAN}(x) \rightarrow \text{MORTAL}(x)), \text{MAN}(\text{Raman}) \vdash \text{MORTAL}(\text{Raman})$

EXAMPLE

Why can PL not prove "Raman is mortal" but can prove it if we are given "If Raman is a man then Raman is mortal"?

Given $\text{MAN}(\text{Raman}) \rightarrow \text{MORTAL}(\text{Raman})$ and $\text{MAN}(\text{Raman})$, PL's Modus Ponens gives $\text{MORTAL}(\text{Raman})$, because both are fixed statements about Raman. But "Every man is mortal" is a statement about all objects; PL has no way to produce the particular case for Raman from it. FOPL's Universal Instantiation rule does exactly this: from $(\forall x)(\text{MAN}(x) \rightarrow \text{MORTAL}(x))$ it derives $\text{MAN}(\text{Raman}) \rightarrow \text{MORTAL}(\text{Raman})$.

Exam tip: "Why is FOPL needed over PL?" is a common 3–5 mark question. Give the Raman argument, show that P, Q, R are unrelated in PL, and write the FOPL version.

Predicates, universe of discourse and quantifiers

Predicate: A sentence $p(x)$ containing a variable x ; it becomes a proposition only when x gets a definite value.

Predicate: ഒരു variable x അടങ്ങിയ $p(x)$ എന്ന sentence; x -ന് ഒരു നിശ്ചിത value കിട്ടുമ്പോൾ മാത്രമേ അത് proposition ആകൂ.

Universe of discourse: The set U of values the variable x may take.

Universe of discourse: x എന്ന variable-ന് എടുക്കാവുന്ന values-ന്റെ set U .

Quantifiers: $\forall$ ("for all"): $p(x)$ is true for every x in U ; $\exists$ ("there exists"): $p(x)$ is true for at least one x in U .

Quantifiers: $\forall$ ("for all"): U -ലെ എല്ലാ x -നും $p(x)$ ശരി; $\exists$ ("there exists"): U -ലെ കുറഞ്ഞത് ഒരു x -നെക്കിലും $p(x)$ ശരി.

A **proposition** is a statement that is true or false. A **predicate** (propositional function) $p(x)$ is a sentence containing a variable x ; it becomes a proposition only when x gets a definite value. The set of values x may take is the **universe of discourse** U . If $p(x)$ is " $x > 5$ ", then $p(6)$ is true and $p(0)$ is false, but $p(x)$ itself is neither. The **existential quantifier** $\exists$ means "there exists (at least one)" and the **universal quantifier** $\forall$ means "for all".

മലയാളത്തിൽ

" $x > 5$ " എന്നത് true-ഓ false-ഓ എന്ന് പറയാൻ പറ്റില്ല, കാരണം x എത്രയാണെന്ന് അറിയില്ല. അതുകൊണ്ട് ഇതൊരു predicate ആണ്, proposition അല്ല. $x = 6$ എന്ന് കൊടുത്താൽ അതൊരു true proposition ആകും. x -ന് എടുക്കാവുന്ന values-ന്റെ set ആണ് universe of discourse. ഒരു ക്ലാസ്സിലെ കുട്ടികളെക്കുറിച്ച് പറയുമ്പോൾ ആ ക്ലാസ്സിലെ കുട്ടികളാണ് U . "ഒരു കുട്ടിയെക്കിലും ഹാജരാണ്" എന്നത് $\exists$, "എല്ലാ കുട്ടികളും ഹാജരാണ്" എന്നത് $\forall$.

$(\exists x \in U) p(x)$: there is at least one x in U with $p(x)$ true

$(\forall x \in U) p(x)$: $p(x)$ is true for every x in U

$\neg(\exists x \in U) p(x) \equiv (\forall x \in U) \neg p(x)$

$\neg(\forall x \in U) p(x) \equiv (\exists x \in U) \neg p(x)$

EXAMPLE

State whether each is true or false: (a) $(\exists x \in \mathbb{R})(x + 1 > 0)$ (b) $(\exists x \in \mathbb{N})(x - 1/2 = 0)$ (c) $(\forall x \in \mathbb{N})(x^2 > x)$.

(a) True: $x = 5$ gives $6 > 0$. (b) False: the only solution is $x = 1/2$, which is not a natural number. (c) False: for $x = 1$, $x^2 = 1$, and $1 > 1$ is false. One counterexample is enough to make a $\forall$ statement false.

EXAMPLE

U = the children of a class, $p(x)$: "x is in the class today". Symbolize "There is at least one child in the class" and its negation.

Statement: $(\exists x \in U) p(x)$. Negation: "There is no child in the class" = $\neg(\exists x \in U) p(x) \equiv (\forall x \in U) \neg p(x)$, i.e. every child is absent.

Exam tip: The IGNOU text reads $(\forall x \in \mathbb{N})(x^2 > x)$ as "for every x not in $\mathbb{N}$ "; that is a misprint. It means "for every x in $\mathbb{N}$ ", and it is false because of $x = 1$.

Symbols of FOPL: constants, variables, functions, predicates, terms, atoms

Term: A variable, a constant, or $f(t_1, \dots, t_n)$ where f is an n -place function symbol and each t_i is a term; it denotes an object.

Term: ഒരു variable, ഒരു constant, അല്ലെങ്കിൽ f ഒരു n -place function symbol-ഉം ഓരോ t_i -ഉം term-ഉം ആയ $f(t_1, \dots, t_n)$; ഇത് ഒരു object-നെ സൂചിപ്പിക്കുന്നു.

Atom: An n -place predicate symbol applied to n terms, $P(t_1, \dots, t_n)$; it has a truth value.

Atom: ഒരു n -place predicate symbol n terms-ൽ പ്രയോഗിച്ചത്, $P(t_1, \dots, t_n)$; ഇതിന് ഒരു truth value ഉണ്ട്.

FOPL has four kinds of symbols. **Constants** name fixed objects (Ram, Mohan, 3). **Variables** are lowercase letters (x, y, z, x_3). **Function symbols** (f, g , father, product) map objects to objects. **Predicate symbols** (P, Q , LIKE, GREATER) map objects to True/False. An n -place symbol takes exactly n arguments. **Terms** denote objects; an **atom** is a predicate applied to terms and has a truth value.

മലയാളത്തിൽ

Function-ഉം predicate-ഉം തമ്മിലുള്ള വ്യത്യാസം ശ്രദ്ധിക്കണം. $\text{father}(\text{Mohan})$ ഒരു ആളെ (object) തരുന്നു: മോഹന്റെ അച്ഛൻ. അതിന് true/false ഇല്ല.

$\text{LOVE}(\text{father}(\text{Mohan}), \text{Mohan})$ ആകട്ടെ "മോഹന്റെ അച്ഛൻ മോഹനെ സ്നേഹിക്കുന്നു" എന്ന true/false പറയാവുന്ന atom ആണ്. അച്ഛന്റെ പേര് അറിയാതെ തന്നെ അദ്ദേഹത്തെക്കുറിച്ച് സംസാരിക്കാൻ function സഹായിക്കുന്നു. $\text{father}(\text{father}(\text{Mohan}))$ എന്നാൽ അപ്പൂപ്പൻ.

Term: (i) a variable is a term; (ii) a constant is a term;

(iii) if f is an n -place function symbol and $t_1, \dots, t_n$ are terms, then $f(t_1, \dots, t_n)$ is a term;

(iv) nothing else is a term.

Atom: a PL atom, or $P(t_1, \dots, t_n)$ where P is an n -place predicate symbol and t_i are terms.

EXAMPLE

Classify each: Ram, x , $\text{father}(\text{Mohan})$, $\text{plus}(\text{plus}(y, 3), y)$, $\text{LIKE}(\text{Ram}, \text{Mohan})$, $\text{GREATER}(5, 2)$, $\text{LOVE}(\text{father}(\text{Mohan}), \text{Mohan})$.

Constants: Ram. Variable: x . Terms (objects): Ram, x , $\text{father}(\text{Mohan})$, $\text{plus}(\text{plus}(y, 3), y) (= (y + 3) + y)$. Atoms (true/false): $\text{LIKE}(\text{Ram}, \text{Mohan})$, $\text{GREATER}(5, 2)$ (True), $\text{LOVE}(\text{father}(\text{Mohan}), \text{Mohan})$. father and plus are function symbols; LIKE , GREATER , LOVE are predicate symbols.

EXAMPLE

What is the "place" (arity) of father in $\text{father}(x)$ and of father_of in $\text{father_of}(x, y)$?

father is a 1-place function symbol (it returns a person). $\text{father_of}(x, y)$, " x is the father of y ", is a 2-place predicate symbol (it returns True/False). GREATER and LIKE are 2-place predicates; $\text{Between}(x, y, z)$ is 3-place.

Exam tip: A term never has a truth value; an atom always has one (once its variables get values). Writing $\text{father}(\text{Mohan})$ as a statement, or $\text{GREATER}(3, 5)$ as an object, loses marks.

Interpretations: giving meaning to a formula

Definition: An interpretation is a non-empty domain D with an element of D assigned to each constant, a function on D to each function symbol, and a relation on D to each predicate symbol.

ഒരു non-empty domain D -ഉം, ഓരോ constant-നും D -ലെ ഒരു element, ഓരോ function symbol-നും D -ൽ ഒരു function, ഓരോ predicate symbol-നും D -ൽ ഒരു relation എന്നീ assignments-ഉം ചേർന്നതാണ് interpretation.

An **interpretation** fixes a non-empty domain D , assigns an element of D to each constant, a function on D to each function symbol, and a relation on D (a True/False value for every argument list) to each predicate symbol. Under an interpretation, every closed formula becomes True or False. Parametric values such as 3 and 5 in greater-than(3, 5) are the **arguments** of the predicate. greater-than(3, y) is still a predicate; only after y gets a value does it become a statement.

മലയാളത്തിൽ

Formula എന്നത് വെറും symbols ആണ്. അതിന് അർത്ഥം കിട്ടുന്നത് interpretation കൊടുക്കുമ്പോഴാണ്: domain ഏതാണ്, ഓരോ constant ഏത് object ആണ്, ഓരോ predicate ഏത് objects-ന് true ആണ്. ഒരു formula ഒരു interpretation-ൽ true-ഉം വേറൊന്നിൽ false-ഉം ആകാം, നാടകത്തിലെ ഒരു dialogue വേറെ വേറെ കഥാപാത്രങ്ങൾ പറയുമ്പോൾ അർത്ഥം മാറുന്നത് പോലെ.

Valid: true under every interpretation

Consistent (satisfiable): true under at least one interpretation

Inconsistent (unsatisfiable): false under every interpretation

EXAMPLE

$D = \{1, 2\}$, $P(1) = T$, $P(2) = F$. Evaluate $(\forall x)P(x)$, $(\exists x)P(x)$, $(\exists x)\neg P(x)$.

$(\forall x)P(x) = P(1) \wedge P(2) = T \wedge F = F$. $(\exists x)P(x) = P(1) \vee P(2) = T$. $(\exists x)\neg P(x) = \neg P(1) \vee \neg P(2) = F \vee T = T$. Over a finite domain, $\forall$ behaves like a big $\wedge$ and $\exists$ like a big $\vee$.

EXAMPLE

Domain: integers; GREATER(x, y) means $x > y$. Evaluate GREATER(3, 5), $(\forall x)(\exists y)$ GREATER(y, x), $(\exists y)(\forall x)$ GREATER(y, x).

GREATER(3, 5): $3 > 5$ is False. $(\forall x)(\exists y)$ GREATER(y, x): for every x take $y = x + 1$, so True. $(\exists y)(\forall x)$ GREATER(y, x): a single integer larger than all integers, including itself? No, so False.

Scope, free and bound variables

Scope: The smallest complete formula that follows a quantifier.

Scope: ഒരു quantifier-ന് തൊട്ടുപിന്നാലെ വരുന്ന ഏറ്റവും ചെറിയ പൂർണ്ണ formula.

Bound occurrence: An occurrence of a variable inside the scope of a quantifier on that variable (or in the quantifier itself).

Bound occurrence: ആ variable-ന്റെ തന്നെ quantifier-ന്റെ scope-നുള്ളിൽ (അല്ലെങ്കിൽ quantifier-ൽ തന്നെ) വരുന്ന variable occurrence.

Free occurrence: An occurrence of a variable that is not bound.

Free occurrence: Bound അല്ലാത്ത variable occurrence.

The **scope** of a quantifier is the smallest complete formula that follows it (usually the bracketed part). An occurrence of a variable is **bound** if it is inside the scope of a quantifier on that variable (or in the quantifier itself); otherwise it is **free**. A variable is free in a formula if at least one occurrence is free, and bound if at least one occurrence is bound, so a variable can be both. Quantifiers are taken to have the least possible scope: $(\forall x)P(x, y) \rightarrow Q(x)$ means $((\forall x)P(x, y)) \rightarrow Q(x)$.

മലയാളത്തിൽ

ഒരു quantifier-ന്റെ "അധികാര പരിധി" ആണ് scope, ഒരു പഞ്ചായത്തിന്റെ അതിർത്തി പോലെ. അതിർത്തിക്കുള്ളിലുള്ള x-കൾ ആ quantifier-ന് "bound" ആണ്; പുറത്തുള്ള x ആരുടെയും നിയന്ത്രണത്തിലല്ല, അതാണ് "free". Brackets ഇല്ലെങ്കിൽ quantifier തൊട്ടടുത്തുള്ള ഏറ്റവും ചെറിയ formula-യെ മാത്രമേ ഭരിക്കൂ എന്ന് ഓർക്കുക. Bound variable ഒരു dummy മാത്രമാണ്: $\int x \, dx = \int y \, dy$ ആയതുപോലെ $(\forall x)f(x) = (\forall y)f(y)$. ഇതിനെ renaming എന്ന് പറയുന്നു.

$(\forall x) Q(x) \rightarrow P(x, y)$: scope of $(\forall x)$ is $Q(x)$; x occurs 3 times

$(\forall x)(Q(x) \rightarrow P(x, y))$: scope of $(\forall x)$ is $(Q(x) \rightarrow P(x, y))$

Renaming: $(\forall x) f(x) \equiv (\forall y) f(y)$, if y does not already occur in the formula

EXAMPLE

In $(\forall x) P(x, y) \rightarrow Q(x)$, mark every occurrence as free or bound.

Scope of $(\forall x)$ is $P(x, y)$. The x in $(\forall x)$ and the x in $P(x, y)$ are bound. The x in $Q(x)$ lies outside the scope, so it is free. The only y is free. So x is both free and bound; y is free only.

EXAMPLE

In $(\forall x)(P(x, y) \rightarrow Q(x))$, find the scope and classify occurrences.

The scope of $(\forall x)$ is $(P(x, y) \rightarrow Q(x))$. All three occurrences of x are bound; the only y is free. Compare with the previous example: only the brackets changed, but the free x disappeared.

EXAMPLE

Find the scopes in $(\forall x)((P(x) \rightarrow Q(x)) \wedge (\exists y)(Q(x) \rightarrow R(y)))$ and in $(\forall x)(P(x) \rightarrow Q(x, y)) \vee (\forall x)(P(x) \rightarrow R(x, 3))$.

First formula: scope of $(\exists y)$ is $(Q(x) \rightarrow R(y))$; scope of $(\forall x)$ is the whole rest, $(P(x) \rightarrow Q(x)) \wedge (\exists y)(Q(x) \rightarrow R(y))$. Second formula: scope of the first $(\forall x)$ is $(P(x) \rightarrow Q(x, y))$; scope of the second $(\forall x)$ is $(P(x) \rightarrow R(x, 3))$. The two x 's belong to different quantifiers.

Exam tip: When asked to "identify free and bound occurrences", count occurrences one by one, including the one inside the quantifier $(\forall x)$ itself, as IGNOU does ("x has three occurrences").

Well-formed formulas; closed and open formulas

Well-formed formula (wff): Atoms are wffs; if E, G are wffs, so are $\neg E, (E \wedge G), (E \vee G), (E \rightarrow G), (E \leftrightarrow G)$, and $(\forall x)E, (\exists x)E$ for x free in E ; nothing else.

Well-formed formula (wff): Atoms wffs ആണ്; E, G wffs ആണെങ്കിൽ $\neg E, (E \wedge G), (E \vee G), (E \rightarrow G), (E \leftrightarrow G)$ എന്നിവയും, x, E -ൽ free ആണെങ്കിൽ $(\forall x)E, (\exists x)E$ എന്നിവയും wffs ആണ്; മറ്റൊന്നുമല്ല.

Closed formula: A wff with no free occurrence of any variable (a sentence); it has a truth value.

Closed formula: ഒരു variable-ന്റെയും free occurrence ഇല്ലാത്ത wff (ഒരു sentence); ഇതിന് truth value ഉണ്ട്.

Open formula: A wff with at least one free occurrence of a variable; it has no truth value.

Open formula: കുറഞ്ഞത് ഒരു variable-ന്റെയെങ്കിലും free occurrence ഉള്ള wff; ഇതിന് truth value ഇല്ല.

A **well-formed formula (wff)** is built recursively: every atom is a wff; if E and G are wffs then $\neg E, (E \wedge G), (E \vee G), (E \rightarrow G), (E \leftrightarrow G)$ are wffs; if E is a wff and x is a free variable in E then $(\forall x)E$ and $(\exists x)E$ are wffs; nothing else is a wff. A wff with no free occurrence of any variable is **closed** (a sentence) and can be called true, false, valid, consistent and so on. A wff with at least one free occurrence is **open**, and cannot be given a truth value.

മലയാളത്തിൽ

Closed formula എന്നാൽ എല്ലാ variables-ഉം ഏതെങ്കിലും quantifier-ന്റെ കീഴിലാണ്; അപ്പോൾ അത് ഒരു പൂർണ്ണ വാക്യമാണ്, true-ഓ false-ഓ പറയാം. Open formula-യിൽ ഒരു free variable എങ്കിലും ഉണ്ട്; "അവൻ ഉയരമുള്ളവനാണ്" എന്ന് പറയുമ്പോൾ "അവൻ" ആരാണെന്ന് അറിയാതെ ശരിയോ തെറ്റോ പറയാൻ പറ്റാത്തത് പോലെ.

Open: $\text{greater}(x, y)$, $\text{greater}(x, 3)$, $(\forall y) \text{greater}(x, y)$ (x is free)

Closed: $(\forall x)(\exists y) \text{greater}(x, y)$, $(\exists y) \text{greater}(y, 1)$, $\text{greater}(9, 2)$

EXAMPLE

Symbolize with IGNOU's notation: (i) There exists a number that is rational. (ii) Every rational number is a real number. (iii) For every number x there exists a number y which is greater than x . Use $Q(x)$: x is rational, $R(x)$: x is real, $\text{LESS}(x, y)$: x is less than y .

(i) $(\exists x) Q(x)$. (ii) $(\forall x)(Q(x) \rightarrow R(x))$. (iii) $(\forall x)(\exists y) \text{LESS}(x, y)$. Each is a closed wff.

EXAMPLE

Symbolize the axioms of natural numbers. $f(x)$: successor of x , $g(x)$: predecessor of x , $E(x, y)$: x equals y . (1) Every number has exactly one immediate successor. (2) 0 is the successor of no number. (3) Every number other than 0 has exactly one immediate predecessor.

(1) $(\forall x)(\exists y)(E(y, f(x)) \wedge (\forall z)(E(z, f(x)) \rightarrow E(y, z)))$

(2) $\neg(\exists x) E(0, f(x))$

(3) $(\forall x)(\neg E(x, 0) \rightarrow (\exists y)(E(y, g(x)) \wedge (\forall z)(E(z, g(x)) \rightarrow E(y, z))))$

The pattern " $E(y, \dots) \wedge (\forall z)(E(z, \dots) \rightarrow E(y, z))$ " is the standard way to say "exactly one".

Translating English sentences into FOPL

First choose predicate names, then find the quantifier from the English ("every, all, any" $\rightarrow \forall$; "some, a, there is, at least one" $\rightarrow \exists$). The golden rule: **$\forall$ goes with $\rightarrow$** and **$\exists$ goes with $\wedge$** . "Every P is Q " is $(\forall x)(P(x) \rightarrow Q(x))$; writing $(\forall x)(P(x) \wedge Q(x))$ would claim that everything in the universe is P . "Some P is Q " is $(\exists x)(P(x) \wedge Q(x))$; writing $(\exists x)(P(x) \rightarrow Q(x))$ is almost always true (any non- P object makes it true), so it says nothing useful.

മലയാളത്തിൽ

ഏറ്റവും കൂടുതൽ മാർക്ക് പോകുന്നത് ഇവിടെയാണ്. ഓർമ്മിക്കാൻ ഒരു വഴി: "എല്ലാ" ($\forall$) വന്നാൽ "ആണെങ്കിൽ" ($\rightarrow$), "ചില" ($\exists$) വന്നാൽ "ഉം" ($\wedge$). "എല്ലാ വിദ്യാർത്ഥികളും മിടുക്കരാണ്" എന്നാൽ "ഒരാൾ വിദ്യാർത്ഥി ആണെങ്കിൽ അയാൾ മിടുക്കനാണ്". "ചില വിദ്യാർത്ഥികൾ മിടുക്കരാണ്" എന്നാൽ "വിദ്യാർത്ഥിയും മിടുക്കനുമായ ഒരാൾ ഉണ്ട്". Quantifiers-ന്റെ order മാറിയാൽ അർത്ഥം പൂർണ്ണമായി മാറും: "എല്ലാവരും ആരെയെങ്കിലും സ്നേഹിക്കുന്നു" $\neq$ "എല്ലാവരും സ്നേഹിക്കുന്ന ഒരാളുണ്ട്".

English pattern	FOPL
All P are Q	$(\forall x)(P(x) \rightarrow Q(x))$
Some P are Q	$(\exists x)(P(x) \wedge Q(x))$
No P is Q	$(\forall x)(P(x) \rightarrow \neg Q(x)) \equiv \neg(\exists x)(P(x) \wedge Q(x))$
Not all P are Q	$\neg(\forall x)(P(x) \rightarrow Q(x)) \equiv (\exists x)(P(x) \wedge \neg Q(x))$
Only P are Q	$(\forall x)(Q(x) \rightarrow P(x))$
Some P are not Q	$(\exists x)(P(x) \wedge \neg Q(x))$

EXAMPLE

Translate the Raman argument into a single FOPL statement.

MAN(x): x is a man; MORTAL(x): x is mortal.

Every man is mortal: $(\forall x)(MAN(x) \rightarrow MORTAL(x))$. Raman is a man: MAN(Raman).

Argument: $(\forall x)(MAN(x) \rightarrow MORTAL(x)) \wedge MAN(Raman) \rightarrow MORTAL(Raman)$.

EXAMPLE (ONE-PLACE PREDICATES)

Translate: (a) All students are clever. (b) Some students are clever. (c) No student is lazy. (d) Not all birds can fly. (e) Only teachers can enter the staff room. (f) Every student who studies passes.

(a) $(\forall x)(Student(x) \rightarrow Clever(x))$

(b) $(\exists x)(Student(x) \wedge Clever(x))$

(c) $(\forall x)(Student(x) \rightarrow \neg Lazy(x))$, or equivalently $\neg(\exists x)(Student(x) \wedge Lazy(x))$

(d) $\neg(\forall x)(Bird(x) \rightarrow Fly(x)) \equiv (\exists x)(Bird(x) \wedge \neg Fly(x))$

(e) EnterStaffRoom(x): x can enter the staff room. $(\forall x)(EnterStaffRoom(x) \rightarrow Teacher(x))$. "Only teachers" puts Teacher on the right of $\rightarrow$.

(f) $(\forall x)(Student(x) \wedge Studies(x) \rightarrow Passes(x))$

EXAMPLE (TWO-PLACE PREDICATES, NESTED QUANTIFIERS)

Loves(x, y): x loves y. Translate: (a) Everyone loves someone. (b) There is someone whom everyone loves. (c) Someone loves everyone. (d) Nobody loves everyone. (e) Everyone loves himself.

- (a) $(\forall x)(\exists y) \text{ Loves}(x, y)$ (y may be different for each x)
- (b) $(\exists y)(\forall x) \text{ Loves}(x, y)$ (one fixed y for all x)
- (c) $(\exists x)(\forall y) \text{ Loves}(x, y)$
- (d) $\neg(\exists x)(\forall y) \text{ Loves}(x, y) \equiv (\forall x)(\exists y) \neg \text{Loves}(x, y)$
- (e) $(\forall x) \text{ Loves}(x, x)$

EXAMPLE (QUANTIFIER INSIDE THE CONSEQUENT)

Translate: (a) Every child has a mother. (b) Anyone who has a friend is happy. (c) Lord Krishna is loved by everyone who loves someone. (d) No one loves nobody.

- (a) Mother(y, x): y is the mother of x. $(\forall x)(\text{Child}(x) \rightarrow (\exists y) \text{ Mother}(y, x))$.
- (b) $(\forall x)((\exists y) \text{ Friend}(x, y) \rightarrow \text{Happy}(x))$. Moving the $\exists$ out of the antecedent turns it into $\forall$: $\equiv (\forall x)(\forall y) (\text{Friend}(x, y) \rightarrow \text{Happy}(x))$.
- (c) $(\forall x)((\exists y) \text{ Love}(x, y) \rightarrow \text{Love}(x, \text{LK}))$.
- (d) "No one is such that he loves nobody": $\neg(\exists x)(\forall y) \neg \text{Love}(x, y) \equiv (\forall x)(\exists y) \text{ Love}(x, y)$, i.e. everyone loves someone.

EXAMPLE (FUNCTIONS AND "EXACTLY ONE")

Translate: (a) Ravi's father loves Ravi. (b) Every person's grandfather is older than the person. (c) Kerala has exactly one capital.

- (a) $\text{Love}(\text{father}(\text{Ravi}), \text{Ravi})$.
- (b) $(\forall x)(\text{Person}(x) \rightarrow \text{Older}(\text{father}(\text{father}(x)), x))$ (paternal grandfather, using the function father).
- (c) $\text{Capital}(x, y)$: x is the capital of y. $(\exists x)(\text{Capital}(x, \text{Kerala}) \wedge (\forall y)(\text{Capital}(y, \text{Kerala}) \rightarrow y = x))$.

EXAMPLE (IGNOU: RATIONAL AND REAL NUMBERS)

P(x): x is a rational number, Q(x): x is a real number. Symbolize: (i) Every rational number is a real number. (ii) Some real numbers are rational numbers. (iii) Not every real number is a rational number.

- (i) $(\forall x)(P(x) \rightarrow Q(x))$
- (ii) $(\exists x)(Q(x) \wedge P(x))$
- (iii) $\neg(\forall x)(Q(x) \rightarrow P(x)) \equiv (\exists x)(Q(x) \wedge \neg P(x))$

Exam tip: Before writing, check two things: (1) $\forall$ with $\rightarrow$, $\exists$ with $\wedge$; (2) the order of mixed quantifiers. Then read your formula back into English word by word. If it says something stronger or weaker than the question, fix it.

Quantifier equivalences, negation, validity and consistency

Valid: A formula that is true under every interpretation.

Valid: എല്ലാ interpretations-ലും ശരിയാകുന്ന formula.

Consistent (satisfiable): A formula that is true under at least one interpretation.

Consistent (satisfiable): കുറഞ്ഞത് ഒരു interpretation-ൽ എങ്കിലും ശരിയാകുന്ന formula.

Inconsistent (unsatisfiable): A formula that is false under every interpretation.

Inconsistent (unsatisfiable): എല്ലാ interpretations-ലും തെറ്റാകുന്ന formula.

$\forall$ distributes over $\wedge$, and $\exists$ distributes over $\vee$. The other two combinations do not hold in general. Negation flips the quantifier and moves inside. Quantifiers of the same kind commute ($(\forall x)(\forall y) \equiv (\forall y)(\forall x)$); $\forall$ and $\exists$ do not. A closed formula can be shown inconsistent or valid by reducing it to the PL patterns $Q \wedge \neg Q$ (always F) or $Q \vee \neg Q$ (always T).

മലയാളത്തിൽ

"എല്ലാവരും പാടും, എല്ലാവരും ആടും" എന്നത് "എല്ലാവരും പാടുകയും ആടുകയും ചെയ്യും" എന്നതിന് തുല്യം: $\forall$ എന്നത് $\wedge$ -ന് മുകളിൽ വിതരണം ചെയ്യാം. എന്നാൽ "ഓരോ സംഖ്യയും odd അല്ലെങ്കിൽ even ആണ്" എന്നത് ശരി, "എല്ലാ സംഖ്യകളും odd, അല്ലെങ്കിൽ എല്ലാ സംഖ്യകളും even" എന്നത് തെറ്റ്. അതുകൊണ്ട് $\forall$ എന്നത് $\vee$ -ന് മുകളിൽ വിതരണം ചെയ്യാൻ പറ്റില്ല. Negation ഉള്ളിലേക്ക് കടക്കുമ്പോൾ $\forall \exists$ ആയും $\exists \forall$ ആയും മാറും.

- (i) $(\forall x)P(x) \wedge (\forall x)Q(x) \equiv (\forall x)(P(x) \wedge Q(x))$
- (ii) $(\exists x)P(x) \vee (\exists x)Q(x) \equiv (\exists x)(P(x) \vee Q(x))$
- (iii) $(\forall x)(P(x) \vee Q(x)) \not\equiv (\forall x)P(x) \vee (\forall x)Q(x)$ (only $\Leftarrow$ holds)
- (iv) $(\exists x)(P(x) \wedge Q(x)) \not\equiv (\exists x)P(x) \wedge (\exists x)Q(x)$ (only $\Rightarrow$ holds)
- (v) $\neg(\forall x)P(x) \equiv (\exists x)\neg P(x)$
- (vi) $\neg(\exists x)P(x) \equiv (\forall x)\neg P(x)$

EXAMPLE

Show (iii) and (iv) fail, with $P(x)$: x is an odd natural number, $Q(x)$: x is an even natural number.

(iii) LHS: every natural number is odd or even, True. RHS: every natural number is odd, or every natural number is even, False. So $LHS \neq RHS$.

(iv) LHS: some natural number is both odd and even, False. RHS: some number is odd and some number is even, True. So $LHS \neq RHS$.

EXAMPLE

Prove $(\exists x)P(x) \wedge (\forall y)\neg P(y)$ is inconsistent.

$(\forall y)\neg P(y) \equiv \neg(\exists y)P(y)$ (rule vi). Rename the bound y as x : $\neg(\exists x)P(x)$. The formula becomes $(\exists x)P(x) \wedge \neg(\exists x)P(x)$. $(\exists x)P(x)$ is closed, so it is simply T or F; call it Q . We get $Q \wedge \neg Q$, which is False in every interpretation. Hence inconsistent.

EXAMPLE

Prove $(\forall x)P(x) \rightarrow (\exists y)P(y)$ is valid.

Remove $\rightarrow$: $\neg(\forall x)P(x) \vee (\exists y)P(y) \equiv (\exists x)\neg P(x) \vee (\exists y)P(y) \equiv (\exists x)\neg P(x) \vee (\exists x)P(x)$ (renaming) $\equiv (\exists x)(\neg P(x) \vee P(x))$ (rule ii). Since the domain is non-empty, pick any element b : $\neg P(b) \vee P(b)$ is True. So the formula is True in every interpretation, i.e. valid.

EXAMPLE (CHECK YOUR PROGRESS 1, EX. 3)

Prove: (i) $P(a) \rightarrow \neg((\forall x)P(x))$ is consistent. (ii) $(\forall x)P(x) \vee (\exists y)\neg P(y)$ is valid.

(i) Remove $\rightarrow$: $\neg P(a) \vee \neg(\forall x)P(x) \equiv \neg P(a) \vee (\exists x)\neg P(x)$. Take an interpretation where $P(a)$ is F; then $\neg P(a)$ is T and the whole formula is T. It is true in at least one interpretation, so it is consistent. (It is not valid: if P is true of everything, both disjuncts are F.)

(ii) $(\exists y)\neg P(y) \equiv \neg(\forall y)P(y) \equiv \neg(\forall x)P(x)$. With $Q = (\forall x)P(x)$ the formula is $Q \vee \neg Q$, always True. Hence valid.

Exam tip: To show a claimed equivalence is false, one counterexample (like odd/even numbers) is enough. To show validity or inconsistency, reduce to $Q \vee \neg Q$ or $Q \wedge \neg Q$ as above.

Quantifier rules of inference: UI, UG, EI, EG

Instantiation (UI, EI): Removing a quantifier by replacing its variable with a constant: UI allows any constant, EI requires a new constant not used before.

Instantiation (UI, EI): Quantifier നീക്കി അതിന്റെ variable-ന് പകരം ഒരു constant വെക്കൽ: UI-യിൽ ഏത് constant-ഉം ആകാം, EI-യിൽ മുമ്പ് ഉപയോഗിക്കാത്ത ഒരു പുതിയ constant വേണം.

Generalization (UG, EG): Adding a quantifier: UG infers $(\forall x) P(x)$ from $P(a)$ for an arbitrary a ; EG infers $(\exists x) P(x)$ from $P(a)$.

Generalization (UG, EG): ഒരു quantifier ചേർക്കൽ: UG ഒരു arbitrary a -ക്കുള്ള $P(a)$ -ൽ നിന്ന് $(\forall x) P(x)$ infer ചെയ്യുന്നു; EG, $P(a)$ -ൽ നിന്ന് $(\exists x) P(x)$ infer ചെയ്യുന്നു.

The 8 inference rules of PL (Modus Ponens, simplification, conjunction, and so on) work on quantifier-free formulas. Four quantifier rules connect FOPL formulas to PL formulas and back, giving 12 rules in all. **UI** and **EI** remove quantifiers; **UG** and **EG** add them. UG and EI must be used with great care; UI and EG can be used freely.

മലയാളത്തിൽ

UI: "എല്ലാവർക്കും ബാധകം" എങ്കിൽ ആർക്കു വേണമെങ്കിലും, രാമനോ കൃഷ്ണനോ, ബാധകം. EI: "ഒരാളുണ്ട്" എന്ന് അറിയാം, പക്ഷേ ആരാണെന്ന് അറിയില്ല; അതിനാൽ അയാൾക്ക് ഒരു പുതിയ പേര് ($a, b, \dots$) കൊടുക്കണം, നേരത്തെ ഉപയോഗിച്ച പേര് പാടില്ല. EG: രാമന് ബാധകമാണെങ്കിൽ "ഒരാൾക്കെങ്കിലും ബാധകം". UG: ഏതൊരാൾക്കും (arbitrary) തെളിയിച്ചാൽ മാത്രം "എല്ലാവർക്കും" എന്ന് പറയാം.

UI: $(\forall x) P(x) \vdash P(a)$, a any constant

UG: $P(a)$ for all (arbitrary) $a \vdash (\forall x) P(x)$

EI: $(\exists x) P(x) \vdash P(a)$, a a NEW constant not used before

EG: $P(a) \vdash (\exists x) P(x)$

EXAMPLE (CORRECT OR NOT?)

(i) From $(\forall x)(F(x) \wedge G(x)) \rightarrow H(x) \vee I(x)$ conclude $F(a) \wedge G(a) \rightarrow H(a) \vee I(a)$ by UI.

Incorrect. The scope of $(\forall x)$ is only $(F(x) \wedge G(x))$; the x 's in $H(x) \vee I(x)$ are free. A correct UI result is $F(a) \wedge G(a) \rightarrow H(x) \vee I(x)$.

EXAMPLE (CORRECT OR NOT?)

(ii) From $(\forall x)(F(x) \wedge G(x) \rightarrow H(x) \vee I(x))$ conclude $F(a) \wedge G(a) \rightarrow H(a) \vee I(a)$ by UI.

Correct. Now the brackets make the whole implication the scope of $(\forall x)$, so every x is replaced by a .

EXAMPLE (CORRECT OR NOT?)

(iii) From $\neg(\forall x)F(x)$ conclude $\neg F(a)$ for an arbitrary a by UI.

Incorrect. $\neg(\forall x)F(x) \equiv (\exists x)\neg F(x)$, which is an existential statement. The right rule is EI, and it only gives $\neg F(c)$ for some new constant c , not for a pre-chosen or arbitrary a .

EXAMPLE (CORRECT OR NOT?)

(iv) From $(\exists x)((F(b) \wedge G(x)) \rightarrow H(c))$ conclude $(F(b) \wedge G(b)) \rightarrow H(c)$ by EI.

Incorrect: b already appears in the formula, and EI must use a new constant. Concluding $(F(b) \wedge G(a)) \rightarrow H(c)$, with a new, is correct.

Exam tip: When a proof has both an $\exists$ premise and a $\forall$ premise, apply EI first (introducing a new constant a) and then UI with that same a . Doing UI first and then EI with the same constant is the classic wrong step.

Proofs with the 12 inference rules; order of quantifiers; entailment

Entailment: A conclusion is entailed by (is a logical consequence of) premises if it must be true whenever all the premises are true.

Entailment: Premises എല്ലാം ശരിയാകുമ്പോഴെല്ലാം ഒരു conclusion നിർബന്ധമായും ശരിയാണെങ്കിൽ, ആ conclusion premises-ൽ നിന്ന് entailed ആണ് (logical consequence ആണ്).

Valid argument: An argument whose conclusion is entailed by its premises.

Valid argument: Conclusion അതിന്റെ premises-ൽ നിന്ന് entailed ആയ argument.

Using FOPL for reasoning takes three steps: (1) **Conceptualisation:** list the relevant objects and relations, including implicit facts such as "a person dead once is dead forever"; (2) **Nomenclature and translation:** name predicates and translate English into FOPL; (3) find a sequence of rule applications that reaches the conclusion. A conclusion is **entailed** (is a logical consequence) by premises if it must be true whenever all the premises are true; an argument is valid exactly when its conclusion is entailed.

മലയാളത്തിൽ

ഒരു proof എഴുതുന്നത് ഒരു കണക്ക് steps ആയി ചെയ്യുന്നത് പോലെയാണ്: ആദ്യം English-നെ FOPL ആക്കുക, പിന്നെ EI/UI ഉപയോഗിച്ച് quantifiers മാറ്റി PL formulas ആക്കുക, PL rules (Modus Ponens, simplification, conjunction) കൊണ്ട് മുന്നോട്ട് പോകുക, അവസാനം EG/UG കൊണ്ട് quantifier തിരികെ ചേർക്കുക. ഓരോ വരിയുടെ അരികിലും ഏത് rule, ഏത് വരികളിൽ നിന്ന് എന്ന് എഴുതണം.

EXAMPLE (IGNOU: CAR REPAIR)

(i) Anyone who repairs his own car is highly skilled and saves a lot of money on repairs. (ii) Some people who repair their own cars have menial jobs. Therefore (iii) some people with menial jobs are highly skilled.
 $R(x)$: x repairs his own car, $H(x)$: highly skilled, $S(x)$: saves money, $M(x)$: has a menial job.

1. $(\forall x)(R(x) \rightarrow H(x) \wedge S(x))$ premise
2. $(\exists x)(R(x) \wedge M(x))$ premise
3. $R(a) \wedge M(a)$ EI on 2 (new constant a)
4. $R(a)$ simplification, 3
5. $R(a) \rightarrow H(a) \wedge S(a)$ UI on 1 with a
6. $H(a) \wedge S(a)$ Modus Ponens, 4, 5
7. $H(a)$ simplification, 6
8. $M(a)$ simplification, 3
9. $M(a) \wedge H(a)$ conjunction, 8, 7
10. $(\exists x)(M(x) \wedge H(x))$ EG on 9. Conclusion (iii) proved.

EXAMPLE (IGNOU: JUVENILES)

(i) Some juveniles who commit minor offences are thrown into prison, and any juvenile thrown into prison is exposed to hardened criminals. (ii) A juvenile exposed to hardened criminals becomes bitter and learns more crime techniques. (iii) Anyone who learns more crime techniques is a menace to society if he is bitter. Therefore (iv) some juveniles who commit minor offences are menaces to society. J, C, P, E, B, T, M: juvenile, commits minor offences, thrown into prison, exposed, bitter, learns techniques, menace.

1. $(\exists x)(J(x) \wedge C(x) \wedge P(x)) \wedge (\forall y)(J(y) \wedge P(y) \rightarrow E(y))$ premise (i)
2. $(\forall x)(J(x) \wedge E(x) \rightarrow B(x) \wedge T(x))$ premise (ii)
3. $(\forall x)(T(x) \wedge B(x) \rightarrow M(x))$ premise (iii)
4. $(\exists x)(J(x) \wedge C(x) \wedge P(x))$ simplification, 1
5. $(\forall y)(J(y) \wedge P(y) \rightarrow E(y))$ simplification, 1
6. $J(b) \wedge C(b) \wedge P(b)$ EI on 4 (new b)
7. $J(b)$, 8. $C(b)$, 9. $P(b)$ simplification, 6
10. $J(b) \wedge P(b) \rightarrow E(b)$ UI on 5
11. $E(b)$ Modus Ponens, (7 $\wedge$ 9), 10
12. $J(b) \wedge E(b) \rightarrow B(b) \wedge T(b)$ UI on 2
13. $B(b) \wedge T(b)$ Modus Ponens, (7 $\wedge$ 11), 12
14. $T(b) \wedge B(b) \rightarrow M(b)$ UI on 3
15. $M(b)$ Modus Ponens, 13, 14
16. $J(b) \wedge C(b) \wedge M(b)$ conjunction, 7, 8, 15
17. $(\exists x)(J(x) \wedge C(x) \wedge M(x))$ EG on 16.

EXAMPLE (ORDER OF QUANTIFIERS)

$F(x, y)$: $y > x$, over the real numbers. Compare $(\forall x)(\exists y)F(x, y)$ and $(\exists y)(\forall x)F(x, y)$.

$(\forall x)(\exists y) y > x$: for each x pick $y = x + 1$. True. Here y depends on x .

$(\exists y)(\forall x) y > x$: one fixed y bigger than every real, including y itself. False.

So $(Q_1x)(Q_2y) \neq (Q_2y)(Q_1x)$ in general when $Q_1 \neq Q_2$. This dependence of y on x is exactly what a Skolem function records later.

Exam tip: The IGNOU text writes "any juvenile thrown into prison is exposed" as $(\forall y)(J(y) \rightarrow E(y))$, dropping "thrown into prison". The faithful translation is $(\forall y)(J(y) \wedge P(y) \rightarrow E(y))$; the proof still works because $P(b)$ is available. The text also cites "(vii) and (xi)" for Modus Ponens where it should be (viii) $J(b)$ and (xi).

Prenex Normal Form (PNF)

Definition: A formula is in Prenex Normal Form if it has the form $(Q_1x_1)\dots(Q_nx_n) M$, where each Q_i is $\forall$ or $\exists$ (the prefix) and the matrix M is quantifier-free.

$(Q_1x_1)\dots(Q_nx_n) M$ എന്ന രൂപത്തിലുള്ള formula ആണ് Prenex Normal Form-ൽ ഉള്ളത്; ഇതിൽ ഓരോ Q_i -ഉം $\forall$ അല്ലെങ്കിൽ $\exists$ ആണ് (prefix), matrix M -ൽ quantifiers ഇല്ല.

A formula is in **prenex normal form** if it looks like $(Q_1x_1)\dots(Q_nx_n) M$, where each Q_i is $\forall$ or $\exists$ and M is quantifier-free. The quantifier string is the **prefix** and M is the **matrix**. All quantifiers are pulled to the front using the laws below, where G contains no free x .

മലയാളത്തിൽ

PNF എന്നാൽ എല്ലാ quantifiers-ഉം formula-യുടെ തുടക്കത്തിൽ വരിയായി നിൽക്കുന്ന രൂപം: മുന്നിൽ prefix, പിന്നിൽ quantifier ഇല്ലാത്ത matrix. ഒരു കല്യാണ ഫോട്ടോയിൽ എല്ലാവരെയും മുൻനിരയിലേക്ക് മാറ്റി നിർത്തുന്നത് പോലെ. പക്ഷേ മാറ്റുമ്പോൾ രണ്ട് പേർക്ക് ഒരേ പേര് (x) ഉണ്ടെങ്കിൽ ആശയക്കുഴപ്പം വരും, അതിനാൽ ആദ്യം ഒരാളുടെ പേര് മാറ്റണം (rename). Quantifiers-ന്റെ ക്രമം മാറ്റരുത്.

$(Qx)P[x] \vee G \equiv (Qx)(P[x] \vee G)$; $(Qx)P[x] \wedge G \equiv (Qx)(P[x] \wedge G)$ (G has no free x)

$\neg(\forall x)P[x] \equiv (\exists x)\neg P[x]$; $\neg(\exists x)P[x] \equiv (\forall x)\neg P[x]$

$(\forall x)P[x] \wedge (\forall x)H[x] \equiv (\forall x)(P[x] \wedge H[x])$; $(\exists x)P[x] \vee (\exists x)H[x] \equiv (\exists x)(P[x] \vee H[x])$

Otherwise rename: $(Q_1x)P[x] \circ (Q_2x)H[x] \equiv (Q_1x)(Q_2z)(P[x] \circ H[z])$, $\circ = \wedge$ or $\vee$

Steps: 1. Remove $\leftrightarrow$ ($P \leftrightarrow G \equiv (P \rightarrow G) \wedge (G \rightarrow P)$) and $\rightarrow$ ($P \rightarrow G \equiv \neg P \vee G$)

2. Remove double negation $\neg\neg P \equiv P$

3. Move $\neg$ in front of atoms (De Morgan and the quantifier negation laws)

4. Rename bound variables if necessary

5. Pull quantifiers to the front, keeping their order

EXAMPLE (IGNOU I)

PNF of $(\forall x)(Q(x) \rightarrow (\exists x)R(x, y))$.

Step 1 (remove $\rightarrow$): $(\forall x)(\neg Q(x) \vee (\exists x)R(x, y))$.

Step 2 (rename the inner x as z): $(\forall x)(\neg Q(x) \vee (\exists z)R(z, y))$.

Step 3 ($\neg Q(x)$ has no z , pull $\exists z$ out): $(\forall x)(\exists z)(\neg Q(x) \vee R(z, y))$. y remains free.

EXAMPLE (IGNOU II)

PNF of $(\forall x)(\neg(\exists y)Q(x, y) \rightarrow ((\exists z)R(z) \rightarrow S(x)))$.

Outer $\rightarrow$: $(\forall x)(\neg\neg(\exists y)Q(x, y) \vee ((\exists z)R(z) \rightarrow S(x)))$.

Inner $\rightarrow$ and $\neg\neg$: $(\forall x)((\exists y)Q(x, y) \vee (\neg(\exists z)R(z) \vee S(x)))$.

$\neg$ inward: $(\forall x)((\exists y)Q(x, y) \vee ((\forall z)\neg R(z) \vee S(x)))$.

Pull out (y and z do not appear elsewhere): $(\forall x)(\exists y)(\forall z)(Q(x, y) \vee \neg R(z) \vee S(x))$.

EXAMPLE (IGNOU III)

PNF of $(\forall x)(\forall y)((\exists z)Q(x, y, z) \wedge ((\exists u)R(x, u) \rightarrow (\exists v)R(y, v)))$.

Remove $\rightarrow$: $(\forall x)(\forall y)((\exists z)Q(x, y, z) \wedge (\neg(\exists u)R(x, u) \vee (\exists v)R(y, v)))$.

$\neg$ inward: $(\forall x)(\forall y)((\exists z)Q(x, y, z) \wedge ((\forall u)\neg R(x, u) \vee (\exists v)R(y, v)))$.

z, u, v are all distinct and local, so pull them out in order: $(\forall x)(\forall y)(\exists z)(\forall u)(\exists v)(Q(x, y, z) \wedge (\neg R(x, u) \vee R(y, v)))$.

EXAMPLE (CHECK YOUR PROGRESS 2, EX. 4 I)

PNF of $(\forall x)P(x) \rightarrow (\exists x)Q(x)$.

$\neg(\forall x)P(x) \vee (\exists x)Q(x) \equiv (\exists x)\neg P(x) \vee (\exists x)Q(x) \equiv (\exists x)(\neg P(x) \vee Q(x))$, using " $\exists$ distributes over $\vee$ ". No renaming needed.

EXAMPLE (CHECK YOUR PROGRESS 2, EX. 4 II)

PNF of $(\forall x)(\forall y)((\exists z)(P(x, y) \wedge P(y, z)) \rightarrow (\exists u)Q(x, y, u))$.

Remove $\rightarrow$: $(\forall x)(\forall y)(\neg(\exists z)(P(x, y) \wedge P(y, z)) \vee (\exists u)Q(x, y, u))$.

$\neg$ inward: $(\forall x)(\forall y)((\forall z)(\neg P(x, y) \vee \neg P(y, z)) \vee (\exists u)Q(x, y, u))$.

Pull out z then u: $(\forall x)(\forall y)(\forall z)(\exists u)(\neg P(x, y) \vee \neg P(y, z) \vee Q(x, y, u))$.

Exam tip: An $\exists$ inside the left side of $\rightarrow$ becomes $\forall$ when pulled out (because of the hidden $\neg$), and a $\forall$ there becomes $\exists$. In Ex. 4(ii) the IGNOU answer key silently changes $P(x, y) \wedge P(y, z)$ into $P(x, z) \wedge P(y, z)$ and drops the quantifier on u; the correct PNF for the question as printed is the one above, $(\forall x)(\forall y)(\forall z)(\exists u)(\neg P(x, y) \vee \neg P(y, z) \vee Q(x, y, u))$. If your copy reads $P(x, z) \wedge P(y, z)$, as the key assumes, the answer is $(\forall x)(\forall y)(\forall z)(\exists u)(\neg P(x, z) \vee \neg P(y, z) \vee Q(x, y, u))$.

Skolemization and (Skolem) Standard Form

Skolemization: Removing each $\exists$ by replacing its variable with a new Skolem constant (no $\forall$ before it) or a new Skolem function of the preceding $\forall$ variables.

Skolemization: ഓരോ $\exists$ -ഉം നീക്കി, അതിന്റെ variable-ന് പകരം ഒരു പുതിയ Skolem constant (മുമ്പിൽ $\forall$ ഇല്ലെങ്കിൽ) അല്ലെങ്കിൽ മുമ്പിലുള്ള $\forall$ variables-ന്റെ ഒരു പുതിയ Skolem function വെക്കൽ.

(Skolem) Standard Form: The form obtained by converting a formula to PNF, putting its matrix in CNF, and Skolemizing.

(Skolem) Standard Form: ഒരു formula-യെ PNF ആക്കി, അതിന്റെ matrix-നെ CNF ആക്കി, Skolemize ചെയ്ത് കിട്ടുന്ന രൂപം.

Skolemization removes existential quantifiers. An $\exists$ variable with no $\forall$ before it in the prefix is replaced by a new **Skolem constant**. An $\exists$ variable that comes after universal variables $y_1, \dots, y_k$ is replaced by a new **Skolem function** $f(y_1, \dots, y_k)$, because its value may depend on them. Every Skolem constant and function must be a new, distinct symbol. The **(Skolem) Standard Form** is obtained by (1) converting to PNF, (2) putting the matrix in CNF, (3) Skolemizing.

മലയാളത്തിൽ

"ഓരോ കുട്ടിക്കും ഒരു അമ്മയുണ്ട്" ($\forall x \exists y$) എന്ന് പറയുമ്പോൾ അമ്മ കുട്ടിയെ ആശ്രയിച്ച് മാറും, അതിനാൽ y -യെ $\text{mother}(x)$ എന്ന function ആക്കുന്നു. "എല്ലാവരും ബഹുമാനിക്കുന്ന ഒരു അധ്യാപകനുണ്ട്" ($\exists y \forall x$) എന്നതിൽ ആ അധ്യാപകൻ ഒരൊറ്റ ആൾ മാത്രം, അതിനാൽ y -യെ ഒരു constant c ആക്കുന്നു. ചുരുക്കത്തിൽ: $\exists$ -ന് മുമ്പിൽ ഏതൊക്കെ $\forall$ ഉണ്ടോ, അവയെല്ലാം Skolem function-ന്റെ arguments ആകും. ഓരോ $\exists$ -നും പുതിയ പേര് വേണം.

$(\exists x)(\forall y) P(x, y)$ becomes $(\forall y) P(c, y)$ (c : Skolem constant)

$(\forall y)(\exists x) P(x, y)$ becomes $(\forall y) P(f(y), y)$ (f : Skolem function)

Rule: $\exists x$ preceded by $\forall y_1, \dots, \forall y_k =$ replace x by $f(y_1, \dots, y_k)$ ($k = 0 \Rightarrow$ constant)

EXAMPLE (IGNOU I)

Skolemize $(\exists x_1)(\exists x_2)(\forall y_1)(\forall y_2)(\exists x_3)(\forall y_3) P(x_1, x_2, x_3, y_1, y_2, y_3)$.

x_1, x_2 : no $\forall$ before them, so distinct constants c, d . x_3 : preceded by $\forall y_1, \forall y_2$, so $f(y_1, y_2)$.

Result: $(\forall y_1)(\forall y_2)(\forall y_3) P(c, d, f(y_1, y_2), y_1, y_2, y_3)$.

EXAMPLE (IGNOU II)

Skolemize $(\exists x_1)(\forall y_1)(\exists x_2)(\forall y_2)(\exists x_3) P(x_1, x_2, x_3, y_1, y_2) \wedge (\exists x_1)(\forall y_3)(\exists x_2)(\forall y_4) Q(x_1, x_2, y_3, y_4)$.

$\exists$ does not distribute over $\wedge$, so rename the second x_1, x_2 as x_5, x_6 and pull everything to the front:

$(\exists x_1)(\forall y_1)(\exists x_2)(\forall y_2)(\exists x_3)(\exists x_5)(\forall y_3)(\exists x_6)(\forall y_4)(P(x_1, x_2, x_3, y_1, y_2) \wedge Q(x_5, x_6, y_3, y_4))$.

$x_1 \rightarrow c; x_2 \rightarrow f(y_1); x_3 \rightarrow g(y_1, y_2); x_5 \rightarrow h(y_1, y_2); x_6 \rightarrow j(y_1, y_2, y_3)$.

Standard form: $(\forall y_1)(\forall y_2)(\forall y_3)(\forall y_4)(P(c, f(y_1), g(y_1, y_2), y_1, y_2) \wedge Q(h(y_1, y_2), j(y_1, y_2, y_3), y_3, y_4))$.

EXAMPLE (CHECK YOUR PROGRESS 2, EX. 5 I)

Standard form of $(\exists x)(\forall y)(\forall v)(\exists z)(\forall w)(\exists u) P(x, y, z, u, v, w)$.

x : no $\forall$ before it $\rightarrow$ constant c . z : after $\forall y, \forall v \rightarrow f(y, v)$. u : after $\forall y, \forall v, \forall w \rightarrow g(y, v, w)$.

Standard form: $(\forall y)(\forall v)(\forall w) P(c, y, f(y, v), g(y, v, w), v, w)$.

EXAMPLE (CHECK YOUR PROGRESS 2, EX. 5 II)

Standard form of $(\forall x)(\exists y)(\exists z)((P(x, y) \vee \neg Q(x, z)) \rightarrow R(x, y, z))$.

Matrix to CNF: $\neg(P(x, y) \vee \neg Q(x, z)) \vee R(x, y, z) \equiv (\neg P(x, y) \wedge Q(x, z)) \vee R(x, y, z) \equiv (\neg P(x, y) \vee R(x, y, z)) \wedge (Q(x, z) \vee R(x, y, z))$.

y and z are both preceded only by $\forall x$, so $y \rightarrow f(x), z \rightarrow g(x)$ (different functions).

Standard form: $(\forall x)((\neg P(x, f(x)) \vee R(x, f(x), g(x))) \wedge (Q(x, g(x)) \vee R(x, f(x), g(x))))$.

Exam tip: The IGNOU answer key has three slips here, and you should write the corrected forms. Ex. 5(i): the key explains the substitutions but its final line still reads $P(x, y, z, u, v, w)$; the answer is $P(c, y, f(y, v), g(y, v, w), v, w)$. Ex. 5(ii): its final line still contains y and z ; they must be replaced by $f(x)$ and $g(x)$ as above. Example (ii): the key writes $Q(h(y_1, y_2), j(y_1, y_2, y_3))$, dropping y_3, y_4 and $\forall y_4$. (Since the two conjuncts share no variables, Skolemizing each one separately is also correct and gives simpler terms: $P(c, f(y_1), g(y_1, y_2), y_1, y_2) \wedge Q(e, k(y_3), y_3, y_4)$; both answers are acceptable.)

Complete conversion to clausal form

Clausal form: A set of clauses (disjunctions of literals) with all variables implicitly universally quantified, obtained via PNF, Skolemization and CNF.

Clausal form: PNF, Skolemization, CNF എന്നിവ വഴി കിട്ടുന്ന clauses-ന്റെ (literals-ന്റെ disjunctions) set; ഇതിലെ എല്ലാ variables-ഉം implicitly universally quantified ആണ്.

Standardizing apart: Renaming variables clause by clause so that no two clauses share a variable.

Standardizing apart: രണ്ട് clauses ഒരേ variable പങ്കിടാതിരിക്കാൻ ഓരോ clause-ലെയും variables-ന് പ്രത്യേകം പേര് മാറ്റി നൽകൽ.

Resolution works on **clauses**: disjunctions of literals, with all variables implicitly universal. The standard recipe combines PNF, Skolemization and CNF. IGNOU's order (PNF $\rightarrow$ CNF matrix $\rightarrow$ Skolemize) and the order below give equivalent clause sets; the order below is easier to apply without mistakes. Finally, each clause gets its own variable names (**standardizing apart**).

മലയാളത്തിൽ

Resolution എന്ന machine-ന് ഒരു പ്രത്യേക രൂപത്തിലുള്ള input മാത്രമേ കഴിക്കാൻ പറ്റൂ: Λ കൊണ്ട് ചേർത്ത $\vee$ -clauses, $\exists$ ഒന്നുമില്ലാതെ. അതിനായി ഈ എട്ട് steps ക്രമത്തിൽ ചെയ്യുക. Exam-ൽ ഓരോ step-ഉം പേര് എഴുതി വേർതിരിച്ച് കാണിച്ചാൽ മുഴുവൻ മാർക്കും കിട്ടും.

1. Eliminate $\leftrightarrow$ and $\rightarrow$
2. Move $\neg$ inward (De Morgan, $\neg(\forall x) \equiv (\exists x)\neg$, $\neg(\exists x) \equiv (\forall x)\neg$, $\neg\neg P \equiv P$)
3. Standardize variables: each quantifier gets its own variable
4. (Optional) Move quantifiers to the front: PNF
5. Skolemize: replace each $\exists$ variable by a Skolem constant/function of the $\forall$ variables that govern it
6. Drop all $\forall$ (they are now implicit)
7. Convert to CNF: distribute $\vee$ over $\wedge$, i.e. $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$
8. Split at $\wedge$ into separate clauses and standardize apart (rename variables clause by clause)

EXAMPLE

Convert to clausal form: $(\forall x)((\forall y)P(x, y) \rightarrow \neg(\forall y)(Q(x, y) \rightarrow R(x, y)))$.

1. Eliminate $\rightarrow$: $(\forall x)(\neg(\forall y)P(x, y) \vee \neg(\forall y)(\neg Q(x, y) \vee R(x, y)))$.
2. $\neg$ inward: $(\forall x)((\exists y)\neg P(x, y) \vee (\exists y)(Q(x, y) \wedge \neg R(x, y)))$.
3. Standardize: $(\forall x)((\exists y)\neg P(x, y) \vee (\exists z)(Q(x, z) \wedge \neg R(x, z)))$.
4. PNF: $(\forall x)(\exists y)(\exists z)(\neg P(x, y) \vee (Q(x, z) \wedge \neg R(x, z)))$.
5. Skolemize (y, z each depend on x): $y \rightarrow f(x), z \rightarrow g(x)$: $(\forall x)(\neg P(x, f(x)) \vee (Q(x, g(x)) \wedge \neg R(x, g(x))))$.
6. Drop $\forall$: $\neg P(x, f(x)) \vee (Q(x, g(x)) \wedge \neg R(x, g(x)))$.
7. CNF: $(\neg P(x, f(x)) \vee Q(x, g(x))) \wedge (\neg P(x, f(x)) \vee \neg R(x, g(x)))$.
8. Clauses: C1: $\neg P(x_1, f(x_1)) \vee Q(x_1, g(x_1))$; C2: $\neg P(x_2, f(x_2)) \vee \neg R(x_2, g(x_2))$.

EXAMPLE

Convert "Everyone who loves all animals is loved by someone": $(\forall x)((\forall y)(\text{Animal}(y) \rightarrow \text{Loves}(x, y)) \rightarrow (\exists y)\text{Loves}(y, x))$.

1. Eliminate $\rightarrow$: $(\forall x)(\neg(\forall y)(\neg \text{Animal}(y) \vee \text{Loves}(x, y)) \vee (\exists y)\text{Loves}(y, x))$.
 2. $\neg$ inward: $(\forall x)((\exists y)(\text{Animal}(y) \wedge \neg \text{Loves}(x, y)) \vee (\exists y)\text{Loves}(y, x))$.
 3. Standardize: $(\forall x)((\exists y)(\text{Animal}(y) \wedge \neg \text{Loves}(x, y)) \vee (\exists z)\text{Loves}(z, x))$.
 5. Skolemize (both inside $\forall x$): $y \rightarrow F(x), z \rightarrow G(x)$: $(\forall x)((\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))) \vee \text{Loves}(G(x), x))$.
 6. Drop $\forall x$. 7. Distribute: $(\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)) \wedge (\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x))$.
 8. Clauses: C1: $\text{Animal}(F(x_1)) \vee \text{Loves}(G(x_1), x_1)$; C2: $\neg \text{Loves}(x_2, F(x_2)) \vee \text{Loves}(G(x_2), x_2)$.
- $F(x)$ is "the animal x fails to love"; $G(x)$ is "the one who loves x".

EXAMPLE

Clausal form of "Every student who studies passes some exam": $(\forall x)(\text{Student}(x) \wedge \text{Studies}(x) \rightarrow (\exists y)(\text{Exam}(y) \wedge \text{Passes}(x, y)))$.

1. $(\forall x)(\neg(\text{Student}(x) \wedge \text{Studies}(x)) \vee (\exists y)(\text{Exam}(y) \wedge \text{Passes}(x, y)))$.
2. $(\forall x)(\neg \text{Student}(x) \vee \neg \text{Studies}(x) \vee (\exists y)(\text{Exam}(y) \wedge \text{Passes}(x, y)))$.
5. y depends on x: $y \rightarrow f(x)$. 6. Drop $\forall x$: $\neg \text{Student}(x) \vee \neg \text{Studies}(x) \vee (\text{Exam}(f(x)) \wedge \text{Passes}(x, f(x)))$.
- 7-8. C1: $\neg \text{Student}(x_1) \vee \neg \text{Studies}(x_1) \vee \text{Exam}(f(x_1))$; C2: $\neg \text{Student}(x_2) \vee \neg \text{Studies}(x_2) \vee \text{Passes}(x_2, f(x_2))$.

Exam tip: Always move $\neg$ inward before Skolemizing. In $\neg(\forall y)P(y)$ the y is really existential; Skolemizing too early treats it as universal and gives wrong clauses.

Substitution, unification and the most general unifier

Substitution: A set $\{t_1/x_1, \dots, t_m/x_m\}$ that replaces each variable x_i by the term t_i throughout an expression.

Substitution: ഒരു expression-ൽ ഉടനീളം ഓരോ variable x_i -നെയും term t_i കൊണ്ട് മാറ്റുന്ന $\{t_1/x_1, \dots, t_m/x_m\}$ എന്ന set.

Unifier: A substitution that makes two expressions identical.

Unifier: രണ്ട് expressions-നെ ഒരുപോലെ (identical) ആക്കുന്ന substitution.

Most general unifier (MGU): The unifier that commits to the least: every other unifier is the MGU followed by further substitutions.

Most general unifier (MGU): ഏറ്റവും കുറച്ച് മാത്രം നിശ്ചയിക്കുന്ന unifier: മറ്റേത് unifier-ഉം MGU-വിന് ശേഷം കൂടുതൽ substitutions പ്രയോഗിച്ചതാണ്.

A **substitution** $\{t_1/x_1, \dots, t_m/x_m\}$ replaces each variable x_i by the term t_i everywhere in an expression; only variables can be replaced. **Unification** is finding a substitution (a **unifier**) that makes two expressions identical. The **most general unifier (MGU)** is the unifier that commits to as little as possible: every other unifier is the MGU followed by extra substitutions. Two constants, two different function symbols, or a variable against a term containing that same variable (the **occurs check**) cannot be unified.

മലയാളത്തിൽ

Unification എന്നാൽ രണ്ട് expressions-നെ ഒരുപോലെ ആക്കാൻ variables-ന് values കൊടുക്കുന്നത്, രണ്ട് പൂരിപ്പിക്കാത്ത form-കൾ ഒരേ രീതിയിൽ പൂരിപ്പിക്കുന്നത് പോലെ. ഇടത്തുനിന്ന് വലത്തോട്ട് term by term നോക്കുക; ആദ്യം വ്യത്യാസം (disagreement) കാണുന്ന സ്ഥലത്ത് ഒരു variable ഉണ്ടെങ്കിൽ അതിനെ മറ്റേ term കൊണ്ട് മാറ്റുക, അതും മുഴുവൻ expression-ലും. Variable ഇല്ലെങ്കിൽ (a vs b , $f(x)$ vs LK) unification fail. x -നെ $f(x)$ ആക്കാൻ പറ്റില്ല, കാരണം അത് അനന്തമായി നീളും: ഇതാണ് occurs check.

Unification algorithm (disagreement-set method):

1. $\sigma = \{ \}$ (empty substitution)
2. If $E_1\sigma$ and $E_2\sigma$ are identical, stop: σ is the MGU
3. Scan left to right; find the first pair of disagreeing terms (s, t)
4. If one of them is a variable v and the other a term t with v not occurring in t : $\sigma = \sigma \circ \{t/v\}$ (apply $\{t/v\}$ to everything, including the terms already in σ); go to 2
5. Otherwise (constant vs different constant, different function symbols, occurs-check failure): FAIL

EXAMPLE (TRACE)

Unify $P(x, f(y), y)$ and $P(g(z), f(a), z)$.

Step	Disagreement	Action	σ so far
1	x vs $g(z)$	x is a variable, $x \notin g(z)$: add $g(z)/x$	$\{g(z)/x\}$
2	$f(y)$ vs $f(a) \rightarrow y$ vs a	add a/y	$\{g(z)/x, a/y\}$
3	y (now a) vs z	add a/z , and update $g(z)$ to $g(a)$	$\{g(a)/x, a/y, a/z\}$
4	none	both become $P(g(a), f(a), a)$	$\text{MGU} = \{g(a)/x, a/y, a/z\}$

EXAMPLE (TRACE)

Unify $\text{Knows}(\text{John}, x)$ and $\text{Knows}(y, \text{Mother}(y))$.

John vs y : $\{\text{John}/y\}$. Now the second expression is $\text{Knows}(\text{John}, \text{Mother}(\text{John}))$. x vs $\text{Mother}(\text{John})$: $\{\text{Mother}(\text{John})/x\}$. $\text{MGU} = \{\text{John}/y, \text{Mother}(\text{John})/x\}$; both become $\text{Knows}(\text{John}, \text{Mother}(\text{John}))$.

EXAMPLE (FAILURES)

(a) $P(a, x)$ and $P(b, y)$. (b) $P(x, x)$ and $P(y, f(y))$. (c) $\text{Love}(x, f(x))$ and $\text{Love}(a, \text{LK})$.

(a) a vs b : two different constants, FAIL.

(b) x vs y : $\{y/x\}$, giving $P(y, y)$ and $P(y, f(y))$. Now y vs $f(y)$: y occurs in $f(y)$, occurs check FAIL.

(c) x vs a : $\{a/x\}$. Then $f(a)$ vs LK : a function term against a constant, neither is a variable, FAIL. ($f(x)$ contains a variable but is not itself a variable.)

EXAMPLE (MGU VS OTHER UNIFIERS)

Unifiers of $P(x)$ and $P(y)$.

$\{y/x\}$ is the MGU; the result is $P(y)$. $\{a/x, a/y\}$ is also a unifier (result $P(a)$), but it is less general: it equals $\{y/x\}$ followed by $\{a/y\}$. Resolution always uses the MGU so that no useful conclusions are lost.

EXAMPLE (CHECK YOUR PROGRESS 3, EX. 6)

Unify, if possible: $Q(u, f(y, z))$, $Q(u, a)$, $Q(u, g(h(k(u))))$.

The first arguments are all u , which agree. The second arguments are $f(y, z)$, the constant a , and $g(h(k(u)))$.

Their leading symbols f , a , g are all different and none is a variable, so no substitution can make them equal.

Not unifiable (not even any two of the three).

EXAMPLE (CHECK YOUR PROGRESS 3, EX. 7)

Are $Q(f(a), g(x))$ and $Q(x, y)$ unifiable?

$f(a)$ vs x : $\{f(a)/x\}$. Now the first formula is $Q(f(a), g(f(a)))$ and the second $Q(f(a), y)$. $g(f(a))$ vs y : $\{g(f(a))/y\}$. Unifiable, MGU = $\{f(a)/x, g(f(a))/y\}$; both become $Q(f(a), g(f(a)))$.

Exam tip: After adding a new pair t/v , apply it to the terms already in σ as well (step 3 of the first trace). Forgetting this gives a "unifier" that does not actually make the expressions equal. Always finish by substituting σ into both expressions and showing they match.

Resolution in FOPL: refutation proofs

Resolution (FOPL): From clauses $(L_1 \vee \alpha)$ and $(\neg L_2 \vee \beta)$, where L_1 and L_2 unify with MGU σ , derive the resolvent $(\alpha \vee \beta)\sigma$.

Resolution (FOPL): L_1, L_2 എന്നിവ MGU σ ഉപയോഗിച്ച് unify ആകുമെങ്കിൽ, $(L_1 \vee \alpha), (\neg L_2 \vee \beta)$ എന്നീ clauses-ൽ നിന്ന് resolvent $(\alpha \vee \beta)\sigma$ derive ചെയ്യുന്നു.

Refutation: Proving a goal G by adding $\neg G$ to the premises and deriving the empty clause $\square$.

Refutation: Premises-ലേക്ക് $\neg G$ ചേർത്ത് empty clause $\square$ derive ചെയ്ത് goal G തെളിയിക്കൽ.

Resolution is a **refutation** method. To prove a goal G from premises: (1) add $\neg G$ to the premises; (2) convert everything to clauses; (3) repeatedly pick two clauses containing complementary literals L and $\neg L'$ where L and L' unify with MGU σ , and form the **resolvent**: the remaining literals of both clauses with σ applied; (4) if the empty clause $\square$ (False) is derived, the premises with $\neg G$ are inconsistent, so G follows. A formula without variables is a **ground** formula; resolving ground clauses is just PL resolution.

മലയാളത്തിൽ

Resolution-ന്റെ തന്ത്രം ഒരു വക്കീലിന്റേത് പോലെയാണ്: "ഞാൻ പറയുന്നത് തെറ്റാണെന്ന് കരുതുക" (goal-ന്റെ negation ചേർക്കുക), എന്നിട്ട് അത് ഒരു വൈരുദ്ധ്യത്തിൽ ($\square$) എത്തുമെന്ന് കാണിക്കുക. ഓരോ step-ലും: ഏത് രണ്ട് clauses, ഏത് literals, ഏത് substitution, കിട്ടിയ resolvent. ഇവ നാലും എഴുതിയാൽ മഴുവൻ മാർക്ക്. Clauses തമ്മിൽ ഒരു variable പേര് വരാതിരിക്കാൻ ആദ്യം rename ചെയ്യുക.

Resolution rule: from $(L_1 \vee \alpha)$ and $(\neg L_2 \vee \beta)$, with $\sigma = \text{MGU}(L_1, L_2)$, derive $(\alpha \vee \beta)\sigma$

Proof ends when $\square$ (the empty clause) is derived

EXAMPLE (IGNOU: RAMAN IS MORTAL)

Premises: $(\forall x)(\text{MAN}(x) \rightarrow \text{MORTAL}(x))$, $\text{MAN}(\text{Raman})$. Prove $\text{MORTAL}(\text{Raman})$.

#	Clause	How obtained
1	$\neg \text{MAN}(x) \vee \text{MORTAL}(x)$	premise ($\rightarrow$ removed, $\forall$ dropped)
2	$\text{MAN}(\text{Raman})$	premise
3	$\neg \text{MORTAL}(\text{Raman})$	negated goal
4	$\neg \text{MAN}(\text{Raman})$	resolve 1, 3 on MORTAL; $\sigma = \{\text{Raman}/x\}$
5	$\square$	resolve 4, 2 on MAN; $\sigma = \{ \}$

The empty clause is derived, so $\text{MORTAL}(\text{Raman})$ follows.

EXAMPLE

Premises: Every student who studies passes. Anyone who passes is happy. Some student studies. Prove: someone is happy.

FOPL: $(\forall x)(\text{Student}(x) \wedge \text{Studies}(x) \rightarrow \text{Passes}(x))$; $(\forall y)(\text{Passes}(y) \rightarrow \text{Happy}(y))$; $(\exists z)(\text{Student}(z) \wedge \text{Studies}(z))$.

Goal $(\exists w)\text{Happy}(w)$; negated goal $\neg(\exists w)\text{Happy}(w) \equiv (\forall w)\neg\text{Happy}(w)$. The $\exists z$ is Skolemized to a constant a.

#	Clause	How obtained
1	$\neg \text{Student}(x) \vee \neg \text{Studies}(x) \vee \text{Passes}(x)$	premise
2	$\neg \text{Passes}(y) \vee \text{Happy}(y)$	premise
3	$\text{Student}(a)$	premise 3, Skolem constant a
4	$\text{Studies}(a)$	premise 3
5	$\neg \text{Happy}(w)$	negated goal
6	$\neg \text{Passes}(w)$	resolve 2, 5 on Happy; $\sigma = \{w/y\}$
7	$\neg \text{Student}(w) \vee \neg \text{Studies}(w)$	resolve 1, 6 on Passes; $\sigma = \{w/x\}$
8	$\neg \text{Studies}(a)$	resolve 7, 3 on Student; $\sigma = \{a/w\}$
9	$\square$	resolve 8, 4 on Studies; $\sigma = \{ \}$

EXAMPLE (IGNOU: LORD KRISHNA)

(i) Lord Krishna is loved by everyone who loves someone. (ii) No one loves nobody. Prove: Lord Krishna is loved by everyone. Love(x, y): x loves y; LK: Lord Krishna.

(i) $(\forall x)((\exists y)\text{Love}(x, y) \rightarrow \text{Love}(x, \text{LK})) \equiv (\forall x)((\forall y)\neg\text{Love}(x, y) \vee \text{Love}(x, \text{LK})) \equiv (\forall x)(\forall y)(\neg\text{Love}(x, y) \vee \text{Love}(x, \text{LK}))$.

(ii) $\neg(\exists x)(\forall y)\neg\text{Love}(x, y) \equiv (\forall x)(\exists y)\text{Love}(x, y)$; Skolemize $y \rightarrow f(x)$: $\text{Love}(x, f(x))$.

Goal $(\forall x)\text{Love}(x, \text{LK})$; negated: $\neg(\forall x)\text{Love}(x, \text{LK}) \equiv (\exists x)\neg\text{Love}(x, \text{LK})$; Skolem constant a: $\neg\text{Love}(a, \text{LK})$.

#	Clause	How obtained
1	$\neg\text{Love}(x, y) \vee \text{Love}(x, \text{LK})$	premise (i)
2	$\text{Love}(u, f(u))$	premise (ii), variable renamed to u (standardized apart)
3	$\neg\text{Love}(a, \text{LK})$	negated goal
4	$\neg\text{Love}(a, y)$	resolve 1, 3: unify $\text{Love}(x, \text{LK}), \text{Love}(a, \text{LK})$; $\sigma = \{a/x\}$
5	$\square$	resolve 4, 2: unify $\text{Love}(a, y), \text{Love}(u, f(u))$; a vs u gives a/u, then y vs f(a) gives f(a)/y; $\sigma = \{a/u, f(a)/y\}$

Note: clauses 2 and 3 cannot be resolved directly, since unifying $\text{Love}(u, f(u))$ with $\text{Love}(a, \text{LK})$ fails on f(a) vs LK.

EXAMPLE (IGNOU: FINDING A RESOLVENT)

Find the resolvent of (i) $\neg Q(x, z, x) \vee Q(w, z, w)$ and (ii) $Q(w, h(v, v), w)$.

Standardize apart: rename (ii) to $Q(w', h(v, v), w')$. Unify $Q(x, z, x)$ with $Q(w', h(v, v), w')$: x vs w' gives $\{w'/x\}$; z vs h(v, v) gives $\{h(v, v)/z\}$; third place w' vs w' agrees. MGU $\sigma = \{w'/x, h(v, v)/z\}$.

(i) $\sigma = \neg Q(w', h(v, v), w') \vee Q(w, h(v, v), w)$. Cancel $\neg Q(w', h(v, v), w')$ against (ii).

Resolvent: $Q(w, h(v, v), w)$.

Exam tip: The IGNOU Krishna solution unifies $\text{Love}(x, f(x))$ with $\neg\text{Love}(a, y)$ using $\{a/x, f(x)/y\}$ and says both become $\text{Love}(a, f(x))$. Applying $x \rightarrow a$ inside f(x) as well gives f(a), so the correct MGU is $\{a/x, f(a)/y\}$ and both literals become $\text{Love}(a, f(a))$. The conclusion ($\square$) is unaffected. In the resolvent example the text uses w in both clauses without renaming; the answer happens to be the same, but always standardize clauses apart first.

Quick recap

- PL treats "Every man is mortal" as an indivisible P, so it cannot prove "Raman is mortal"; FOPL splits statements into predicates and arguments and adds $\forall, \exists$.
- Terms (constants, variables, $f(t_1, \dots, t_n)$) denote objects; atoms $P(t_1, \dots, t_n)$ have truth values.
- Scope is the smallest complete formula after a quantifier; occurrences inside it are bound, others are free. Closed wff: no free variables.
- Translation: $\forall$ with $\rightarrow$, $\exists$ with $\wedge$. "No P is Q" = $(\forall x)(P(x) \rightarrow \neg Q(x))$; "Only P are Q" = $(\forall x)(Q(x) \rightarrow P(x))$.
- $(\forall x)(\exists y) \neq (\exists y)(\forall x)$: order of mixed quantifiers matters.
- $\forall$ distributes over $\wedge$, $\exists$ over $\vee$; not $\forall$ over $\vee$ or $\exists$ over $\wedge$. $\neg \forall \equiv \exists \neg$, $\neg \exists \equiv \forall \neg$.
- Valid: true in every interpretation; consistent: true in some; inconsistent: true in none.
- Quantifier rules: UI, EG freely; EI needs a new constant; UG only for an arbitrary element. 8 PL rules + 4 quantifier rules = 12.
- PNF: prefix of quantifiers + quantifier-free matrix. Remove $\rightarrow$, push $\neg$ in, rename, pull quantifiers out in order.
- Skolemization: $\exists$ with no $\forall$ before it $\rightarrow$ new constant; $\exists$ after $\forall y_1 \dots \forall y_k \rightarrow$ new function $f(y_1, \dots, y_k)$.
- Clausal form: eliminate $\rightarrow$, $\neg$ inward, standardize, Skolemize, drop $\forall$, CNF, split and standardize apart.
- Unification: scan left to right, bind a variable to the disagreeing term, apply everywhere; fail on constant clash, functor clash or occurs check. Use the MGU.
- Resolution refutation: add $\neg$ goal, convert to clauses, resolve with MGUs until $\square$. Record clause pair, literal and substitution at each step.

Check yourself

Q. 1. Let $C(x)$: x is a used-car dealer, $H(x)$: x is honest. Translate into English: (i) $(\exists x)C(x)$ (ii) $(\exists x)H(x)$ (iii) $(\forall x)(C(x) \rightarrow \neg H(x))$ (iv) $(\exists x)(C(x) \wedge H(x))$ (v) $(\exists x)(H(x) \rightarrow C(x))$.

(i) There is at least one used-car dealer. (ii) There is at least one honest person. (iii) No used-car dealer is honest (all used-car dealers are dishonest). (iv) At least one used-car dealer is honest. (v) There is something which, if it is honest, is a used-car dealer. This is not "some honest person is a used-car dealer"; it is true as soon as anything is dishonest, which is why $\exists$ should go with $\wedge$, not $\rightarrow$.

Q. 2. Translate into FOPL: (a) Every Keralite likes fish. (b) Some doctors are not rich. (c) Every teacher has a student who respects him. (d) There is a teacher whom every student respects.

(a) $(\forall x)(\text{Keralite}(x) \rightarrow \text{LikesFish}(x)).$

(b) $(\exists x)(\text{Doctor}(x) \wedge \neg \text{Rich}(x)).$

(c) $(\forall x)(\text{Teacher}(x) \rightarrow (\exists y)(\text{Student}(y) \wedge \text{Respects}(y, x))).$

(d) $(\exists x)(\text{Teacher}(x) \wedge (\forall y)(\text{Student}(y) \rightarrow \text{Respects}(y, x))).$

(c) and (d) differ in quantifier order: in (c) the student may differ per teacher; in (d) one teacher is respected by all students.

Q. 3. In $(\forall x)(P(x) \rightarrow Q(x, y)) \wedge R(x)$, identify the scope of $(\forall x)$ and the free and bound occurrences. Is the formula open or closed?

Scope of $(\forall x)$: $(P(x) \rightarrow Q(x, y))$. The x in $(\forall x)$, $P(x)$ and $Q(x, y)$ are bound; the x in $R(x)$ is free; y is free. So x is both free and bound, y is free, and the formula is open.

Q. 4. Show that $(\forall x)(\exists y) \text{Loves}(x, y)$ and $(\exists y)(\forall x) \text{Loves}(x, y)$ are not equivalent. Which one implies the other?

Domain $\{\text{Ram}, \text{Sita}\}$: Ram loves Sita, Sita loves Ram, and no one loves himself/herself. Then $(\forall x)(\exists y)\text{Loves}(x, y)$ is True (each loves someone) but $(\exists y)(\forall x)\text{Loves}(x, y)$ is False (no single person is loved by both, since nobody loves himself). So they are not equivalent. The implication goes one way: $(\exists y)(\forall x)\text{Loves}(x, y) \rightarrow (\forall x)(\exists y)\text{Loves}(x, y)$, because the single y that works for everyone also works for each x .

Q. 5. Find a prenex normal form of $(\exists x)P(x) \rightarrow (\forall y)(Q(y) \vee (\exists z)R(y, z))$.

Remove $\rightarrow$: $\neg(\exists x)P(x) \vee (\forall y)(Q(y) \vee (\exists z)R(y, z)).$

$\neg$ inward: $(\forall x)\neg P(x) \vee (\forall y)(Q(y) \vee (\exists z)R(y, z)).$

All variables are distinct, so pull out in order: $(\forall x)(\forall y)(\exists z)(\neg P(x) \vee Q(y) \vee R(y, z)).$

Q. 6. Give the Skolem standard form of $(\forall x)(\exists y)(\forall z)(\exists u)(P(x, y) \wedge (Q(z) \vee R(u, x)))$, and list its clauses.

y comes after $\forall x$: $y \rightarrow f(x)$. u comes after $\forall x, \forall z$: $u \rightarrow g(x, z)$. The matrix is already in CNF.

Standard form: $(\forall x)(\forall z)(P(x, f(x)) \wedge (Q(z) \vee R(g(x, z), x))).$

Clauses (standardized apart): C1: $P(x_1, f(x_1))$; C2: $Q(z) \vee R(g(x_2, z), x_2).$

Q. 7. Find the MGU, if it exists: (a) $P(f(x), y, g(y))$ and $P(f(a), z, g(b))$ (b) $Q(x, g(x))$ and $Q(h(y), y)$ (c) $R(x, f(a), x)$ and $R(b, y, z)$.

(a) $f(x)$ vs $f(a)$: $\{a/x\}$. y vs z : $\{z/y\}$. $g(y) = g(z)$ vs $g(b)$: z vs b : $\{b/z\}$, which updates y to b . MGU = $\{a/x, b/y, b/z\}$; both become $P(f(a), b, g(b))$.

(b) x vs $h(y)$: $\{h(y)/x\}$, giving $Q(h(y), g(h(y)))$ and $Q(h(y), y)$. $g(h(y))$ vs y : y occurs inside $g(h(y))$, occurs check fails. Not unifiable.

(c) x vs b : $\{b/x\}$. $f(a)$ vs y : $\{f(a)/y\}$. x (now b) vs z : $\{b/z\}$. MGU = $\{b/x, f(a)/y, b/z\}$; both become $R(b, f(a), b)$.

Q. 8. Prove the car-repair argument by resolution: $(\forall x)(R(x) \rightarrow H(x) \wedge S(x))$, $(\exists x)(R(x) \wedge M(x)) \vdash (\exists x)(M(x) \wedge H(x))$.

Clauses: 1. $\neg R(x) \vee H(x)$ and 2. $\neg R(y) \vee S(y)$ (from premise 1, CNF split, standardized apart); 3. $R(a)$ and 4. $M(a)$ (premise 2, Skolem constant a); 5. $\neg M(z) \vee \neg H(z)$ (negated goal $\neg(\exists x)(M(x) \wedge H(x)) \equiv (\forall z)(\neg M(z) \vee \neg H(z))$).

6. $\neg H(a)$: resolve 5, 4 on M , $\sigma = \{a/z\}$.

7. $\neg R(a)$: resolve 6, 1 on H , $\sigma = \{a/x\}$.

8. $\square$: resolve 7, 3 on R , $\sigma = \{\}$.

Hence the conclusion follows. (Clause 2 is not needed.)

Q. 9. Given: All Keralites like fish curry. Anyone who likes fish curry eats rice. Anu is a Keralite. Prove by resolution that someone eats rice.

$K(x)$: Keralite, $F(x)$: likes fish curry, $E(x)$: eats rice.

1. $\neg K(x) \vee F(x)$ (premise); 2. $\neg F(y) \vee E(y)$ (premise); 3. $K(\text{Anu})$ (premise); 4. $\neg E(w)$ (negated goal $\neg(\exists w)E(w) \equiv (\forall w)\neg E(w)$).

5. $\neg F(w)$: resolve 2, 4 on E , $\sigma = \{w/y\}$.

6. $\neg K(w)$: resolve 1, 5 on F , $\sigma = \{w/x\}$.

7. $\square$: resolve 6, 3 on K , $\sigma = \{\text{Anu}/w\}$.

So someone (in fact Anu) eats rice.

UNIT 6

Rule Based Systems and Other Formalism

Expert system-കൾ knowledge എങ്ങനെ സൂക്ഷിക്കുന്നു, അതിൽ നിന്ന് എങ്ങനെ പുതിയ conclusion-ൽ എത്തുന്നു എന്നതാണ് ഈ unit. IF-THEN rules (forward, backward chaining, conflict resolution), പിന്നെ semantic nets, frames, scripts എന്നീ മറ്റ് knowledge representation രീതികളും ഇവിടെ പഠിക്കുന്നു.

Expert systems and knowledge representation

Rule based systems

Advantages, disadvantages and working memory

Uncertainty and certainty factors

Forward chaining

Backward chaining

Forward or backward? Fan-in and fan-out

Conflict resolution

Semantic nets

Frames

Scripts and their components

Primitive action symbols

Writing a script: examples, advantages, disadvantages

Expert systems and knowledge representation

Expert system: A computer program that represents knowledge in a particular domain and reasons with it to solve problems or reach goals, as a human expert would.

Expert system: ഒരു പ്രത്യേക domain-ലെ knowledge represent ചെയ്യുകയും, ഒരു human expert-നെപ്പോലെ അത് ഉപയോഗിച്ച് reason ചെയ്ത് problems solve ചെയ്യുകയോ goals-ൽ എത്തുകയോ ചെയ്യുന്ന computer program.

Knowledge-based system: An AI program that solves tasks by applying rules of thumb to a symbolic representation of knowledge; usually a synonym for expert system.

Knowledge-based system: Knowledge-ന്റെ ഒരു symbolic representation-ൽ rules of thumb പ്രയോഗിച്ച് tasks solve ചെയ്യുന്ന AI program; സാധാരണയായി expert system-ന്റെ പര്യായം.

An **expert system** is a computer program that possesses or represents knowledge in a particular domain and can reason with this knowledge to solve a problem or reach a goal, the way a human expert would. AI programs that reach expert-level competence are also called **knowledge-based systems**: they solve tasks by applying rules of thumb to a symbolic representation of knowledge instead of mainly algorithmic or statistical methods. The two terms are usually used as synonyms.

Building an expert system is called **knowledge engineering**, and the person who does it is a **knowledge engineer**. The knowledge engineer must choose a **knowledge representation scheme**. A good scheme must (1) let us express the knowledge we want, (2) support **inference** (deriving new knowledge from given facts, because we cannot write down everything explicitly), and (3) have well-defined syntax and semantics.

മലയാളത്തിൽ

നമ്മുടെ നാട്ടിലെ പരിചയസമ്പന്നനായ ഒരു doctor-ന്റെ അറിയും ചിന്താശീലിയും ഒരു program-ലേക്ക് പകർത്തിയാൽ അതാണ് expert system. ആ അറിവ് computer-ന് മനസ്സിലാക്കുന്ന രൂപത്തിൽ എഴുതുന്ന ആളാണ് knowledge engineer. എല്ലാ കാര്യവും നേരിട്ട് എഴുതി വെക്കാൻ പറ്റില്ല, അതുകൊണ്ട് ഉള്ള facts-ൽ നിന്ന് പുതിയ കാര്യം infer ചെയ്യാനുള്ള കഴിവ് representation-ന് ഉണ്ടാകണം. Predicate logic കഴിഞ്ഞ unit-ൽ കണ്ടു; ഇവിടെ rules, semantic nets, frames, scripts എന്നിവ നോക്കാം.

Expert system = domain knowledge + inference (reasoning) capability

Knowledge-based system $\approx$ expert system (synonyms in practice)

Knowledge engineering: building an expert system; knowledge engineer: the builder

Good KR scheme: expressive, supports inference, well-defined syntax and semantics

Popular KR schemes: first order logic, semantic networks, frames, scripts, rule-based systems

EXAMPLE

Why must a KR scheme support inference? Give an illustration.

Suppose the system is told "Every mammal is warm-blooded" and "A lion is a mammal". Nobody wrote "A lion is warm-blooded", yet the system should answer "Yes" when asked. Storing every such consequence explicitly is impossible, so the scheme must let the system deduce it when needed.

Exam tip: IGNOU's example of a limit on expressiveness: "Every symmetric and transitive relation on a domain need not be reflexive" cannot be expressed in First Order Logic, because it quantifies over relations, not over objects.

Rule based systems

Definition: A rule-based system stores knowledge as IF-THEN rules and consists of a rule base, a set of facts, and an interpreter that chooses which rule to apply and executes it.

Knowledge-നെ IF-THEN rules ആയി സൂക്ഷിക്കുന്ന system ആണ് rule-based system; ഇതിൽ ഒരു rule base, facts-ന്റെ ഒരു set, ഏത് rule പ്രയോഗിക്കണമെന്ന് തിരഞ്ഞെടുത്ത് അത് നടപ്പാക്കുന്ന ഒരു interpreter എന്നിവ അടങ്ങിയിരിക്കുന്നു.

Instead of storing knowledge as a static list of true statements, a **rule-based system** stores it as **IF-THEN rules**, each saying what conclusion (or action) follows under given conditions. It has three parts: (i) a **rule base** (the set of IF-THEN rules), (ii) a set of **facts**, and (iii) an **interpreter** that decides which rule to apply given the current facts, and then carries out that rule's action.

There are two kinds. **Forward chaining** starts from the facts and keeps applying rules to get new conclusions until the goal appears (data-driven). **Backward chaining** starts from the goal and looks for rules that could prove it, turning their conditions into sub-goals, until every sub-goal is a known fact (goal-driven).

മലയാളത്തിൽ

ഒരു rule-based system-ൽ മൂന്ന് കാര്യം: "IF ഇത് THEN അത്" എന്ന രീതിയിലുള്ള rules, നമുക്ക് അറിയാവുന്ന facts, പിന്നെ ഏത് rule ഇപ്പോൾ ഉപയോഗിക്കണം എന്ന് തീരുമാനിക്കുന്ന interpreter. Facts-ൽ നിന്ന് തുടങ്ങി മുന്നോട്ട് പോകുന്നത് forward chaining; "ഇത് ശരിയാണോ?" എന്ന goal-ൽ നിന്ന് തുടങ്ങി പിന്നോട്ട് facts വരെ എത്തുന്നത് backward chaining.

Rule-based system = rule base (IF-THEN rules) + facts + interpreter

IF part = condition / antecedent; THEN part = action / consequent

Forward chaining: facts → rules → goal (data-driven)

Backward chaining: goal → rules → sub-goals → facts (goal-driven)

EXAMPLE (FROM THE UNIT)

Rule base: R1: If A is an animal and A barks, then A is a dog. F1: Rocky is an animal. F2: Rocky barks.

The interpreter matches R1 with A = Rocky. Both conditions (F1, F2) hold, so it fires R1 and adds F3: Rocky is a dog. The rule base now holds R1, F1, F2, F3.

Note: Section 6.2 opens with a paragraph on planning and classical planning environments (fully observable, deterministic, finite, static, discrete). It is background only; the rest of the section is about rule-based systems.

Advantages, disadvantages and working memory

Working memory: The store of facts in a rule-based system: assertions (lists of symbols such as (at-home ram)) that denote facts and can be added or removed.

Working memory: Rule-based system-ൽ facts സൂക്ഷിക്കുന്ന ഇടം: facts-നെ സൂചിപ്പിക്കുന്ന, ചേർക്കാനും നീക്കാനും കഴിയുന്ന assertions ((at-home ram) പോലുള്ള symbols-ന്റെ lists).

Modularity: Each rule is an independent piece of knowledge, so rules can be added, deleted or modified without side effects.

Modularity: ഓരോ rule-ഉം സ്വതന്ത്രമായ ഒരു knowledge ഭാഗമാണ്, അതിനാൽ side effects ഇല്ലാതെ rules ചേർക്കാനും ഒഴിവാക്കാനും മാറ്റാനും കഴിയും.

Advantages: each rule is an independent piece of knowledge. Its IF part contains all the conditions it needs and its THEN part says what to do, so the interpreter looks nowhere else. No rule calls another rule, so the rule base is highly **modular**: rules can be added, deleted or modified without side effects.

Disadvantages: when the rule base becomes very large it is hard to check (i) whether a new rule is **redundant** (already covered by earlier rules) and (ii) whether the rule base is **consistent** (two rules with similar conditions but conflicting actions).

The facts are kept in a **working memory** (WM). Lexically it uses application-specific symbols; structurally, each assertion is a list of such symbols, e.g. (at-home ram); semantically, assertions denote facts; and assertions can be added to or removed from it.

മലയാളത്തിൽ

ഓരോ rule-ഉം ഒറ്റയ്ക്ക് നിൽക്കുന്ന ഒരു അറിവാണു്; ഒന്ന് മറ്റൊന്നിനെ വിളിക്കുന്നില്ല. അതുകൊണ്ട് ഒരു rule മാറ്റിയാലും ബാക്കിയുള്ളവയെ ബാധിക്കില്ല (modularity). പക്ഷേ ആയിരക്കണക്കിന് rules ആയാൽ, പുതിയ rule പഴയതിന്റെ ആവർത്തനമാണോ (redundant), രണ്ട് rules പരസ്പരം വിരുദ്ധമാണോ (inconsistent) എന്ന് നോക്കാൻ ബുദ്ധിമുട്ടാണ്. Working memory എന്നാൽ system-ന്റെ "നോട്ട് ബുക്ക്": ഇപ്പോഴത്തെ facts അതിൽ, അവ ചേർക്കാനും മാറ്റാനും പറ്റും.

Advantage: independent rules $\Rightarrow$ modular $\Rightarrow$ easy to add / delete / modify

Disadvantage: large rule base $\Rightarrow$ hard to check redundancy and consistency

Working memory: assertions (lists of symbols) denoting facts; can be added or removed

EXAMPLE

Spot the problem in this rule base: Ra: IF (fever X) THEN ADD (give-paracetamol X). Rb: IF (fever X) AND (child X) THEN ADD (give-paracetamol X). Rc: IF (fever X) THEN ADD (no-medicine X).

Rb is **redundant**: whenever Rb applies, Ra already applies and gives the same conclusion. Ra and Rc are **inconsistent**: same condition, conflicting actions. With three rules this is easy to see; with 3000 rules it is not.

Uncertainty and certainty factors

Definition: A certainty factor is a number attached to an assertion showing how certain it is; in IGNOU's description it ranges from 0 (definitely false) to 1 (definitely true).

ഒരു assertion എത്രത്തോളം ഉറപ്പാണെന്ന് കാണിക്കുന്ന, അതിനോട് ചേർത്ത ഒരു സംഖ്യയാണ് certainty factor; IGNOU-വിന്റെ വിവരണത്തിൽ ഇത് 0 (തീർച്ചയായും false) മുതൽ 1 (തീർച്ചയായും true) വരെയാണ്.

Rule-based systems usually work in domains where conclusions are rarely certain. Two sources of uncertainty: (1) the theory of the domain may be vague or incomplete, and (2) case data may be imprecise or unreliable, with evidence missing or in conflict. So developers add a **certainty (or probability) computing procedure** on top of the rules. In IGNOU's description, each assertion gets a number between 0 and 1: 0 means definitely false, 1 means definitely true.

മലയാളത്തിൽ

Doctor പോലും "ഇത് 100% dengue ആണ്" എന്ന് പറയാറില്ല; "സാധ്യത കൂടുതലാണ്" എന്നേ പറയൂ. അറിവ് പൂർണ്ണമല്ലാത്തതും, patient പറയുന്ന വിവരങ്ങൾ കൃത്യമല്ലാത്തതുമാണ് കാരണം. അതുകൊണ്ട് ഓരോ fact-നും conclusion-നും ഒരു certainty factor (0 മുതൽ 1 വരെ) കൊടുക്കുന്നു.

Sources of uncertainty: vague/incomplete domain theory; imprecise/unreliable/missing/conflicting case data

Certainty factor (IGNOU): 0 = definitely false, 1 = definitely true

Theories for handling it: probability, fuzzy sets, possibility theory, Dempster-Shafer theory

EXAMPLE (FROM THE UNIT)

- (1) The assertion (at-home ram) may carry a certainty factor of 0.7.
- (2) A MYCIN-style rule: IF patient has symptoms s1, s2, s3 and s4 AND background conditions t1, t2 and t3 hold THEN the patient has disease d6 with certainty 0.75.

Exam tip: In the real MYCIN system, certainty factors run from -1 (definitely false) to +1 (definitely true), with 0 meaning "no evidence either way". For IGNOU answers use the unit's 0 to 1 description, but do not be surprised by the -1 to +1 range in other books.

Forward chaining

Forward chaining: Data-driven reasoning that starts from the facts in working memory and repeatedly fires rules whose conditions match them, until the goal is reached or no rule fires.

Forward chaining: Working memory-യിലെ facts-ൽ നിന്ന് തുടങ്ങി, conditions match ചെയ്യുന്ന rules ആവർത്തിച്ച് fire ചെയ്യുന്ന data-driven reasoning; goal-ൽ എത്തുകയോ ഒരു rule-ഉം fire ചെയ്യാതാകുകയോ ചെയ്യുന്നതുവരെ ഇത് തുടരുന്നു.

Conflict set: The set of all rules whose conditions currently match facts in working memory.

Conflict set: നിലവിൽ working memory-യിലെ facts-മായി conditions match ചെയ്യുന്ന എല്ലാ rules-ന്റെയും set.

In forward chaining, facts sit in a working memory that keeps changing. Rules are **condition-action** (antecedent-consequent) rules: conditions are patterns matched against WM items, and actions add or delete items. The interpreter runs the **recognize-act cycle**: (1) find all rules whose every condition can be matched with WM facts using consistent substitutions (this restricted unification is called **pattern matching**); this set is the **conflict set**; (2) if more than one rule applies, pick one by a **conflict resolution** strategy; (3) fire it, updating WM; repeat until no rule fires or the goal is reached.

മലയാളത്തിൽ

Forward chaining ഒരു ചങ്ങല പോലെയാണ്: ഇപ്പോഴുള്ള facts നോക്കുക, ഏതൊക്കെ rules-ന്റെ IF part ശരിയാകുന്നു എന്ന് കണ്ടുപിടിക്കുക (conflict set), അതിൽ ഒന്ന് തിരഞ്ഞെടുത്ത് fire ചെയ്യുക, working memory update ചെയ്യുക, വീണ്ടും ആദ്യം മുതൽ. പുതിയ rule ഒന്നും fire ചെയ്യാനില്ലാതാകുമ്പോഴോ goal കിട്ടുമ്പോഴോ നിർത്തും. ഇതിനെയാണ് recognize-act cycle എന്ന് പറയുന്നത്.

Recognize-act cycle:

1. Match: conflict set = all rules whose antecedents match WM facts
 2. Select: choose one rule (conflict resolution)
 3. Act: fire it (ADD / DELETE facts) $\Rightarrow$ new WM
 4. Repeat until no rule fires or goal reached
- A rule fires at most once on the same data (refraction)

EXAMPLE 1 (FROM THE UNIT): THE MONDAY EXAMPLE

Initial WM: (day monday), (at-home ram), (does-not-like ram).

R1: IF (day monday) THEN ADD (working-with ram)

R2: IF (day monday) THEN ADD (talking-to ram)

R3: IF (talking-to X) AND (working-with X) THEN ADD (busy-at-work X)

R4: IF (busy-at-work X) OR (at-office X) THEN ADD (not-at-home X)

R5: IF (not-at-home X) THEN DELETE (at-home X) (see the tip below)

R6: IF (working-with X) THEN DELETE (does-not-like X)

Strategy: do not fire a rule twice on the same data; among the rest, pick the lowest-numbered rule. X = ram throughout.

Cycle	Conflict set	Fires	Change	WM after the cycle
1	R1, R2	R1	+ (working-with ram)	(day monday), (at-home ram), (does-not-like ram), (working-with ram)
2	R2, R6	R2	+ (talking-to ram)	... , (working-with ram), (talking-to ram)
3	R3, R6	R3	+ (busy-at-work ram)	... , (talking-to ram), (busy-at-work ram)
4	R4, R6	R4	+ (not-at-home ram)	... , (busy-at-work ram), (not-at-home ram)
5	R5, R6	R5	– (at-home ram)	(day monday), (does-not-like ram), (working-with ram), (talking-to ram), (busy-at-work ram), (not-at-home ram)
6	R6	R6	– (does-not-like ram)	(day monday), (working-with ram), (talking-to ram), (busy-at-work ram), (not-at-home ram)
7	none	stop		final WM as in cycle 6

The unit stops after cycle 5 ("the forward chaining will continue like this"); cycle 6 is where it continues: R6 is the only rule left, and it deletes (does-not-like ram). Then nothing new can fire.

EXAMPLE 2: A KERALA CLINIC

Rules for patient X:

D1: IF (fever X) AND (cough X) THEN ADD (respiratory-infection X)

D2: IF (respiratory-infection X) THEN ADD (advise-rest X)

D3: IF (respiratory-infection X) AND (breathing-difficulty X) THEN ADD (refer-to-hospital X)

D4: IF (fever X) AND (joint-pain X) AND (rash X) THEN ADD (suspect-chikungunya X)

D5: IF (suspect-chikungunya X) THEN ADD (order-blood-test X)

Initial WM for Meenu: (fever meenu), (cough meenu), (joint-pain meenu), (rash meenu). Strategy: lowest-numbered rule, no rule twice.

Cycle	Conflict set	Fires	Added to WM
1	D1, D4	D1	(respiratory-infection meenu)
2	D2, D4	D2	(advise-rest meenu)
3	D4	D4	(suspect-chikungunya meenu)
4	D5	D5	(order-blood-test meenu)
5	none	stop	

D3 never enters the conflict set because (breathing-difficulty meenu) is not in WM. Final advice: rest, and a blood test for suspected chikungunya.

EXAMPLE 3: WHY THE ORDER OF FIRING MATTERS

WM: (milk-available). Ra: IF (milk-available) THEN ADD (tea-made). Rb: IF (milk-available) THEN DELETE (milk-available) (the cat drank the milk).

Order Ra, Rb: WM goes to (milk-available), (tea-made), then Rb leaves (tea-made): tea was made. Order Rb, Ra: after Rb the WM is empty, Ra no longer matches, and no tea is made. Same rules, same start, different final WM.

Exam tip: The unit prints R5 as "DELETE (happy X)" but its trace says R5 removes (at-home ram). With R5 as printed, firing it changes nothing because (happy ram) is not in WM, and (at-home ram) would stay. The trace only makes sense if R5 is IF (not-at-home X) THEN DELETE (at-home X), which is what we used above. Mention the correction if you reproduce this example.

Backward chaining

Definition: Backward chaining is goal-driven reasoning that starts from the goal, finds rules whose consequent matches it, and tries to prove their conditions as sub-goals, down to known facts.

Goal-ൽ നിന്ന് തുടങ്ങി, consequent അതുമായി match ചെയ്യുന്ന rules കണ്ടെത്തി, അവയുടെ conditions-നെ sub-goals ആയി known facts വരെ prove ചെയ്യാൻ ശ്രമിക്കുന്ന goal-driven reasoning ആണ് backward chaining.

If we already have a **specific hypothesis** to test, forward chaining wastes effort drawing irrelevant conclusions. Backward chaining starts from the goal: if the goal matches a fact, it succeeds; otherwise the system finds a rule whose **consequent** matches the goal and tries to prove each condition of that rule as a new **sub-goal**, by the same procedure. If a rule fails, it **backtracks** and tries another rule with the same consequent. It does not need to update a working memory; it keeps track of the goals still to be proved. This is essentially how **Prolog** works. For backward chaining, the THEN part is written as a state that becomes true (e.g. (mammal X)), not an ADD/DELETE action.

മലയാളത്തിൽ

ഒരു detective-നെ പോലെ ചിന്തിക്കുക: "ഇവനാണോ കള്ളൻ?" എന്ന സംശയത്തിൽ നിന്ന് തുടങ്ങി, അത് ശരിയാക്കാൻ എന്തൊക്കെ തെളിവ് വേണം എന്ന് നോക്കുന്നു. ഓരോ തെളിവും ഒരു sub-goal ആകുന്നു. ഒരു വഴി പരാജയപ്പെട്ടാൽ പിന്നോട്ട് വന്ന് (backtrack) അടുത്ത rule നോക്കുന്നു. എല്ലാ sub-goal-ഉം അറിയാവുന്ന facts-ൽ എത്തിയാൽ hypothesis തെളിഞ്ഞു.

prove(G): if G is a fact $\Rightarrow$ success

else for each rule "IF C1 AND ... AND Cn THEN G": if prove(C1), ..., prove(Cn) all succeed $\Rightarrow$ success

if no rule works $\Rightarrow$ fail (backtrack to the caller)

Reasoning runs goal $\rightarrow$ rules $\rightarrow$ facts ("anchoring in facts")

EXAMPLE 1 (FROM THE UNIT): ANIMAL IDENTIFIER, IS RAJA A LION?

Facts: (has-hair raja), (big-mouth raja), (long-pointed-teeth raja), (claws raja).

Rules: 1. (gives-milk X) $\rightarrow$ (mammal X) 2. (has-hair X) $\rightarrow$ (mammal X) 3. (mammal X) $\wedge$ (eats-meat X) $\rightarrow$ (carnivorous X) 4. (mammal X) $\wedge$ (long-pointed-teeth X) $\wedge$ (claws X) $\rightarrow$ (carnivorous X) 5. (mammal X) $\wedge$ (does-not-eat-meat X) $\rightarrow$ (herbivorous X) 6. (carnivorous X) $\wedge$ (dark-spots X) $\rightarrow$ (cheetah X) 7. (herbivorous X) $\wedge$ (long-legs X) $\wedge$ (long-neck X) $\wedge$ (dark-spots X) $\rightarrow$ (giraffe X) 8. (carnivorous X) $\wedge$ (big-mouth X) $\rightarrow$ (lion X) 9. (herbivorous X) $\wedge$ (long-trunk X) $\wedge$ (big-size X) $\rightarrow$ (elephant X) 10. (herbivorous X) $\wedge$ (white-color X) $\wedge$ (black-strips X) $\rightarrow$ (zebra X)

Goal: (lion raja). Indentation shows the depth of sub-goals.

```
Goal lion(raja): try rule 8, needs carnivorous, big-mouth
  Sub-goal carnivorous(raja): try rule 3, needs mammal, eats-meat
    Sub-goal mammal(raja): try rule 1, needs gives-milk
      gives-milk(raja): not a fact, no rule concludes it FAIL
    rule 1 fails; try rule 2, needs has-hair
      has-hair(raja): FACT
    mammal(raja) PROVED (rule 2)
    Sub-goal eats-meat(raja): not a fact, no rule FAIL
  rule 3 fails; try rule 4, needs mammal, long-pointed-teeth, claws
    mammal(raja): already proved
    long-pointed-teeth(raja): FACT
    claws(raja): FACT
  carnivorous(raja) PROVED (rule 4)
  Sub-goal big-mouth(raja): FACT
lion(raja) PROVED (rule 8)
```

Conclusion: raja is a lion. Notice that the system never looked at rules 5, 7, 9, 10 (herbivore rules): only goal-relevant rules are used.

EXAMPLE 2: A HYPOTHESIS THAT FAILS

Same facts. Goal: (cheetah raja).

Rule 6 needs (carnivorous raja) and (dark-spots raja). Exactly as above, carnivorous(raja) is proved through rule 4 (after rule 3 fails). Then dark-spots(raja) is neither a fact nor the consequent of any rule, so it fails. Rule 6 was the only rule for cheetah, so the goal **fails**: the system cannot conclude that raja is a cheetah.

EXAMPLE 3: KERALA CLINIC, BACKWARD

Using rules D1 to D5 of the previous topic and Meenu's facts, is (order-blood-test meenu) true?

Goal order-blood-test: only D5 concludes it, sub-goal suspect-chikungunya. Only D4 concludes that, sub-goals fever, joint-pain, rash: all three are facts. So D4 succeeds, then D5 succeeds: **yes**. The rules about cough and respiratory infection (D1, D2, D3) were never touched, unlike the forward run which fired D1 and D2 first.

Forward or backward? Fan-in and fan-out

Fan-out: The number of conclusions the given facts can lead to; fan-out > fan-in favours backward chaining.

Fan-out: തന്നിരിക്കുന്ന facts എത്തിച്ചേരുന്ന conclusions-ന്റെ എണ്ണം; fan-out > fan-in ആണെങ്കിൽ backward chaining ആണ് ഉചിതം.

Fan-in: The number of input paths leading to the conclusion we want; fan-in > fan-out favours forward chaining.

Fan-in: നമുക്ക് വേണ്ട conclusion-ലേക്ക് നയിക്കുന്ന input paths-ന്റെ എണ്ണം; fan-in > fan-out ആണെങ്കിൽ forward chaining ആണ് ഉചിതം.

Forward chaining suits problems where we know all the initial facts but have little idea what the conclusion will be. Backward chaining suits problems where we have a specific hypothesis to check. IGNOU states the choice in terms of **fan-out** (how many conclusions the facts can lead to) and **fan-in** (how many input paths or questions lead to the conclusion we want).

മലയാളത്തിൽ

Facts-ൽ നിന്ന് ഒരുപാട് conclusions-ലേക്ക് വഴി പിരിയുന്നുണ്ടെങ്കിൽ (fan-out കൂടുതൽ), forward പോയാൽ ആവശ്യമില്ലാത്ത കറേ കാര്യങ്ങൾ കണ്ടുപിടിക്കേണ്ടി വരും; അപ്പോൾ goal-ൽ നിന്ന് പിന്നോട്ട് (backward) പോകുന്നതാണ് നല്ലത്. മറിച്ച്, ഒരു conclusion-ലേക്ക് ഒരുപാട് വഴികൾ വന്നുചേരുന്നുണ്ടെങ്കിൽ (fan-in കൂടുതൽ), goal-ൽ നിന്ന് പിന്നോട്ട് പോയാൽ ഒരുപാട് ചോദ്യങ്ങൾ വരും; അപ്പോൾ forward ആണ് നല്ലത്.

Fan-out > fan-in (facts lead to many conclusions, few paths to our conclusion) $\Rightarrow$ backward chaining

Fan-in > fan-out (many paths to our conclusion, facts lead to few conclusions) $\Rightarrow$ forward chaining

Fan-in $\approx$ fan-out, few facts available, want to check if one of many possible conclusions is true $\Rightarrow$ backward chaining

All facts known, conclusion unknown $\Rightarrow$ forward; specific hypothesis to test $\Rightarrow$ backward

Point	Forward chaining	Backward chaining
Starts from	Initial facts	Goal / hypothesis
Driven by	Data	Goal
Rule matched on	IF part (antecedent)	THEN part (consequent)
Keeps	An updated working memory	A list of goals still to prove
Weakness	May derive many irrelevant facts	Needs a hypothesis to start
Best when	Fan-in high; all facts known, answer unknown	Fan-out high; specific hypothesis to test
Example	Monitoring, "what follows from these readings?"	Prolog, diagnosis "is it disease D?"

EXAMPLE

A patient's symptoms (fever, headache) are consistent with 40 different diseases, but each disease has only a few telltale tests. The doctor suspects typhoid. Which chaining?

The facts fan out to many conclusions, while the path to "typhoid" is narrow. Fan-out > fan-in, and we have a specific hypothesis, so **backward chaining**: start from typhoid and check only its conditions.

Conflict resolution

Definition: Conflict resolution is the strategy for choosing which rule to fire when the conflict set contains more than one applicable rule (e.g. refraction, recency, specificity).

Conflict set-ൽ ഒന്നിലധികം applicable rules ഉള്ളപ്പോൾ ഏത് rule fire ചെയ്യണമെന്ന് തിരഞ്ഞെടുക്കാനുള്ള strategy ആണ് conflict resolution (ഉദാ. refraction, recency, specificity).

When more than one rule is applicable (the conflict set has several rules), the interpreter must choose which one to fire. IGNOU lists three strategies. (1) **Refraction**: don't fire a rule twice on the same data. (2) **Recency**: fire rules that use more recent WM elements before older ones, so the system follows one chain of reasoning instead of drawing new conclusions from old data. (3) **Specificity**: fire rules with more specific preconditions before more general ones, so special (non-standard) cases are handled. Even so, the most important thing is to write rules whose preconditions say as precisely as possible when they should fire.

മലയാളത്തിൽ

ഒരേ സമയം രണ്ടോ മൂന്നോ rules fire ചെയ്യാൻ തയ്യാറാണെങ്കിൽ ഏത് ആദ്യം? ഒരേ data-യിൽ ഒരു rule വീണ്ടും വീണ്ടും fire ചെയ്യരുത്; ഏറ്റവും പുതിയ fact ഉപയോഗിക്കുന്ന rule-ന് മുൻഗണന; കൂടുതൽ conditions ഉള്ള, കൂടുതൽ specific ആയ rule-ന് മുൻഗണന. ഉദാഹരണത്തിന് "പനി ഉണ്ടെങ്കിൽ വിശ്രമിക്കുക" എന്നതിനേക്കാൾ "പനിയും ശ്വാസംമുട്ടലും ഉണ്ടെങ്കിൽ ആശുപത്രിയിലേക്ക്" എന്ന rule ആദ്യം പരിഗണിക്കണം.

Conflict set: all rules currently applicable

1. Refraction: never fire a rule twice on the same data
 2. Recency: prefer rules matching the most recently added WM elements
 3. Specificity: prefer rules with more (more specific) conditions
- Consistency problem: two rules with similar conditions but conflicting actions

EXAMPLE 1: SPECIFICITY, KERALA CLINIC

Rules D1 to D5 as before. Appu's WM: (fever appu), (cough appu), (breathing-difficulty appu).

Cycle	Conflict set	Lowest-numbered rule first	Specificity first
1	D1	D1: + respiratory-infection	D1: + respiratory-infection
2	D2, D3	D2: + advise-rest	D3 (2 conditions > 1): + refer-to-hospital
3	the other one	D3: + refer-to-hospital	D2: + advise-rest

If the system stops at its first piece of advice, rule order says "rest at home" to a patient who cannot breathe properly. Specificity gets the special case right: "refer to hospital".

EXAMPLE 2: RECENCY ON THE MONDAY EXAMPLE

Same rules (with corrected R5) and initial WM. Cycle 1 conflict set {R1, R2}: both use (day monday), so they tie; fire R1, adding (working-with ram). Cycle 2 conflict set {R2, R6}: R6 uses (working-with ram), which is newer than (day monday), so **recency picks R6**, which deletes (does-not-like ram).

Cycle	Conflict set	Fires (recency)	Change
1	R1, R2	R1	+ (working-with ram)
2	R2, R6	R6	- (does-not-like ram)
3	R2	R2	+ (talking-to ram)
4	R3	R3	+ (busy-at-work ram)
5	R4	R4	+ (not-at-home ram)
6	R5	R5	- (at-home ram)

Final WM: (day monday), (working-with ram), (talking-to ram), (busy-at-work ram), (not-at-home ram): the same set as with rule order, reached in a different sequence. Here no DELETE removes a fact that another rule still needs, so the end result is the same; in the tea example of the previous topic, it is not.

Exam tip: Section 6.2.3 repeats the whole Monday example word for word from 6.2.1. For a question on conflict resolution, list the three strategies with one line of reason each and show one conflict set where two strategies pick different rules.

Semantic nets

Semantic network: A graph representation of knowledge in which nodes represent concepts, objects or entities and labelled directed arcs represent binary relations between them.

Semantic network: Nodes concepts, objects അല്ലെങ്കിൽ entities-നെയും labelled directed arcs അവ തമ്മിലുള്ള binary relations-നെയും സൂചിപ്പിക്കുന്നു, knowledge-ന്റെ ഒരു graph representation.

Inheritance: Inferring properties of a node from its class and superclasses through instance and subclass links.

Inheritance: Instance, subclass links വഴി ഒരു node-ന്റെ properties അതിന്റെ class-ൽ നിന്നും superclasses-ൽ നിന്നും infer ചെയ്യൽ.

A **semantic network** is a graph: **nodes** represent concepts, objects or entities, and **labelled directed arcs** represent binary relations between them. It is a pictorial, structured representation in which related knowledge

is clustered together, so the knowledge needed for a task lies close together in the net, somewhat like human memory. It maps naturally to and from natural language. The two most important relations are **subclass** (class to superclass) and **instance** (object to its class); others are has-part, colour, etc. Subclass and instance links allow **inheritance**: new facts are inferred from the explicitly stored ones.

മലയാളത്തിൽ

Semantic net എന്നാൽ ഒരു ചിത്രം: കാര്യങ്ങൾ node-കൾ, അവ തമ്മിലുള്ള ബന്ധം പേരെഴുതിയ അമ്പുകൾ. ഒരു വാക്യത്തിലെ event-നെ നടുവിൽ വെച്ച്, ആർ ചെയ്തു (agent), ആരോട് (object), എന്തുകൊണ്ട് (instrument), എവിടെ (place), എപ്പോൾ (time) എന്ന് അമ്പുകളായി വരയ്ക്കുന്നു. "Raja ഒരു Lion ആണ്, Lion ഒരു Mammal ആണ്, Mammal warm-blooded ആണ്" എന്ന് links ഉണ്ടെങ്കിൽ "Raja warm-blooded ആണ്" എന്ന് എഴുതാതെ തന്നെ inheritance വഴി കിട്ടും. ചിത്രമായതിനാൽ മനുഷ്യർക്ക് മനസ്സിലാക്കാൻ എളുപ്പമാണ്, അതാണ് ഇതിന്റെ ജനപ്രീതിക്ക് കാരണം.

Semantic net = directed graph; nodes = concepts/objects; labelled arcs = binary relations

Key relations: subclass (class → superclass), instance (object → class)

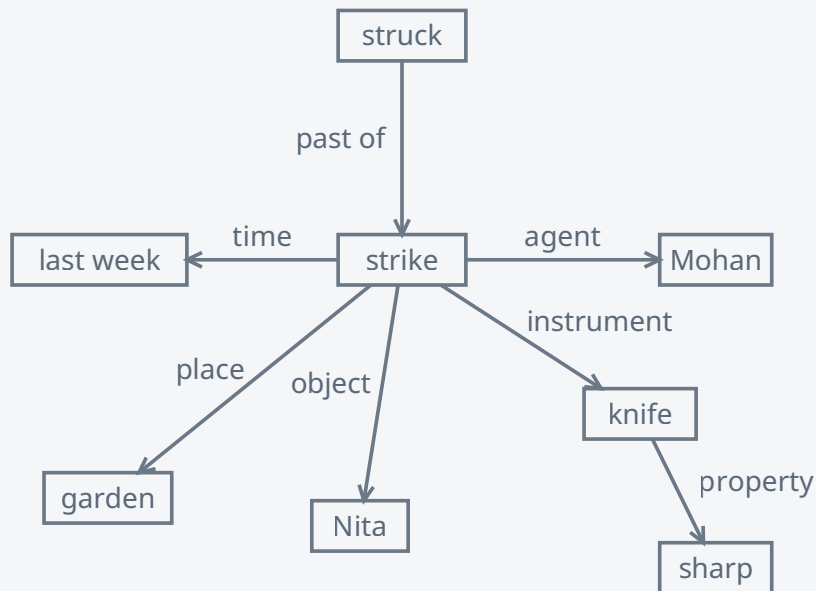
Other relations: has-part, colour, agent, object, instrument, place, time, ...

Inheritance: a node gets the properties of its class and superclasses

EXAMPLE 1 (FROM THE UNIT, FIGURE 6.1)

Represent: Mohan struck Nita in the garden with a sharp knife last week.

Make the event "strike" the central node and hang each part of the sentence on it by a case relation.

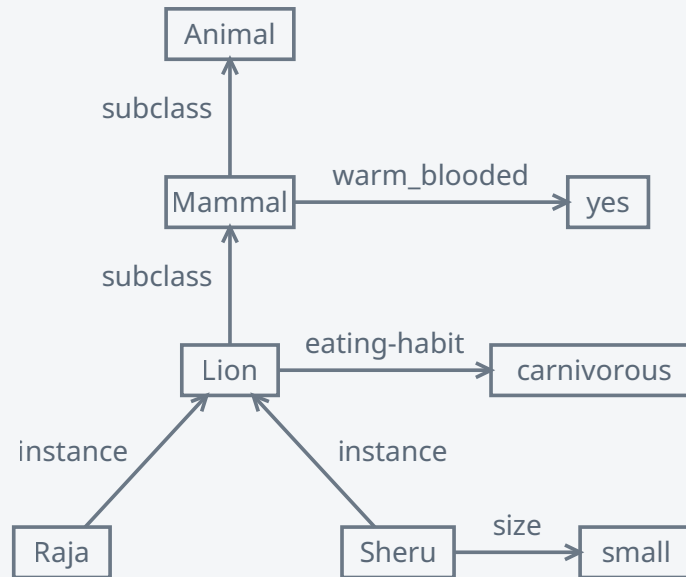


Arcs start at the event node strike; "struck" is the past of strike, and "sharp" is a property of the knife.

From	Arc	To
struck	past of	strike
strike	agent	Mohan
strike	object	Nita
strike	instrument	knife
strike	place	garden
strike	time	last week
knife	property	sharp

EXAMPLE 2: INHERITANCE

Using the lion knowledge of the next topic as a net, is Raja warm-blooded? What does Raja eat?



Subclass and instance links. Follow arcs upward to inherit properties.

Raja –instance → Lion –subclass → Mammal, and Mammal –warm_blooded → yes. So Raja is warm-blooded. Raja –instance → Lion –eating-habit → carnivorous, so Raja is carnivorous. Neither fact is stored at the Raja node.

Exam tip: In a sentence net, always make the verb (event) a node and put the case labels (agent, object, instrument, place, time) on the arcs. Give each separate event its own node; do not reuse one "strike" node for two different strikings.

Frames

Definition: A frame is a record-like structure that stores all knowledge about one concept as slots (attributes) with values, which may include defaults, pointers to other frames and attached procedures.

ഒരു concept-നെക്കുറിച്ചുള്ള എല്ലാ knowledge-ഉം slots (attributes)-ഉം അവയുടെ values-ഉം ആയി സൂക്ഷിക്കുന്ന record പോലുള്ള structure ആണ് frame; values-ൽ default values, മറ്റ് frames-ലേക്കുള്ള pointers, attached procedures എന്നിവയും ഉണ്ടാകാം.

A **frame** is a variant of the semantic net in which all information about one concept is stored in a single complex entity, like a **record** data structure. A frame has **slots** (attributes) filled with **values**. A slot can also hold default values, conditions for filling it, pointers to other frames, and procedures that run when needed (e.g. to compute a value). Frames support **inheritance** via subclass and instance slots, and represent typical objects or events (a car, a rectangle). Other names for frame-like structures: schema, script, prototype, object. Frame languages usually use LISP as the host language; object-oriented languages can also represent frames.

Inheritance is simple when each object has a single parent and each slot a single value. With **multiple inheritance** (more than one parent class) we must decide which parent to inherit from, e.g. a lion may inherit from "wild animals" or "circus animals".

മലയാളത്തിൽ

Frame ഒരു form പോലെയാണ്: "പേര്, വയസ്സ്, വിലാസം" എന്നിങ്ങനെ കള്ളികൾ (slots), അതിൽ പൂരിപ്പിക്കുന്ന വിവരം (values). ഒരു slot-ൽ default value-ഓ, value കണക്കാക്കാനുള്ള procedure-ഓ ആകാം. "Lion" frame-ൽ size medium എന്നുണ്ടെങ്കിൽ Raja-യ്ക്കും അത് inherit ചെയ്യും; പക്ഷേ Sheru-ന്റെ frame-ൽ size small എന്ന് സ്വന്തമായി കൊടുത്താൽ അതാണ് ഉപയോഗിക്കുക. ഒന്നിലധികം parent ഉണ്ടെങ്കിൽ (ഉദാ: wild animal-ഉം circus animal-ഉം) ഏതിൽ നിന്ന് എടുക്കണം എന്നതാണ് multiple inheritance-ലെ പ്രശ്നം.

Frame = named record of slots; slot = (attribute, value)

A slot may hold: value, default value, filling conditions (restrictions), pointer to another frame, attached procedure

subclass slot: class → superclass; instance slot: object → class

Lookup: own slot value first; if absent, inherit from class, then superclass, ...

Multiple inheritance: more than one parent ⇒ must choose which parent to inherit from

EXAMPLE 1 (FROM THE UNIT)

Mammal:	subclass: Animal	warm_blooded: yes	
Lion:	subclass: Mammal	eating-habit: carnivorous	size: medium
Raja:	instance: Lion	colour: dull-yellow	owner: Amar Circus
Sheru:	instance: Lion	size: small	

Queries: (a) Raja's size? Raja has no size slot, so inherit from Lion: **medium**. (b) Sheru's size? Sheru's own slot says **small**, which overrides Lion's default. (c) Is Sheru warm-blooded? Sheru → Lion → Mammal: **yes**. (d) Raja's owner? Own slot: **Amar Circus**.

EXAMPLE 2: A KERALA TEMPLE ELEPHANT

Elephant:	subclass: Mammal	diet: herbivorous	legs: 4
Temple-elephant:	subclass: Elephant	decoration: nettipattam	
Kesavan:	instance: Temple-elephant	temple: Guruvayur	

Kesavan's decoration: nettipattam (from Temple-elephant). Diet: herbivorous (from Elephant). Warm-blooded: yes (from Mammal, two levels up). If Kesavan were also an instance of "Forest-elephant" with a different decoration, that would be a multiple-inheritance conflict.

Scripts and their components

Definition: A script is a structured, frame-like representation that describes a stereotyped sequence of events in a particular context, such as eating in a restaurant.

ഒരു restaurant-ൽ ഭക്ഷണം കഴിക്കൽ പോലെ, ഒരു പ്രത്യേക context-ലെ stereotyped sequence of events വിവരിക്കുന്ന, frame പോലുള്ള structured representation ആണ് script.

A **script** is a structured representation describing a **stereotyped sequence of events** in a particular context, such as going to the movies, eating in a restaurant, shopping in a supermarket or visiting an eye doctor. Scripts use a frame-like structure and are used in natural language understanding systems. They are useful because events tend to occur in known runs or patterns, events are causally related, there are entry conditions that allow an event to take place, and there are prerequisites for events.

മലയാളത്തിൽ

ഒരു കല്യാണ സദ്യയ്ക്ക് പോയാൽ എന്ത് നടക്കും എന്ന് നമുക്ക് മുൻകൂട്ടി അറിയാം: ഹാളിൽ കയറുന്നു, ഇരിക്കുന്നു, ഇല വെക്കുന്നു, ചോറും കറികളും വിളമ്പുന്നു, പായസം, പിന്നെ ഇല മടക്കി എടുന്നേൽക്കുന്നു. ഇങ്ങനെ സ്ഥിരമായി ഒരേ ക്രമത്തിൽ നടക്കുന്ന സംഭവങ്ങളുടെ ചട്ടക്കൂടാണ് script. "അവൻ സദ്യ കഴിഞ്ഞ് ഇല മടക്കി" എന്ന് കേട്ടാൽ അവൻ ഊണ് കഴിച്ചു എന്ന് പറയാതെ തന്നെ നമ്മൾ ഊഹിക്കുന്നു; script ഉപയോഗിച്ച് computer-നും അങ്ങനെ ചെയ്യാം.

Script components:

Entry conditions: must hold before the events can occur

Results: conditions true after the events have occurred

Props: slots for the objects involved

Roles: the participants (IGNOU: "the actions that the individual participants perform")

Track: a variation of the script; tracks may share components

Scenes: the sequence of events

EXAMPLE

Identify the components for "going to the movies".

Track: multiplex (another track: village theatre). Props: ticket, seat, screen, popcorn. Roles: viewer, ticket clerk, usher. Entry conditions: viewer wants to see a film, has money, the show is on. Results: viewer has less money, viewer has seen the film (and is entertained). Scenes: 1 entering and buying the ticket, 2 finding the seat, 3 watching, 4 exiting.

Exam tip: IGNOU's wording describes roles as "the actions that the individual participants perform", but in its own bank script the roles are people (customer, employee, cashier), and the actions appear in the scenes. Write roles as the participants, as the bank example does.

Primitive action symbols

Primitive actions: A small set of basic action symbols from Schank's conceptual dependency theory (e.g. ATRANS, PTRANS) to which many different verbs are mapped.

Primitive actions: പല verbs-ഉം map ചെയ്യപ്പെടുന്ന, Schank-ന്റെ conceptual dependency theory-യിൽ നിന്നുള്ള അടിസ്ഥാന action symbols-ന്റെ ഒരു ചെറിയ set (ഉദാ. ATRANS, PTRANS).

ATrans: Transfer of an abstract relationship such as possession or ownership (e.g. give).

ATrans: Possession അല്ലെങ്കിൽ ownership പോലുള്ള ഒരു abstract relationship-ന്റെ transfer (ഉദാ. give).

PTRANS: Transfer of the physical location of an object (e.g. go).

PTRANS: ഒരു object-ന്റെ physical location-ന്റെ transfer (ഉദാ. go).

Scene lines in a script are written with a small set of **primitive actions** (from Schank's conceptual dependency theory), so that many different verbs map to the same few meanings. For example, "give", "hand over" and "sell" all involve ATRANS (a transfer of possession).

മലയാളത്തിൽ

ഭാഷയിൽ ആയിരക്കണക്കിന് ക്രിയകളുണ്ട്, പക്ഷേ അർത്ഥത്തിൽ അവ കുറച്ച് അടിസ്ഥാന പ്രവൃത്തികളിൽ ഒതുങ്ങും. "കൊടുത്തു", "വിറ്റു" രണ്ടിലും ഉടമസ്ഥത മാറുന്നു: ATRANS. "പോയി", "വന്നു" രണ്ടിലും സ്ഥലം മാറുന്നു: PTRANS. "പറഞ്ഞു", "അറിയിച്ചു": വിവരം കൈമാറുന്നു, MTRANS. ഈ primitives ഉപയോഗിച്ചാണ് script-ലെ scenes എഴുതുന്നത്.

Symbol	Meaning	Example
ATrans	Transfer of an abstract relationship (possession, ownership)	give
PTRANS	Transfer of the physical location of an object	go
PROPEL	Apply physical force to an object	push
MOVE	Move a body part by its owner	kick
GRASP	Grab an object by an actor	hold
INGEST	Take an object into an animal's body	eat, drink
EXPEL	Expel something from an animal's body	cry (tears)
MTRANS	Transfer mental information	tell
MBUILD	Build new information mentally	decide
CONC	Conceptualize or think about an idea	think
SPEAK	Produce sound	say
ATTEND	Focus a sense organ	listen

EXAMPLE

Write primitives for: (a) Ravi gave a book to Anu. (b) Anu went to Kochi. (c) Anu told Ravi the news. (d) Ravi drank tea. (e) Anu kicked the ball.

(a) Ravi ATRANS book to Anu. (b) Anu PTRANS Anu to Kochi. (c) Anu MTRANS news to Ravi. (d) Ravi INGEST tea. (e) Anu MOVE foot, and the ball is PROPELled (Anu PROPEL ball).

Exam tip: ATRANS vs PTRANS is the favourite confusion. Ownership or possession changes $\Rightarrow$ ATRANS (give, buy, sell). Only the place changes $\Rightarrow$ PTRANS (go, walk, carry). Schank's standard list has 11 primitives; IGNOU's table also includes CONC.

Writing a script: examples, advantages, disadvantages

A script is written as a header (script name, track, props, roles, entry conditions, results) followed by numbered scenes, each a list of primitive actions. Its advantages: it can **predict events** that were not mentioned, and it builds a **single coherent interpretation** from a collection of observations. Its disadvantages: it is **less general than frames** and may not suit all kinds of knowledge.

മലയാളത്തിൽ

Script എഴുതുമ്പോൾ ആദ്യം header (പേര്, track, props, roles, entry conditions, results), പിന്നെ scenes ഓരോന്നായി primitives ഉപയോഗിച്ച്. പറയാത്ത കാര്യങ്ങൾ ഊഹിക്കാൻ കഴിയും എന്നതാണ് വലിയ ഗുണം. പക്ഷേ ഒരു പ്രത്യേക സാഹചര്യത്തിന് മാത്രമാണ് ഒരു script; എല്ലാ തരം അറിവും ഇങ്ങനെ എഴുതാൻ പറ്റില്ല.

Advantages: predicts unstated events; builds one coherent interpretation from many observations

Disadvantages: less general than frames; not suitable for all kinds of knowledge

EXAMPLE 1 (FROM THE UNIT): WITHDRAWING MONEY FROM A BANK

SCRIPT: Withdraw money TRACK: Bank
PROPS: Money, Counter, Form, Token
ROLES: P = Customer, E = Employee, C = Cashier
ENTRY: P has no or less money; the bank is open
RESULT: P has more money

Scene 1: Entering

P PTRANS P into the bank
P ATTEND eyes to E
P MOVE P to E

Scene 2: Filling the form

P MTRANS signal to E
E ATRANS form to P
P PROPEL form for writing
P ATRANS form to E
E ATRANS token to P

Scene 3: Withdrawing money

P ATTEND eyes to counter
P PTRANS P to queue at the counter
P ATRANS token to C
C ATRANS money to P

Scene 4: Exiting the bank

P PTRANS P out of the bank

Using this script, the sentence "Rahul went to the bank and came back with cash" lets the system infer that he filled a form, got a token, and the cashier gave him the money.

EXAMPLE 2: A KERALA WEDDING SADHYA

SCRIPT: Eating a sadhya TRACK: Wedding hall (another track: Onam at home)
PROPS: Banana leaf, Table, Rice, Curries, Payasam, Water
ROLES: G = Guest, S = Server, H = Host
ENTRY: G is invited; G is hungry; the sadhya is ready
RESULT: G is not hungry; G is happy; H has fed the guests

Scene 1: Entering

G PTRANS G into the dining hall
G ATTEND eyes to an empty seat
G PTRANS G to the seat

Scene 2: Serving

S PTRANS leaf to table in front of G
S ATRANS rice, curries to G

Scene 3: Eating

G INGEST rice and curries
S ATRANS payasam to G
G INGEST payasam

Scene 4: Exiting

G MOVE hand to fold the leaf
G SPEAK thanks to H
G PTRANS G out of the hall

Prediction: told only "Anu folded her leaf and left", the script lets the system answer "Did Anu eat?" with yes (scene 3 precedes scene 4), and "Was Anu hungry at the end?" with no (result).

Exam tip: The unit's Scene 2 is garbled ("P ATRANS form to P / ATRANS form to P"). The sensible reading, used above, is that P hands the filled form back to E (P ATRANS form to E) and E gives P a token (E ATRANS token to P); otherwise P has no token to give the cashier in Scene 3. Handing over the token is also written above as ATRANS, since possession changes; the unit writes PTRANS there.

Quick recap

- Expert system = domain knowledge + reasoning; knowledge-based system is a synonym; the builder is a knowledge engineer.
- A good KR scheme is expressive, supports inference, and has well-defined syntax and semantics.
- Rule-based system = rule base (IF-THEN) + facts (working memory) + interpreter.
- Rules are independent, so the system is modular; large rule bases are hard to check for redundancy and consistency.
- Uncertainty comes from vague domain theory and unreliable data; certainty factors (IGNOU: 0 to 1) express it.

- Forward chaining is data-driven: recognize-act cycle of match → conflict set → select → fire, until nothing fires or the goal appears.
- Backward chaining is goal-driven: match the goal with rule consequents, prove conditions as sub-goals, backtrack on failure; Prolog works this way.
- Fan-out > fan-in ⇒ backward chaining; fan-in > fan-out ⇒ forward chaining.
- Conflict resolution: refraction (not twice on same data), recency (newest facts first), specificity (more specific rules first).
- The order in which rules fire can change the final working memory, especially when rules delete facts.
- Semantic net: nodes = concepts, labelled directed arcs = binary relations; subclass and instance links give inheritance.
- Frame: record of slots with values, defaults, restrictions and attached procedures; own values override inherited ones; multiple inheritance needs a choice of parent.
- Script: stereotyped event sequence with entry conditions, results, props, roles, track and scenes.
- Scenes use primitives: ATRANS (possession), PTRANS (location), MTRANS (information), INGEST, PROPEL, MOVE, GRASP, EXPEL, MBUILD, SPEAK, ATTEND, CONC.

Check yourself

Q. 1. (CYP 1) In the "Animal Identifier System", use forward chaining to identify the animal raja.

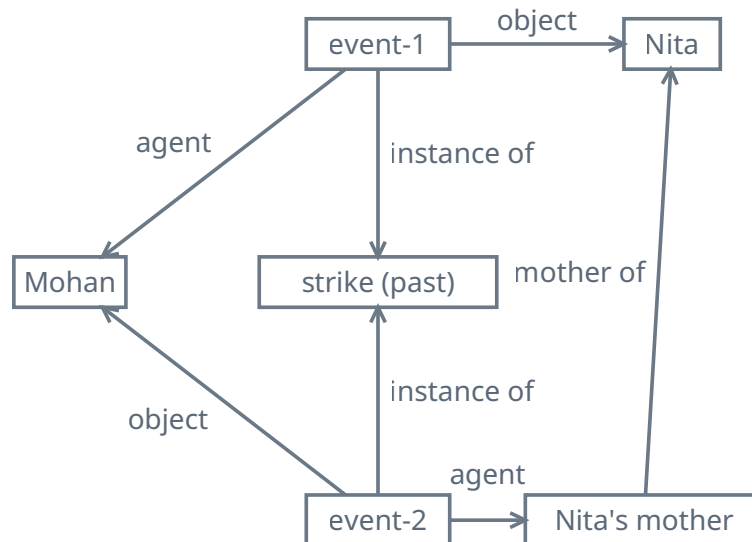
WM: (has-hair raja), (big-mouth raja), (long-pointed-teeth raja), (claws raja). Rules 1 to 10 as in the backward chaining topic. Strategy: fire the lowest-numbered applicable rule, never twice.

Cycle	Conflict set	Fires	Added
1	Rule 2 (has-hair)	2	(mammal raja)
2	Rule 4 (mammal, long-pointed-teeth, claws)	4	(carnivorous raja)
3	Rule 8 (carnivorous, big-mouth)	8	(lion raja)
4	none	stop	

Rule 1 needs gives-milk, rule 3 needs eats-meat, rule 5 needs does-not-eat-meat, rule 6 needs dark-spots: none are in WM, so they never apply. Final WM contains (lion raja): **raja is a lion**, the same answer backward chaining gave.

Q. 2. (CYP 2) Draw a semantic network for: Mohan struck Nita and Nita's mother struck Mohan.

There are two separate striking events, so use two event nodes, both instances of "strike" (past tense). Nita's mother is a new node linked to Nita by "mother of".



event-1: Mohan (agent) struck Nita (object). event-2: Nita's mother (agent) struck Mohan (object).

Arcs: event-1 –agent → Mohan, event-1 –object → Nita, event-2 –agent → Nita's mother, event-2 –object → Mohan, Nita's mother –mother of → Nita, event-1 and event-2 –instance of → strike (past).

Q. 3. (CYP 3) Define a frame for the entity date, with day, month and year (numbers with the well-known restrictions) and a procedure compute-day-of-week.

Date:

day: integer; restriction $1 \leq \text{day} \leq 31$, and
day $\leq \text{days-in}(\text{month}, \text{year})$ (30 for Apr, Jun, Sep, Nov;
28 for Feb, or 29 in a leap year)
month: integer; restriction $1 \leq \text{month} \leq 12$
year: integer
day-of-week: if-needed: compute-day-of-week(day, month, year)

Independence-day:

instance: Date
day: 15 month: 8 year: 1947
day-of-week: (not stored; computed on request) = Friday

The restrictions are slot-filling conditions, and day-of-week is an attached procedure run only when the value is asked for. 15 August 1947 was indeed a Friday.

Q. 4. Using the animal rules and raja's facts, can backward chaining prove that raja is a cheetah? Show the trace.

Goal cheetah(raja): only rule 6, needs carnivorous(raja) and dark-spots(raja).

carnivorous(raja): rule 3 needs mammal (proved by rule 2 after rule 1 fails on gives-milk) and eats-meat (fails), so rule 3 fails; rule 4 needs mammal (proved), long-pointed-teeth (fact), claws (fact), so carnivorous is proved.

dark-spots(raja): not a fact and no rule concludes it: fails. No other rule concludes cheetah, so the goal **fails**.

Q. 5. Give four differences between forward and backward chaining, and say when each is preferred.

(1) Forward starts from facts; backward from a goal. (2) Forward is data-driven; backward is goal-driven. (3) Forward matches rule antecedents with WM; backward matches rule consequents with goals. (4) Forward updates a working memory; backward keeps a list of goals to prove. Forward is preferred when all facts are known but the conclusion is not, or when fan-in > fan-out. Backward is preferred when there is a specific hypothesis, or when fan-out > fan-in (e.g. medical diagnosis, Prolog).

Q. 6. In the Monday example (with R5 corrected), after R1 fires the conflict set is {R2, R6}. Which rule fires under (a) lowest rule number, (b) recency? Does the final WM differ?

(a) R2 (lower number). (b) R6: it matches (working-with ram), added in cycle 1, while R2 matches only the older (day monday).

Both runs end with (day monday), (working-with ram), (talking-to ram), (busy-at-work ram), (not-at-home ram). The final WM is the same here because no deleted fact is needed by another rule; only the sequence differs.

Q. 7. List the three conflict resolution strategies. In the clinic rules, why is specificity the right choice between D2 and D3?

Refraction (don't fire a rule twice on the same data), recency (prefer rules using newer WM elements), specificity (prefer rules with more specific preconditions).

D2: respiratory-infection → advise-rest. D3: respiratory-infection ∧ breathing-difficulty → refer-to-hospital. Whenever D3 applies, D2 also applies. D3 is the special, non-standard case, so it should win: a patient with breathing difficulty must be referred, not told to rest.

Q. 8. List the components of a script and write a short script for buying a train ticket at a railway counter.

Components: entry conditions, results, props, roles, track, scenes.

```
SCRIPT: Buy train ticket    TRACK: Railway counter
PROPS: Money, Ticket, Counter, Reservation form
ROLES: P = Passenger, C = Clerk
ENTRY: P wants to travel; P has money; counter is open
RESULT: P has a ticket; P has less money
Scene 1: P PTRANS P to the queue at the counter
Scene 2: P MTRANS destination to C; P ATRANS money to C
Scene 3: C ATRANS ticket to P
Scene 4: P PTRANS P away from the counter
```

Q. 9. Using the lion frames (Mammal, Lion, Raja, Sheru), answer: Sheru's size, Raja's size, Raja's warm_blooded, and explain the problem of multiple inheritance.

Sheru's size = small (own slot overrides the Lion default). Raja's size = medium (inherited from Lion). Raja's warm_blooded = yes (Raja → Lion → Mammal).

Multiple inheritance: if Raja were an instance of both "wild animal" and "circus animal", and these gave different values for a slot (say, habitat: forest vs circus), the system must decide which parent to inherit from.

Q. 10. Write the primitive for each: (a) Meera sold her scooter to Vinod. (b) The boat carried the tourists to Alappuzha. (c) Teacher told the class the result. (d) The baby cried.

(a) Meera ATRANS scooter to Vinod (ownership changes; Vinod ATRANS money to Meera too). (b) Boat PTRANS tourists to Alappuzha (location changes). (c) Teacher MTRANS result to class. (d) Baby EXPEL tears.

Based on the IGNOU unit texts. Check definitions against the PDFs before the exam.